

V



ARQUITECTURA

GENERACIONES DE COMPUTADORAS

★ Primera Generación (1954-1959)

Las computadoras estaban constituidas por **válvulas electrónicas de vacío**, que disipaban gran cantidad de calor y ocupaban una amplia superficie.

- Ejecutaban miles de instrucciones por segundo.
- Almacenaban de 10.000 a 20.000 bytes en su memoria.
- Los medios de entrada/salida eran tarjetas o cintas perforadas.
- La memoria principal tenía un tiempo de acceso muy grande en comparación con los tiempos de procesamiento de la UCP.
- Se usaban cintas magnéticas como memoria secundaria.



Las tareas se ejecutaban de manera puramente **secuencial**, cada una en tres tiempos:

1. El programa, almacenado en tarjetas o cintas perforadas, era cargado en la memoria principal por un programa llamado cargador.
2. Se ejecutaba el programa instrucción por instrucción
3. Se imprimían los resultados. Las operaciones de entrada, procesamiento y salida de los datos estaban encadenadas en el tiempo, por lo que la duración de todo el proceso era igual a la suma de todas y cada una de las operaciones.

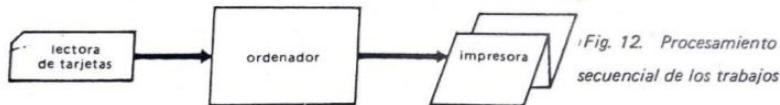


Fig. 12. Procesamiento secuencial de los trabajos

★ Segunda Generación (1959-1964)

Las computadoras de la 2da generación estaban constituidas por **transistores**, más confiables, de menor tamaño, menor disipación de calor y más rápidos que las válvulas de vacío para cambiar de estado.

- Velocidades de procesamiento de centenares de miles de instrucciones por segundo.
- Simultaneidad entre un cálculo y una operación de E/S, aunque el encadenamiento de los trabajos seguía siendo secuencial.
- Desproporción entre las velocidades de cálculo interno y las velocidades de E/S (lectura de tarjetas o impresión)

Esta desproporción hacía que la CPU no se usara más que un pequeño porcentaje de tiempo. La solución a esto fue que las operaciones de E/S se realizaran usando como soporte de almacenamiento unidades de cinta magnética, mucho más rápidas que las lectoras de tarjetas y las impresoras. Para lograr esto, se copiaba la información contenida en el soporte tarjeta a soporte cinta magnética y de cinta magnética a impresora, usando para esto una computadora auxiliar. La computadora principal no conocía ni operaba más que con las cintas magnéticas. Esto se llamó **procesamiento por lotes**, ya que era necesario esperar a que el lote de trabajos cargados en la cinta magnética se procese por completo antes de poder obtener los resultados de cualquiera de ellos.

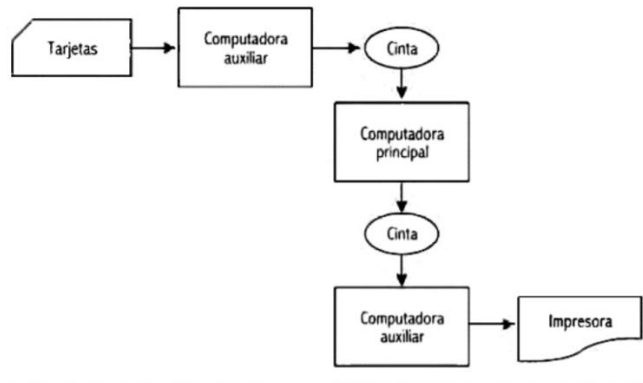


Fig. 1.4. Procesamiento por lotes.

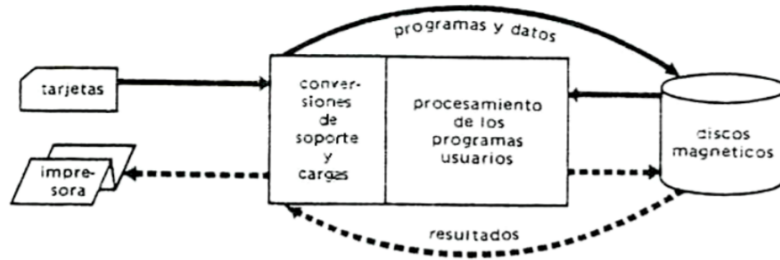
★ Tercera Generación (1964-1971)

Utilizaban tecnología de **circuitos integrados**, (miles de componentes electrónicos incluidos en una sola pastilla de silicio).

- Estos circuitos eran de pequeña y mediana escala de integración (**SSI y MSI**)
- Incremento de la velocidad interna de la computadora y la reducción de la energía que usaban.
- Permite multiprogramación, que consiste en que varios programas puedan residir simultáneamente en memoria.
- Dividen su memoria en 2 zonas: una reservada a los “trabajos de usuario” y la otra a la “conversión de soportes y carga”.

Se pueden alojar varios procesos en memoria pero en un instante dado solo uno de ellos utiliza la CPU, de igual manera los otros pueden efectuar operaciones de E/S en forma simultánea. Cuando el programa que utiliza la CPU se detiene en espera de una operación de E/S, otro programa toma su lugar dejando al primero suspendido y evitando así tiempos inactivos en la CPU. Mmmmmmmmwq,.zzzas1||67loooooo0000005,

Los trabajos de usuario es similar a la computadora principal y la conversión de soportes y carga a la computadora auxiliar de la generación anterior. Sin embargo, hay una diferencia importante: **la “carga por lotes” se sustituye por la “carga continua” de los trabajos a medida que se presentan**. Los trabajos son puestos en cola de espera en un disco magnético y el sistema operativo es el que se encarga de cargarlos en memoria para ejecutarlos según su nivel de prioridad. Los resultados son transferidos al disco, de donde serán extraídos más tarde a través de la impresora. No tenemos que esperar a que se procese todo un lote de programas para obtener el resultado de uno de ellos.



Surge la posibilidad de trabajar a distancia o teleprocesamiento, y la posibilidad de trabajar en forma conversacional.

A fin de atender un gran número de procesos o usuarios, una computadora puede asignar en forma alternada una parte del tiempo de CPU a cada uno, lo que producía un efecto de aparente simultaneidad de ejecución. Esta forma de organizar el procesamiento se denominó **tiempo compartido o time sharing**, y dio lugar a **sistemas multiusuario y multitarea**.

Por último, se desarrolla el concepto de **máquina virtual**, que simplifica la labor del programador, ya que éste no conoce más que una máquina ficticia, llamada máquina virtual, creada y controlada por el sistema operativo, que no presenta ni limitaciones de configuración (sobre todo en capacidad de memoria central) ni las limitaciones debidas a la actividad compartida de varios usuarios.

★ Cuarta Generación (1971-198..?)

Circuitos integrados pasaron a ser **LSI (Large Scale Integration)** con una mayor miniaturización y más circuitos dentro del chip (millones de transistores)

- Permitieron incluir una CPU completa en una sola pastilla, que se denominó **microprocesador**.
- El procesamiento se realiza en mayor medida en **tiempo real**.
- Los **sistemas interactivos**, hacen posible la consulta y actualización de datos, así como el acceso a grandes bancos de datos conectados en redes.

Debido a esto empiezan a aparecer las microcomputadoras, es decir las computadoras personales o PC hogareñas, surgiendo gran variedad de periféricos para las mismas y comunicándose entre sí a través de módems y redes globales.

Hasta esta etapa, los avances tecnológicos estuvieron concentrados en lograr mejorar el hardware de la computadora y así obtener equipos más pequeños, menos costosos y más rápidos. A partir de entonces, también se puso atención en la necesidad de mejorar el software para que permitiera una mayor velocidad de procesamiento, ya que los avances en el hardware parecían agotados.

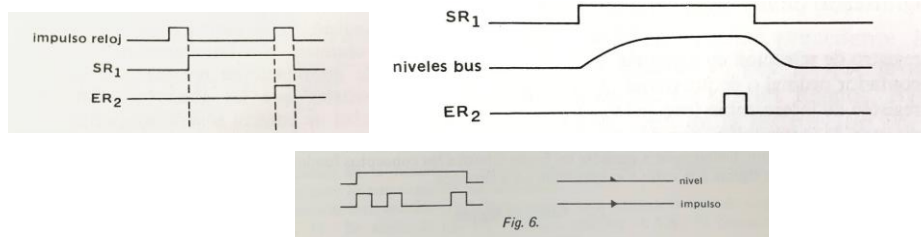
Generación	Periodo	Composición	Procesamiento
0	Hasta 1945	Sistemas mecánicos y electromecánicos	Programa no formaba parte de la máquina. No siguen instrucciones
1	1945-1955	Tubos al vacío	Resuelve una tarea por vez puramente secuencial
2	1955-1965	Transistores	Procesamiento por lotes
3	1965-1980	Circuitos integrados (con transistores)	Carga Continua. Multiprogramación
4	Desde 1980	Chip de silicio con una CPU entera (microprocesadores) PC.	Procesamiento en Tiempo Real

CIRCUITOS LOGICOS DIGITALES

★ SEÑALES LOGICAS

★ **Señal de Nivel Φ** – tiempo entre que termina un impulso y termina el siguiente impulso – Todas las señales que llegan a un bus (salida de un registro S-algo) son de nivel

★ **Señal Impulsional θ** - coincide con el batido de clock - Señales que salen de un bus a un registro (las de entrada EN-algo) son impulsionales



ALGEBRA DE BOOLE

Boole definió hacia 1850 un algebra aplicable a razonamientos sobre proposiciones lógicas: una proposición solo puede ser falsa(0) o verdadera(1). Shannon, en 1938, aplico este algebra al análisis de circuitos digitales. La corriente pasa o no.

Variable lógica: se denota como A, B, C, etc. Y puede tomar el valor 1(verdadero) o 0(falso). Una variable binaria representa a un bit.

FUNCIONES Y FORMAS CANONICAS

★ **Minitermino:** es un producto booleano en el que cada una de las n variables aparece una sola vez (negada o sin negar).

★ **Maxitermino:** es una suma booleana en el que cada una de las n variables aparece una sola vez (negada o sin negar).

A	B	Miniterminos
0	0	$\bar{A} \cdot \bar{B}$
0	1	$\bar{A} \cdot B$
1	0	$A \cdot \bar{B}$
1	1	$A \cdot B$

A	B	Maxiterminos
0	0	$A + B$
0	1	$A + B$
1	0	$A + B$
1	1	$A + B$

Minitermino o Producto Canónico: Es el producto de las variables en juego o sus negaciones individuales que hacen que el producto valga uno.

Maxitermino o Suma Canónica: Es la suma de las variables en juego o sus negaciones individuales, que hacen que la suma valga cero.

Formas Normales: Formas normalizadas de escribir formulas en lógica proposicional

★ **Forma normal conjuntiva:** Conjunción de disyunciones de literales. *Producto de sumas*. para hallarla, de las filas donde la función es verdadera, se toma el minitermino correspondiente

★ **Forma normal disyuntiva:** Disyunción de conjunciones de literales. *Suma de productos*. para hallarla, de las filas donde la función es falsa, se toma el maxitermino correspondiente.

A	B	C	Maxiterminos	Miniterminos	Salida
0	0	0	$A + B + C$	$\bar{A} \cdot \bar{B} \cdot \bar{C}$	1
0	0	1	$A + B + \bar{C}$	$\bar{A} \cdot \bar{B} \cdot C$	0
0	1	0	$A + \bar{B} + C$	$\bar{A} \cdot B \cdot \bar{C}$	0
0	1	1	$A + \bar{B} + \bar{C}$	$\bar{A} \cdot B \cdot C$	1
1	0	0	$\bar{A} + B + C$	$A \cdot \bar{B} \cdot \bar{C}$	0
1	0	1	$\bar{A} + B + \bar{C}$	$A \cdot \bar{B} \cdot C$	1
1	1	0	$\bar{A} + \bar{B} + C$	$A \cdot B \cdot \bar{C}$	1
1	1	1	$\bar{A} + \bar{B} + \bar{C}$	$A \cdot B \cdot C$	0

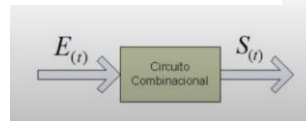
$$FND = \bar{A} \cdot \bar{B} \cdot \bar{C} + \bar{A} \cdot B \cdot C + A \cdot \bar{B} \cdot C + A \cdot B \cdot \bar{C}$$

$$FNC = (A + B + \bar{C}) \cdot (A + \bar{B} + C) \cdot (\bar{A} + B + C) \cdot (\bar{A} + \bar{B} + \bar{C})$$

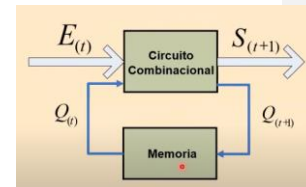
CIRCUITOS COMBINACIONALES Y SECUENCIALES

★ **Circuito Lógico:** Circuito que trata señales lógicas

★ **Combinacional:** Señales de salida no dependen mas que de las entradas en el mismo instante de tiempo, **no dependen de las salidas ni el tiempo**. Basada en Algebra de Boole. Son determinísticos, ante una combinación dada de entradas devuelve la misma salida. Están formados por combinaciones de puertas lógicas que propagan los valores de verdad a través del circuito



★ **Secuencial:** Salida depende de las entradas y del tiempo. Tiene elementos de retardo o de memoria. Basada en Teoría de Autómatas.



CIRCUITOS SECUENCIALES

Autómata Finito

Es un sistema que almacena un conjunto de estados finitos, de manera que su salida es una función de los estímulos externos y de los estados por los cuales debe pasar.

Los autómatas son entes matemáticos, cuyo estudio permite dar un soporte teórico a numerosos desarrollos relacionados con la informática.

Aquí se restringe el estudio del autómata a un ángulo externo tratándolo como una caja negra con entradas y salidas. Siendo:

E el conjunto de **entradas**

S el conjunto de salidas

Instantes sucesivos discretos $t, t+1, t+2, \dots$

Suponemos que una entrada $E(t)$ en el instante t produce una respuesta $S(t+1)$ en el instante $t+1$. Dicha respuesta depende de $E(t)$ y a su vez también de la función tiempo $H(t)$.

La función $H(t)$ está completamente determinada por la historia previa del autómata.

El autómata está constituido por un número finito de elementos, y por consiguiente un número finito de elementos de memoria. Y por lo tanto **$H(t)$ no puede tomar más que un número finito de valores distintos llamados ESTADOS INTERNOS DEL AUTOMATA.**

Funciones de Transición: Si al estado del autómata en el instante t lo denominamos $Q(t)$ entonces:

$$S(t + 1) = F[Q(t), E(t)]$$

$$Q(t + 1) = G[Q(t), E(t)]$$

Con estas 2 ecuaciones podemos calcular paso a paso el comportamiento del autómata cualquiera sea la sucesión de estímulos.

DIAGRAMA DE TRANSICION

Grafo orientado que permite ver como se produce cada cambio de estado en función a una entrada y una posible salida. Flechas indican transiciones entre estados indicando primero la entrada y luego salida.

Por ejemplo, la flecha superior del diagrama significa que si el autómata esta en el estado 0 ante una entrada 0 produce una salida 0 y regresa al estado 0

ELEMENTOS DE MEMORIA: BIESTABLE (Flip-Flops)

Es un sistema o circuito lógico con dos estados estables. Esta constituido fundamentalmente por 2 circuitos NOT montados en oposición. No puede haber más de dos estados lógicos diferentes que, en ausencia de estímulos exteriores, no tienen por qué cambiar. Por esta razón este circuito funciona como elemento de memoria, almacenando en cada uno 1 bit.

★ **Síncronos:** sus entradas modifican o no el estado de este, dependiendo de una señal de sincronización.

★ **Asíncronos:** sus entradas no dependen de una señal de sincronización para modificar el estado.

Entradas al FF → **señal de nivel**

Salidas del FF → señal impulsional

Tabla de estado o funcionamiento:

Flip-flop RS:

Construido con compuertas NOR o NAND

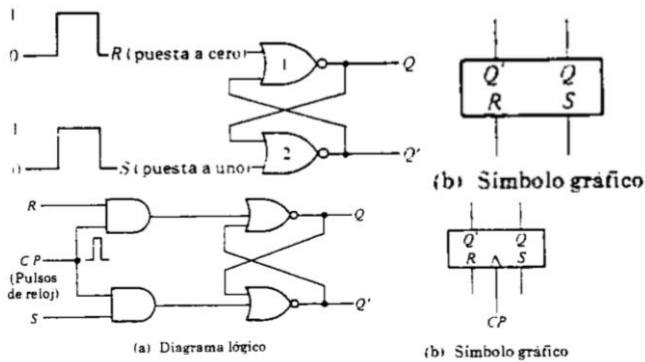


Tabla de funcionamiento

R	S	Q_t	Q_{t+1}
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	?
1	1	1	?

Tabla de excitación

Q_t	Q_{t+1}	R	S
0	0	X	0
0	1	0	1
1	0	1	0
1	1	0	X

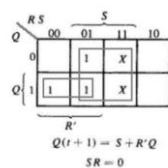
Funciones:

A partir de las funciones que nos dicen como será el estado siguiente ante una cierta combinación de entradas y estado actual ("Q' = ") se pueden construir las tablas de funcionamiento o tablas de estado.

$$Q' = Q(t+1)$$

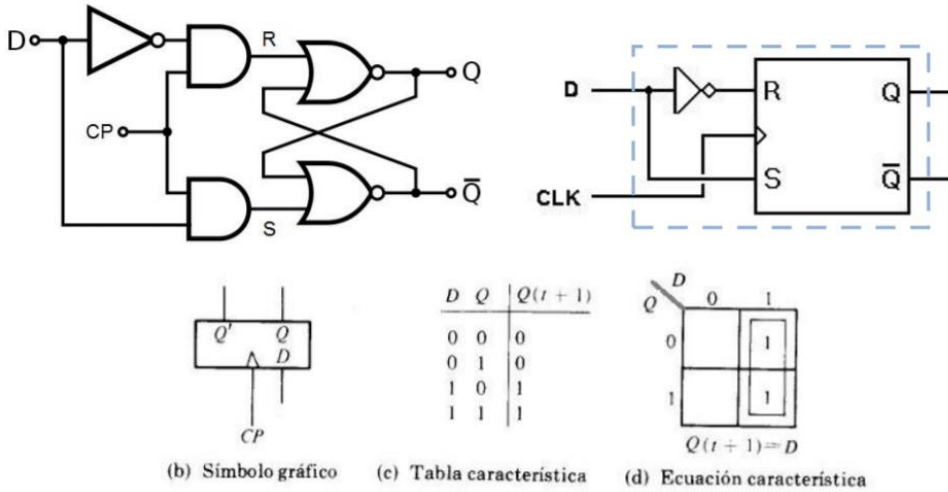
$$Q = Q(t)$$

$$Q' = (S + Q) \cdot \bar{R}$$



Flip-flop D (Data o Delay) → D = 0 RESET | D = 1 SET

- Modificación de RS.
- Une las entradas SET y RESET evitando la indeterminación en 1-1. → Almacenar un 0 o un 1 en un RS implica tener dos entradas diferentes (una en 0 y la otra en 1); en un FF D solo una entrada es suficiente para almacenar el valor deseado.
- Solo síncrono → evita reset espontáneo

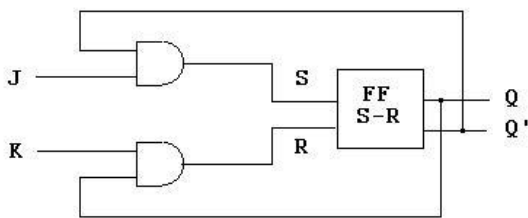


Flip-flop JK:

- Modificación RS
- Resuelve indeterminación en 1-1 $\rightarrow$ complementando $Q(t)$
- Puede ser síncrono o asíncrono

A partir de función de RS $\rightarrow S = J\bar{Q} \mid R = KQ$

$$Q' = (J\bar{Q}) + (\bar{K}Q)$$



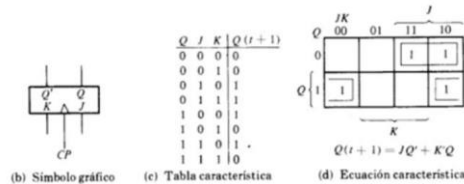
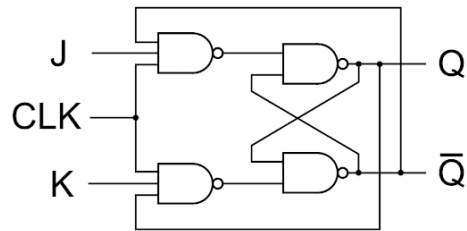


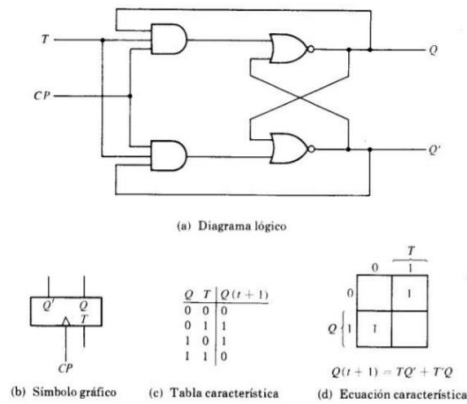
Figura 6-6 Flip-flop JK temporizado

El funcionamiento es el siguiente:

- ★ La salida Q se aplica junto con K y CP a una compuerta AND de manera que el flip-flop se ponga a cero durante un pulso de reloj solamente si Q fue 1 previamente.
- ★ De manera similar la salida Q se aplica junto con J y CP a una compuerta AND de manera que el flipflop se ponga a uno con un pulso de reloj, solamente si Q fue 0 previamente.
- ★ Cuando $J = 1$ y $K = 1$, el pulso de reloj activa solo una de las compuertas AND, aquella conectada a la salida igual a 1. Entonces si $Q = 1$, la salida de la AND superior se convertirá en 1 cuando se aplique el pulso de reloj, y el flip-flop se pondrá a uno. Si $Q = 0$, la salida de la AND inferior se convierte en 1 poniendo a uno al flip-flop. En cualquier caso, el estado se complementa.

Flip-flop T (Toggle - Conmutar) :

- Modificación de JK.
- Une las entradas J y K.
- Puede ser sincrónico o asíncrono.
- Si entra 0 $\rightarrow$ mantiene Q. Si entra 1 $\rightarrow$ complementa Q.



Monoestable

Sistema de dos estados con **solo una posición estable**. Bascula bajo el efecto de un impulso de entrada y retorna a su posición de equilibrio.

El estado estable es el estado CERO. Bajo un impulso, pasa a UNO. El temporizador θ al cabo de un tiempo produce un RESET.

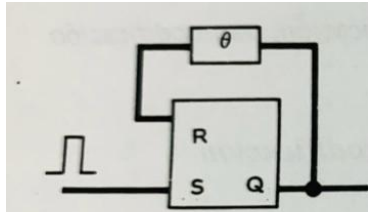


Fig. 47. Monoestable

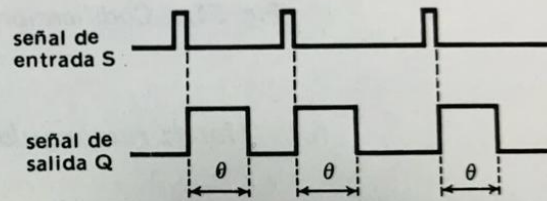


Fig. 48. Cronograma del monoestable

Maestro – Esclavo

Se construye con dos flip-flops separados

Consiste en un flip-flop maestro, un flip-flop esclavo y un inversor.

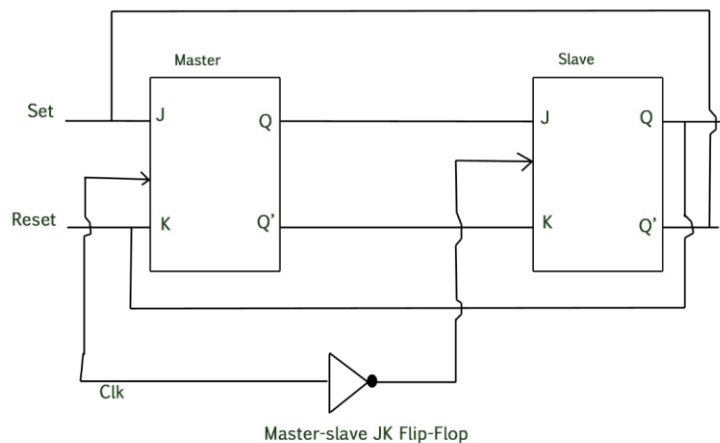
Cuando el pulso del reloj es 0, la salida del inversor es 1.

Cuando el maestro está habilitado el esclavo está deshabilitado, y viceversa.

Cuando la entrada de reloj del esclavo es 1 (el reloj del esclavo es el clock negado), el esclavo se habilita (mientras el maestro está deshabilitado) y la salida Q es igual a la salida Y, y la salida Q' es igual a la salida Y'.

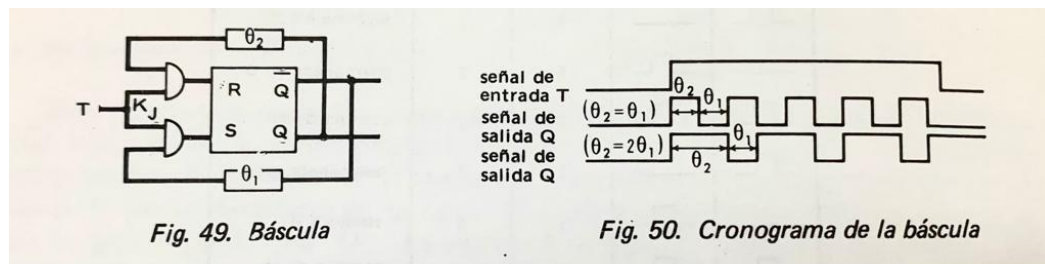
Cuando el reloj está en 1, se habilita el maestro y se deshabilita el esclavo. La información de las entradas R y S se transmiten al flip-flop maestro.

Cuando el pulso regresa a 0, el flip-flop maestro se aísla lo cual previene que las entradas externas lo afecten. Y el flip-flop esclavo irá al mismo estado que el maestro.



Báscula (cambia de estado)

Sistema de dos estados con ninguno de ellos estable.

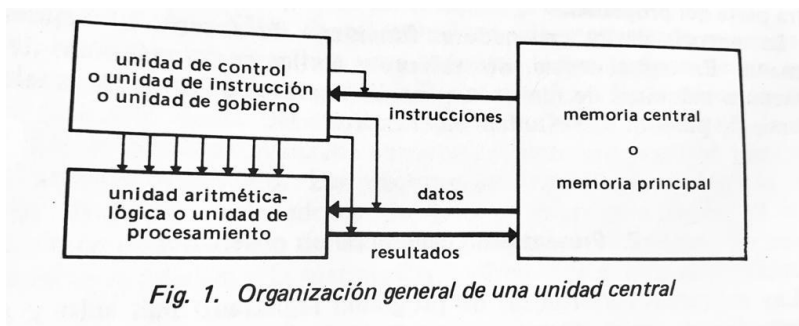


Puede realizarse uniendo las dos entradas de un JK. Mientras se mantenga a 1 la entrada T, la báscula cambia constantemente de estado. Pueden utilizarse las básculas para realizar relojes que dan un tren de impulsos regularmente espaciados.

INTRODUCCION A LA ARQUITECTURA VON NEUMANN

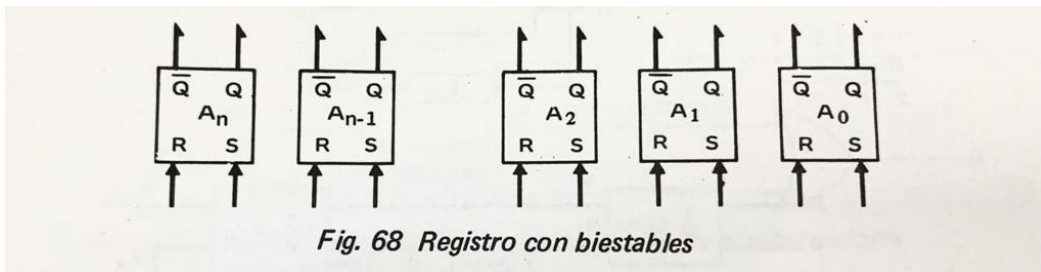
Componentes principales:

- UCP (Unidad Central de Proceso) – UC Y ALU – Procesamiento de datos e instrucciones
- MEMORIA CENTRAL (de programas y datos)
- UNIDADES E/S -Canales y Periféricos-



REGISTROS

- Elementos de memoria de almacenamiento temporal de información, instrucciones y datos.
- Compuesto por varios biestables.
- Permite almacenar información de varios dígitos.



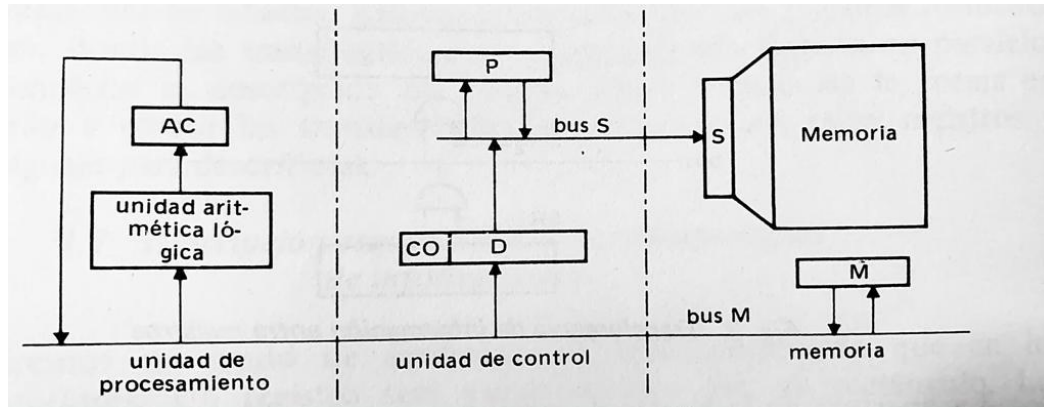
Operaciones elementales sobre los registros (con biestables RS)

1. Un registro puede ponerse a 0 enviando un pulso por la entrada R de cada biestable.
2. El contenido de un registro puede complementarse.
3. El contenido de un registro puede desplazarse a izquierda o a derecha.
4. El contenido de un registro puede incrementarse o decrementarse.

ARQUITECTURA ABACUS (Meinadier)

Esta diseñada como una maquina sincrónica, con un registro de instrucción de una dirección (código de operación y dirección de operando) en tal sentido que los cálculos se realizan contra el valor almacenado en el registro acumulador de la ALU (Arithmetic Logic Unit).

El reloj o *clock* que sincroniza a todos sus componentes, produce un batido por cada θ segundos, siendo un ciclo de memoria 2θ .



Esquema simplificado de ABACUS

Unidades de ABACUS:

★ **ALU – Unidad Aritmética Lógica:** Es la unidad de la máquina que se encarga de realizar las operaciones aritmético-lógicas, y almacenar información temporalmente en un registro acumulador (resultados de los cálculos u operandos).

★ **UC – Unidad de Control:** Lleva el conteo de las próximas instrucciones y lleva el registro de instrucción el cual es decodificado para así ver que operación se quiere realizar

★ **MC - Memoria Central:** Es la unidad encargada del almacenamiento y recuperación de la información e instrucciones.

Estos órganos se comunican a través de registros y buses:

Posee dos Buses:

★ **BUS M:** Transporta Datos.

★ **BUS S:** Transporta Instrucciones.

Por razones meramente académicas dividiremos la arquitectura en tres secciones: La unidad de Procesamiento la cual contiene la ALU, la unidad de Control y la Memoria Central. Las mismas están interconectadas por los Buses M y S.

Al bus M le llega una instrucción que proviene de la Memoria Central y la lleva al registro de instrucciones para ser decodificada. En caso de que esta deba buscar un operando, el campo DIR proveerá al Bus S la ubicación en memoria del mismo, el cual será leído y llevada a la ALU por el Bus M.

Los registros que la componen son los siguientes:

S:	registro de selección en memoria (registro especial de la memoria que almacena la dirección de memoria de una instrucción o un operando proveniente del bus S)	<i>m</i> bits
P:	contador ordinal o de programa (lleva un conteo de la ubicación de la instrucción actual)	<i>m</i> bits
M:	registro de intercambio (reg. Palabra de memoria) (Almacena una instrucción u operando. se utiliza para cargar y obtener palabras almacenadas en la memoria)	<i>n</i> bits
I:	registro de instrucción, con dos partes: CO: código de operación	<i>p</i> bits
	D: dirección	<i>m</i> bits
AC:	Registro acumulador. Donde se almacenan operandos y el resultado de una operacion	<i>n</i> bits

Como vimos un ciclo de memoria esta compuesto por 2θ, y además toda arquitectura atraviesa por:

- ★ Fase búsqueda de instrucción
- ★ -posible- búsqueda de operando
- ★ Ejecución de la operación
- ★ Preparación para la próxima instrucción

Interrupción de este desarrollo ideal:

1. IT (interrupción) → teclado, mouse
 - a. CPU debe tratarlo prioritariamente
2. petición de ciclo de memoria E/S
 - a. PCMES (Petición de ciclo de memoria) → Pedido del canal de la CPU, para hacer LEC o ESC, entre memoria y periférico.

INSTRUCCIÓN DE SUMA

Descripción de la instrucción de suma y las señales de gobierno necesarias para la ejecución de la misma. SUM/ SUS/ AND/ OR son todas instrucciones que tienen búsqueda de operando y su cronograma (señal de gobierno que se activa en cada θ) cuenta con dos ciclos de memoria (4θ).

La suma:

Sumar el contenido del operando (alojado en memoria en la dirección indicada por el campo DIR del RI) con el contenido del acumulador, para luego almacenar el resultado en el AC.

Se descompone en cuatro fases:

★ Fase 1: Búsqueda de la instrucción

Suponiendo que la dirección de la próxima instrucción a ejecutar se encuentra en el contador de programa:

1. Transferir el contenido del contador de programa al registro de selección de memoria:

(P) → S

2. Buscar en la memoria central la instrucción y transferirla al registro de palabra memoria. Para ello hay que lanzar un nuevo ciclo de memoria y el registro S indica la dirección de la instrucción, por lo tanto a M (el cual es puesto a cero previamente) llega el contenido del contenido de S

$$((S)) \rightarrow M$$

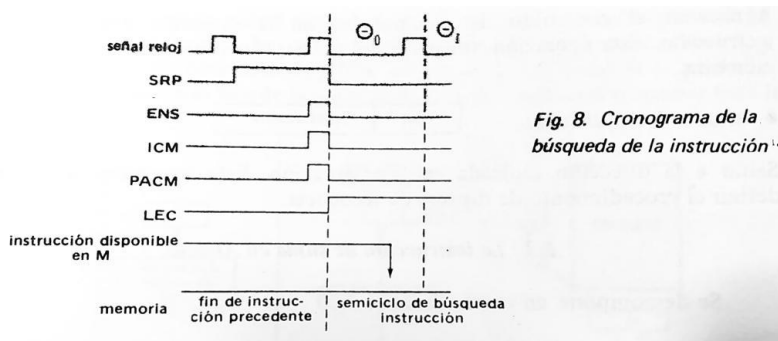
Señales de gobierno involucradas:

1. $(P) \rightarrow S : SRP \mid ENS$

Durante el ultimo θ de la instrucción anterior se incrementa el contador de programa, para así dejar en el mismo la instrucción actual a tratar. Ahora que CP tiene la dirección de la instrucción de suma, habilita la salida del registro con la señal de nivel **SRP** para que esta llegue al Bus S, luego la señal de muestreo **ENS** habilita que ingrese al registro de selección. Para esto la señal de nivel SRP tiene que estar activa hasta que termine la de muestreo para que este latente la información del Bus

2. $((S)) \rightarrow M: ICM \mid PACM \mid LEC$

Luego para tener a la instrucción en el registro de palabra memoria, hay que lanzar un ciclo de memoria con la señal impulsional **ICM**, indicar que se trata de una lectura manteniendo la señal de nivel **LEC** durante todo el intervalo del ciclo de memoria (y en cualquier caso hasta el inicio de la regeneración) y, si fuera necesario, poner a cero el registro M con la señal de muestreo **PACM**



★ Fase 2: Búsqueda de operando

3. La instrucción se encuentra en M antes del fin del primer θ . Debe enviarse el contenido del registro palabra memoria (la instrucción) al Registro Instrucción para así decodificar la misma. Estos dos registros se conectan por el Bus M. Entonces, durante el primer θ del segundo ciclo, se activa la señal de nivel **SRM**, y se mantiene hasta que en el próximo batido del clock la señal de muestreo **ENI** (sincronizada con el segundo θ del segundo ciclo), habilita que la información ingrese al RI.

$$(M) \rightarrow I$$

4. Al decodificar el CO y ver que se trata de una suma, en el campo DIR se encuentra la dirección del operando. Por lo tanto, ahora el contenido de DIR debe pasar al registro S, quienes están conectados por el Bus S. Entonces se activa la señal de nivel **SRD**, y luego para validar el ingreso a S la señal de muestreo **ENS**.

(DIR) → S

5. Una vez que tenemos en S la dirección del operando se pasa a leerla y colocarla en el registro M. Volvemos a lanzar un ciclo de memoria, poner a cero M y activar la señal de nivel **LEC** (*en la suma la lectura esta activa durante los dos ciclos de memoria*)

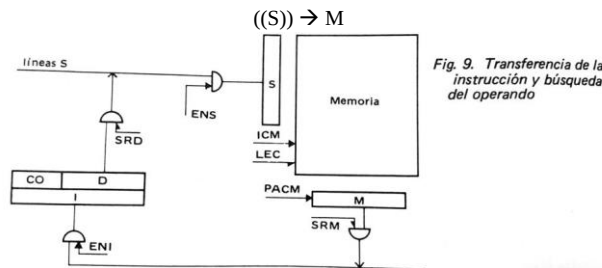


Fig. 9. Transferencia de la instrucción y búsqueda del operando

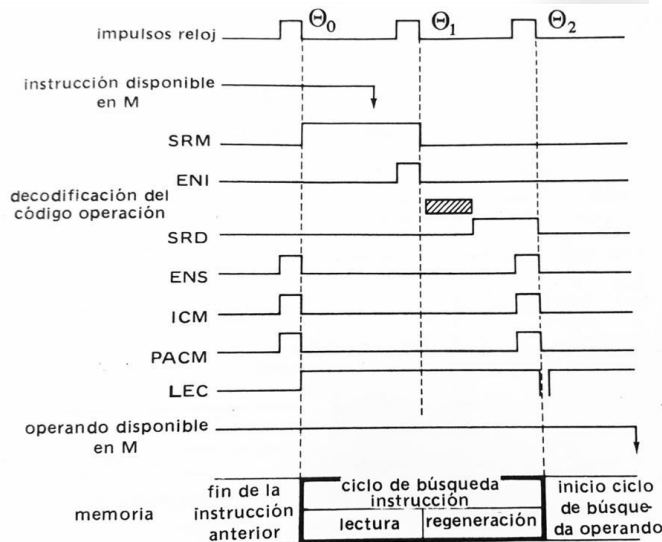
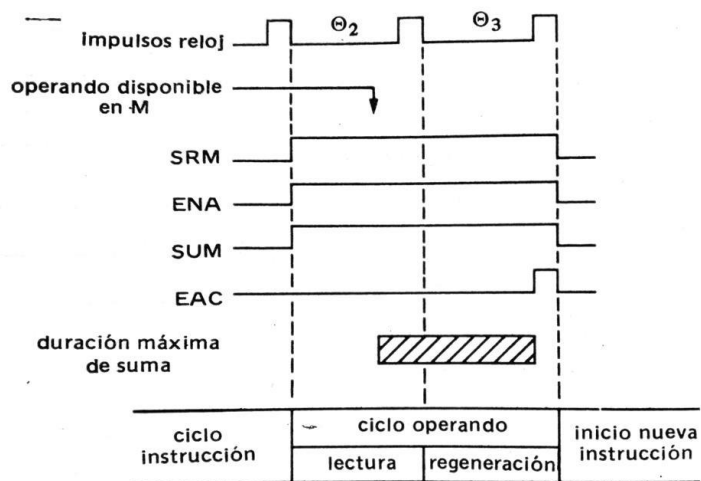
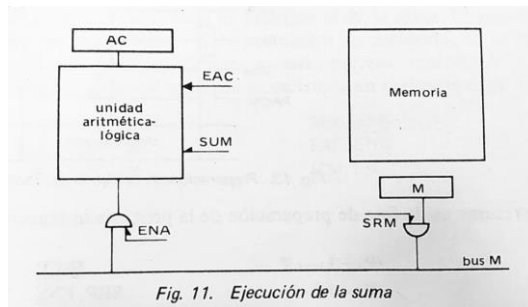


Fig. 10. Cronograma de la búsqueda del operando

★ Fase 3: Ejecución de la operación

6. Hay que sumar el contenido del registro M con el contenido del acumulador. Para ello debe estar disponible el operando en el Bus M, entonces se habilita la señal de nivel **SRM**, luego la señal de muestreo **ENA** habilita el ingreso a la ALU. Entonces debemos habilitar la señal de nivel **SUM** la cual permanece latente hasta que se complete la operación (hasta el fin del segundo ciclo). Una vez realizada la suma se guarda el resultado en el acumulador, para lo cual habilitamos la señal de muestro **EAC**

(M) + (AC) → AC



★ Fase 4: Preparación de la próxima instrucción

7. Se incrementa el contador de programa para que en este este disponible la dirección de la siguiente instrucción. Para esto se usa la señal de muestreo **INCP**

$$(P) + 1 \rightarrow P$$

8. Regresamos al inicio. Debemos enviar el contenido de P a S para leer la nueva instrucción, entonces hay que lanzar un nuevo ciclo de memoria. Para pasar el contenido de P habilitamos la señal de nivel **SRP**, la cual permanece activa hasta el impulso **ENS**.

$$(P) \rightarrow S$$

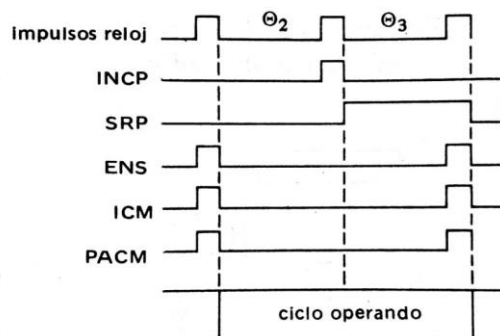


Fig. 13. Preparación de la siguiente instrucción

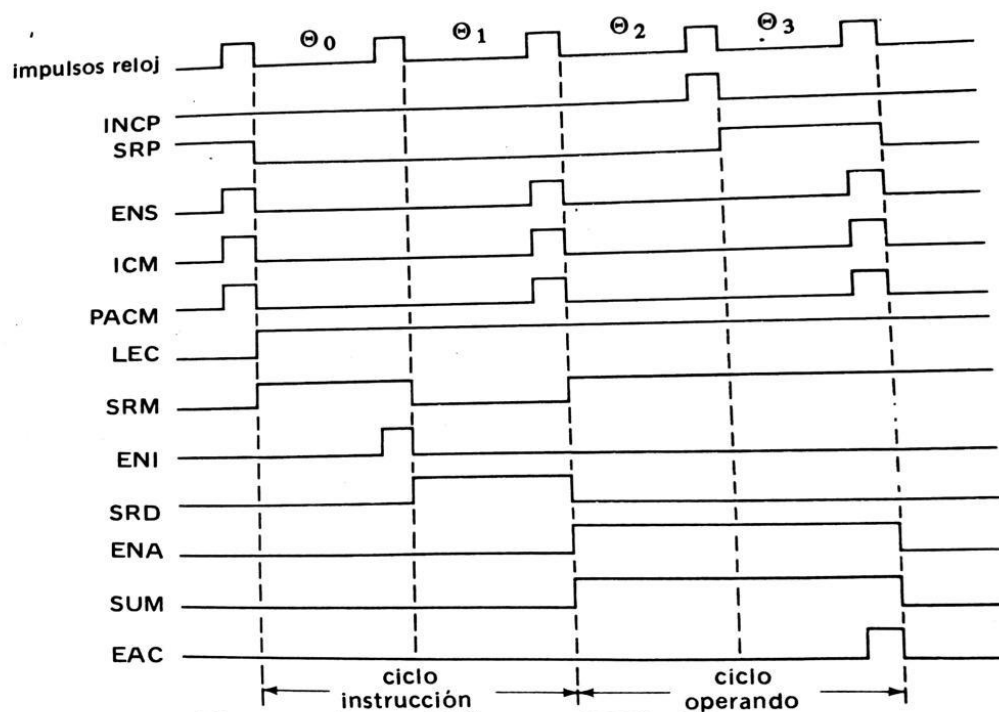


Fig. 14. Cronograma de la suma en Abacus

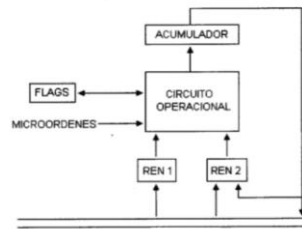
ABACUS COMPLETA CON SUS SEÑALES DE GOBIERNO

Unidad Aritmética y Lógica: Se compone de una unidad capaz de ejecutar toda instrucción de calculo. Para esto cuenta de Operadores o Unidades Funcionales destinadas a realizar una o varias operaciones aritméticas o lógicas

Esta gobernada por la unidad de control, y tiene como entradas operandos almacenados en los registros internos de la CPU, pudiendo estos ser suministrados por la memoria también. Los resultados se almacenan a su vez en registros de la CPU o en memoria. Adicionalmente, la ALU también puede activar los indicadores o flags que señalan las características de los resultados de la ultima operación realizada.

La ALU está formada por los siguientes elementos:

1. Operadores
2. Registros de entrada (REN): Se cargan los operandos
3. Registro acumulador: Se cargan los resultados, y en el modelo que estudiamos también operandos
4. Registro de estado (flags, indicadores de estado): suministran información acerca de la última operación realizada. Hay indicadores de Error y del estado del acumulador (como Z (cero), overflow, carry, signo).



OPERADORES

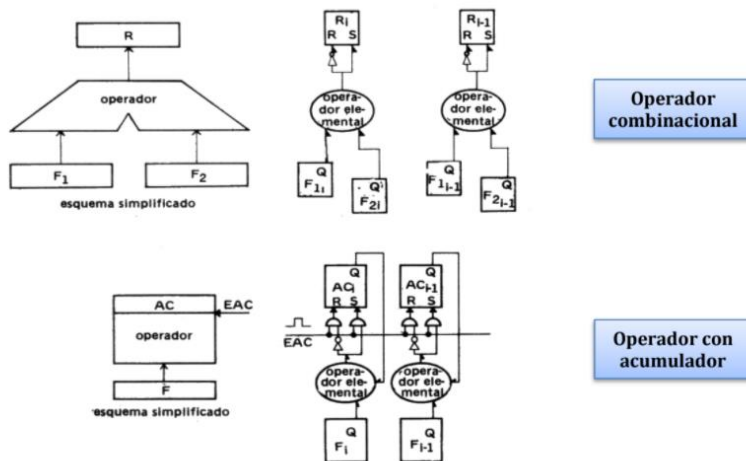
- ★ **Serie:** que trabajan según el régimen de bit tras bit, apenas se utilizan, siendo un ejemplo de esto el sumador serie.
- ★ **Paralelo:** sobre los que se concentra nuestro estudio. Cabe resaltar sin embargo que algunos no lo son más que parcialmente, como el caso del multiplicador ordinario en paralelo que trabaja en paralelo con los bits del multiplicando, pero en serie con los del multiplicador.
- ★ **Combinacional:** ejecutan todas las operaciones en una sola fase. Y por esto pueden ser gobernados por señales de nivel, que permanecen activas durante toda la operación (lo utilizamos en ABACUS).
- ★ **Secuencial:** ejecutan las operaciones en varias fases gobernadas por impulsos distribuidos por un dispositivo de control y que poseen un elemento de memoria(biestable) para los resultados parciales.

ALU ELEMENTAL

Veremos las operaciones que pueden ejecutarse con bits del mismo peso, como las operaciones lógicas y la suma binaria, dejando para más adelante operaciones más complejas como la multiplicación y la división.

Se encuentran dos montajes posibles de los operadores:

- ★ **Operadores puramente combinacionales:** se montan entre dos registros fuentes F1 y F2 (o dos buses fuentes) para los operandos y un registro R para el resultado (o un bus para el resultado).
- ★ **Operadores con acumulador:** no exigen más que un registro fuente para almacenar uno de los operandos, ya que el otro es memorizado por el acumulador durante todo el tiempo de la operación. Cuando a las salidas de los operadores elementales quedan establecidos los niveles lógicos, un impulso EAC introduce el resultado en el acumulador.



OPERACIONES LOGICAS

Fuera de la complementación, que se consigue por circuitos NOT en el caso del operador combinacional y por la complementación del acumulador, en el caso del operador con acumulador, las demás se reducen esencialmente a operaciones AND, OR y OR exclusivo (XOR o ORX).

En el caso del **operador combinacional**, se sustituyen los operadores elementales por compuertas AND para la intersección, OR para la reunión y XOR para la operación OR exclusivo. En el caso del **operador con acumulador**, la organización presentada arriba se simplifica partiendo de la idea siguiente: en vez de ejecutar la operación usando los bits del registro fuente y la realimentación del acumulador, se observa que basta, según el valor del bit fuente F_i con dejar inalterado el bit del acumulador o forzarle a un valor no dependiente más que de F_i .

Por ejemplo, para el AND, el bit acumulador no cambia si $F_i = 1$, pero debe poner a cero si $F_i = 0$. Esto puede agruparse en un mismo operador como se muestra abajo.

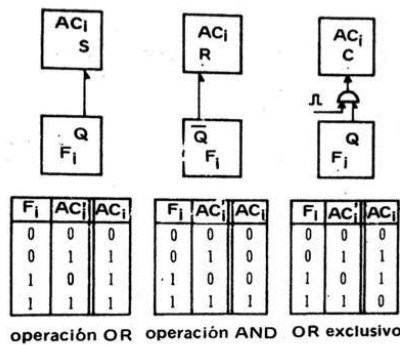
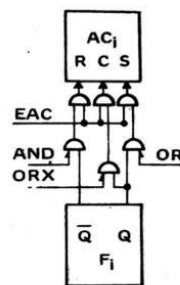


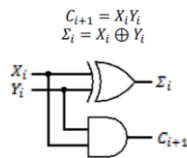
Fig. 3. Operaciones lógicas



SUMADORES

El **semisumador** realiza la suma de dos dígitos binarios. El resultado de la suma consta de dos dígitos: uno es el resultado S y el otro es el arraste (o carry) C_y .

X_i	Y_i	C_{i+1}	Σ_i
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

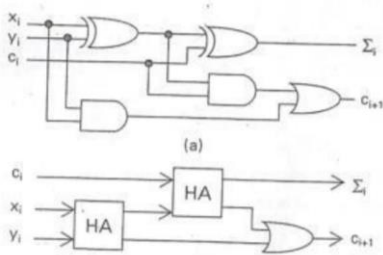


El **sumador completo** realiza la suma de dos dígitos binarios y el acarreo de una etapa anterior. De igual manera que en el semisumador, el resultado de la suma es el resultado de la operación y el arrastre.

$$C_{i+1} = Y_i C_i \oplus [X_i Y_i] + X_i Y_i$$

$$\Sigma_i = X_i \oplus Y_i \oplus C_i$$

X_i	Y_i	C_i	C_{i+1}	Σ_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



$$C_{i+1} = \bar{x}_i y_i c_i + x_i \bar{y}_i c_i + x_i y_i \bar{c}_i + x_i y_i c_i$$

$$= (\bar{x}_i y_i + x_i \bar{y}_i) c_i + x_i y_i (c_i + \bar{c}_i)$$

$$= (x_i \oplus y_i) c_i + x_i y_i \quad (\text{usando XOR})$$

$$= y_i c_i + x_i c_i + x_i y_i \quad (\text{simplificada})$$

$$\Sigma_i = \bar{x}_i \bar{y}_i c_i + \bar{x}_i y_i \bar{c}_i + x_i \bar{y}_i \bar{c}_i + x_i y_i c_i$$

$$= c_i (\bar{x}_i \bar{y}_i + x_i y_i) + \bar{c}_i (\bar{x}_i y_i + x_i \bar{y}_i)$$

$$= c_i (\overline{x_i \oplus y_i}) + \bar{c}_i (x_i \oplus y_i) \quad \text{si } z = x_i \oplus y_i \text{ entonces } \bar{z} = \overline{x_i \oplus y_i} \text{ y reemplazando}$$

$$= c_i \bar{z} + \bar{c}_i z = c_i \oplus z \quad \text{reemplazando de vuelta ...}$$

$$= x_i \oplus y_i \oplus c_i$$

Tenemos un **sumador en serie** cuando conectamos un semisumador seguido de un grupo de sumadores uno detrás de otro. Cada etapa de sumador debe esperar que la anterior finalice para ejecutarse.

Estos circuitos tienen el problema de que el número de niveles aumenta cuando el número de bits aumenta, lo que hace que el retardo sea mayor a medida que n crece.

Sustractor

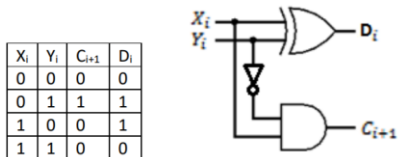
Un **sustractor** es un circuito combinacional que tiene como entrada dos números binarios de n bits y da como salida la sustracción binaria de uno por el otro. Normalmente este circuito no es usado, ya que hacer una resta es análogo a una suma donde se aplica complemento a la base al sustraendo.

$$A - B = A + \text{NOT } B$$

Un **semisustractor** posee como entrada dos dígitos binarios. Su salida es el resultado de la diferencia y un acarreo.

$$C_{i+1} = \bar{X}_i Y_i$$

$$D_i = X_i \oplus Y_i$$

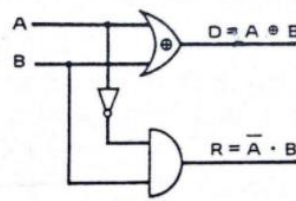


Un **sustractor completo** posee como entradas dos dígitos binarios y el acarreo de la entrada previa. Genera como resultados la diferencia y el acarreo.

entradas		salidas	
A	B	R	D
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

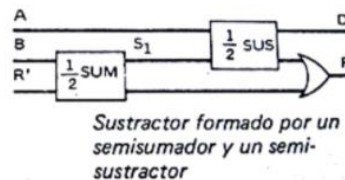
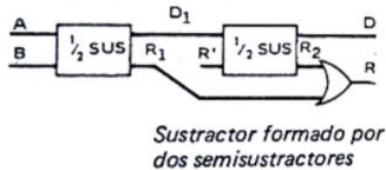
$$R = \bar{A} \cdot B$$

$$D = \bar{A} \cdot B + A \cdot \bar{B} = A \oplus B$$



Análogamente al sumador, la etapa de sustractor puede obtenerse de dos maneras diferentes:

- ✓ Se restan sucesivamente B de A y, después, R' de la diferencia D_1 obtenida.
- ✓ Se suman B y R' y el resultado se resta de A



SUMADOR BINARIO PARALELO

Compuesto por un semisumador para los bits de menor peso. Y para el resto de ellos sumadores completos ya que se cuenta con la propagación del arrastre o *carry* de la etapa anterior.

Es decir, **está constituido por una cadena de etapas de sumadores, en que la etapa de sumador de peso i transmite a la de peso $i + 1$ el arrastre eventual.** El arrastre de la etapa de mayor peso indica desbordamiento si está activada.

En la práctica no son muy usados los sumadores en serie, si los paralelos, ya que su gran ventaja es que permiten que en el tiempo entre un batido de clock y el próximo se realice el calculo de cada etapa y se resuelva la suma. Es mucho mas costoso que el serial al necesitar mas componentes.

SUMA Y RESTA BINARIA

Suma y resta binaria

Como sabemos, existen 3 formas de representar números enteros (o en coma fija) en las computadoras.

Todas ellas implican tratar el bit más significativo (el más a la izquierda) de la palabra como un bit de signo:

0 indica positivo y 1 indica negativo. Ellas son:

- ☐ magnitud y signo
- ☐ complemento a 2 (complemento auténtico o complemento a la base)
- ☐ complemento a 1 (complemento restringido o complemento a la base menos uno)

En la representación en magnitud y signo, el bit más significativo es el de signo y los restantes $n - 1$ bits representan la magnitud del entero. El rango de variabilidad de los valores en este formato es

$-(2^{n-1} - 1) \leq x \leq 2^{n-1} - 1$

$n - 1$. Por ejemplo, si se usan 4 bits ($n = 4$), el rango de variabilidad estará

comprendido entre -7 y $+7$. Permite 2 representaciones del cero: $+0$ y -0 (1000 y 0000).

En la representación en complemento a 1, el bit más significativo es el de signo y los $n - 1$ bits restantes se representan con su magnitud binaria real para el caso de los positivos y en complemento a 1 de la magnitud para el caso de los negativos. El rango de variabilidad es $-(2^{n-1} - 1) \leq x \leq 2^{n-1} - 1$. Así, para

$n - 1$

$n - 1$. Así, para

$n = 4$, el rango de variabilidad estará comprendido entre -7 y $+7$. También permite dos representaciones del cero, 1111 y 0000.

En la representación en complemento a 2, el bit más significativo es el de signo y los $n - 1$ bits restantes se representan con su magnitud binaria real para el caso de los positivos y en el complemento a 2 de la magnitud para el caso de los negativos. El rango de variabilidad es $-(2^{n-1}) \leq x \leq 2^{n-1} - 1$. Así para $n = 4$,

$n - 1$

$) \leq x \leq 2^{n-1} - 1$

$n - 1$. Así para $n = 4$,

el rango de variabilidad estará comprendido entre -8 y $+7$. Se tiene una sola representación del cero.

Comentado [mjo1]: arreglar

Comentado [mjo2R1]:

decimal	Magnitud y signo	Complemento a 1	Complemento a 2
7	0111	0111	0111
6	0110	0110	0110
5	0101	0101	0101
4	0100	0100	0100
3	0011	0011	0011
2	0010	0010	0010
1	0001	0001	0001
+0	0000	0000	0000
-0	1000	1111	-
-1	1001	1110	1111
-2	1010	1101	1110
-3	1011	1100	1101
-4	1100	1011	1100
-5	1101	1010	1011
-6	1110	1001	1010
-7	1111	1000	1001
-8	-	-	1000

Las 3 representaciones son iguales del +0 para arriba y las 3 representan hasta el mismo número positivo

Hoy en día la más usada es la representación en complemento a 2, por lo que es con la que vamos a trabajar.

Las 3 representaciones son

iguales del +0 para arriba y

las 3 representan hasta el

mismo número positivo

Restar un número de otro equivale a sumarle su opuesto. De esta manera la resta se consigue usando la suma, lo que evita tener que situar un sustractor junto al sumador. Así, la regla dé la résta queda: “para restar un número (sustraendo) de otro (minuendo), se le suma al minuendo el complemento a 2 del sustraendo”. Así, la ALU realiza las restas transformándolas en sumas de la siguiente manera:

$$\begin{aligned}
 (+A) - (+B) &\Rightarrow (+A) + (-B) \\
 (+A) - (-B) &\Rightarrow (+A) + (+B) \\
 (-A) - (+B) &\Rightarrow (-A) + (-B) \\
 (-A) - (-B) &\Rightarrow (-A) + (+B)
 \end{aligned}$$

A continuación, se muestra un ejemplo de cada resta y su correspondiente ejecución como una suma:

Podemos destacar que:

- ☐ Tanto operandos como resultados negativos están complementados a 2. La ALU entrega los resultados tal cual son obtenidos (complementado si es negativo o sin complementar si es positivo) y es función del programa interpretar que cuando el signo es negativo el valor está enmascarado en su complemento.
- ☐ El signo forma parte de la operación, sumándose como cualquier otro bit.
- ☐ En las operaciones se consideran todos los bits del formato, inclusive los 0 a la izquierda.

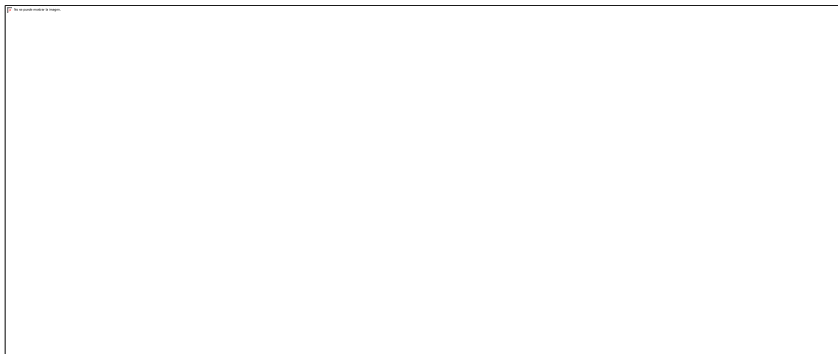
❑ En algunos casos, hay un bit acarreo más allá del final de la palabra (sombreado en el tercer ejemplo de arriba). Este bit se descarta.

SUMADOR DECIMAL

GOBIERNO SECUENCIAL DE DESPLAZAMIENTO

Acumulador montado como un registro de desplazamiento, a izquierda o derecha dependiendo de si se activa la señal de gobierno DESI o DESD respectivamente. El contador, indica cuantos desplazamientos se deben hacer, se ve restado por cada señal de desplazamiento del acumulador, y detiene su operación cuando este llega a cero. Para que sea cero todos los biestables deben contener un cero, ya que al estar conectados por una OR basta con que uno de ellos contenga un 1 para que se haga el desplazamiento.

Esta unidad veremos que luego es utilizada en la ALU que opera con coma flotante.

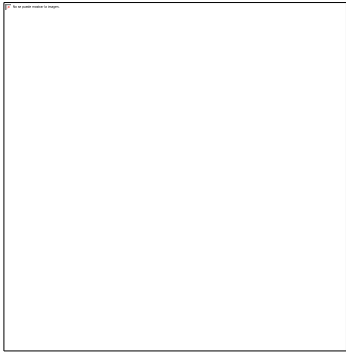


ALU PARA ABACUS

Es un circuito combinacional que se encarga de realizar las operaciones elementales de tipo aritmético (suma, resta, multiplicación, división) y del tipo lógico (AND, OR , XOR, NOT).

Esta montada entre un **Bus M**, donde se mantienen los niveles lógicos representativos del segundo operando, y un **acumulador AC** que mantiene los niveles lógicos correspondientes al primer operando hasta que la **señal de muestreo EAC** introduce el resultado en el acumulador. Consta de un **sumador en paralelo** y de un conjunto de puertas para ejecutar las operaciones lógicas y distribuir las informaciones de acuerdo con las operaciones aritméticas por efectuar. Además, **el acumulador está montado como registro de desplazamiento**.

Las operaciones están gobernadas por señales procedentes del secuenciador de la computadora, que son todas de nivel, de manera que al cabo de un cierto tiempo después de su posicionamiento el resultado de la operación se estabiliza bajo la forma de niveles lógicos, que serán introducidos en el acumulador por el impulso EAC.



→ arreglar

Sostiene la salida de M sobre el bus todo el tiempo que requiera para que la ALU complete la información. No contiene un registro de entrada B y puede utilizar directamente el bus, ya que no se comunica con el exterior, ya que le BUS M esta pensado únicamente para los cálculos aritmético-lógicos. Hoy en día ya no se utiliza un solo Bus M, sino un conjunto de buses.

9 operaciones de ALU para ABACUS:

Estas se realizan entre bits del mismo peso, con el Bus M y el registro acumulador

- ★ **SUM** *adición*: suma al contenido del acumulador la información presente sobre el bus M usando el sumador en paralelo.
- ★ **SUS** *sustracción*: se resta la información en el bus M (2do operando, sustraendo) del contenido del acumulador (1er operando, minuendo). En este caso, la resta se hace complementando a 1 la información del bus M y después sumándola en el sumador paralelo con el contenido del acumulador. Si se quisiera hacer por complemento a 2, debería enviarse un acarreo de entrada a la etapa de sumador de menor peso de manera de sumar 1 al complemento a 1.
- ★ **OR** *reunión lógica* entre la información en el bus M y el contenido del acumulador.
- ★ **XOR** *OR exclusivo* entre la información en el bus M y el contenido del acumulador. Se hace usando las etapas de sumador, inhibiendo la propagación de los arrastres, lo que las convierte en semisumadores, donde la suma de los dígitos de entrada equivale a un ORX de estos.
- ★ **AND** *intersección lógica* entre la información en el bus M y el contenido del acumulador.
- ★ **DESD** *desplazamiento a derecha* del contenido del acumulador en una posición.
- ★ **DESI** *desplazamiento a izquierda* del contenido del acumulador en una posición.
- ★ **CAR** *carga* transferencia forzada al acumulador de la información presente sobre el bus M. esta info llega complementada a la entrada RESET y sin complementar a SET.
- ★ **CARC** *carga con complementación*: el mismo proceso, pero después de complementar cada dígito.

En el siguiente esquema presentamos una unidad para ABACUS, representada para un bit.



Multiplicación secuencial por suma-desplazamiento

El resultado de una multiplicación consiste en la suma de los productos parciales convenientemente desplazados de cada bit del multiplicador por el multiplicando.

Cuando el bit del multiplicador es 0, el producto parcial es 0. Cuando el bit del multiplicador es 1, el producto parcial es igual al multiplicando.

Lo primero que debemos hacer, entonces, es verificar sucesivamente cada bit del multiplicador, de donde se deducen los productos parciales que, convenientemente desplazados, se totalizan en el acumulador.

En el caso de que tengamos un multiplicando y un multiplicador de n bits, el acumulador necesario deberá tener capacidad para $2n$ bits, es decir, el resultado se obtiene en doble longitud. Para lograr esto, el acumulador debe tener un anexo llamado "Multiplicador-Cociente" (MC), llamado así debido a que contiene al multiplicador en la multiplicación y al cociente en la división.

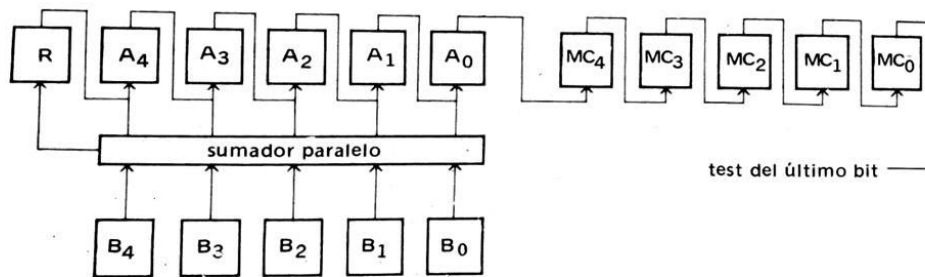


Fig. 11. Principio del operador de multiplicación por suma-desplazamiento

1. Se carga el multiplicador (el de abajo) en el acumulador
2. Se desplaza a derecha el contenido del conjunto AC + MC, lo que resulta en poner a 0 el acumulador y el multiplicador en el MC
3. Se carga el multiplicando (el de arriba) en el registro B (que podría ser el registro palabra memoria M)
4. Se sigue el siguiente algoritmo:



DIVISIÓN SECUENCIAL POR SUSTRACCIÓN-DESPLAZAMIENTO

Se usa la misma unidad aritmética que en la multiplicación, pero cambiaremos el sumador paralelo por el sustractor paralelo, invertiremos el sentido del desplazamiento (ahora a izquierda) y añadiremos un dispositivo de comparación.

Se resta del dividendo el divisor o 0 según que el correspondiente bit del cociente valga 1 o 0.

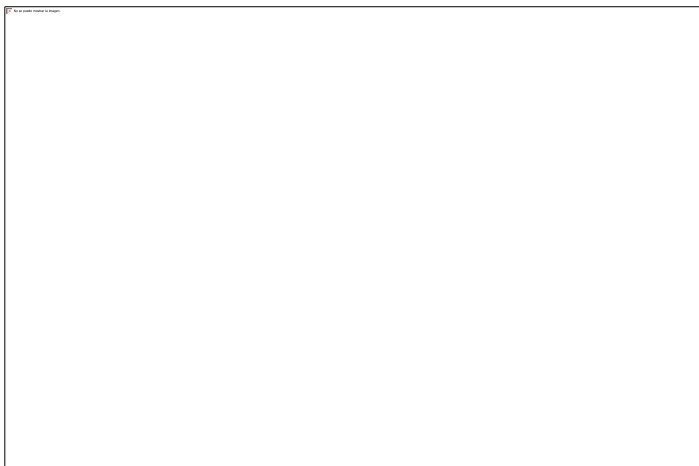
El comparador opera entre los bits de mayor peso del dividendo y el divisor, a fin de determinar el bit correspondiente del cociente.

El desplazamiento a izquierda del MC tiene como objetivo poner el bit de mayor peso del dividendo en A0 (bit de menor peso del acumulador), y libéral el biestable MC0 (bit de menor peso del MC) que pasara a contener el primer bit del cociente, y así sucesivamente

El resultado de la división, es decir el cociente, queda almacenado en el MC y el resto se encuentra en el AC.



1. Se carga el dividendo (el de la izquierda) en el AC
2. Se desplaza a derecha AC + MC, para poner a cero el AC y dejar el dividendo en el MC
3. Se carga el divisor en B.
4. Luego se sigue el siguiente algoritmo:



Multiplicacion	Division
multiplicad OR → AC	divid ENDO → AC
multiplic ANDO → B	divis OR → B
DESD	DESI
Sumador Paralelo	Sustractor Paralelo

DIVISION CON Y SIN RESTAURACION

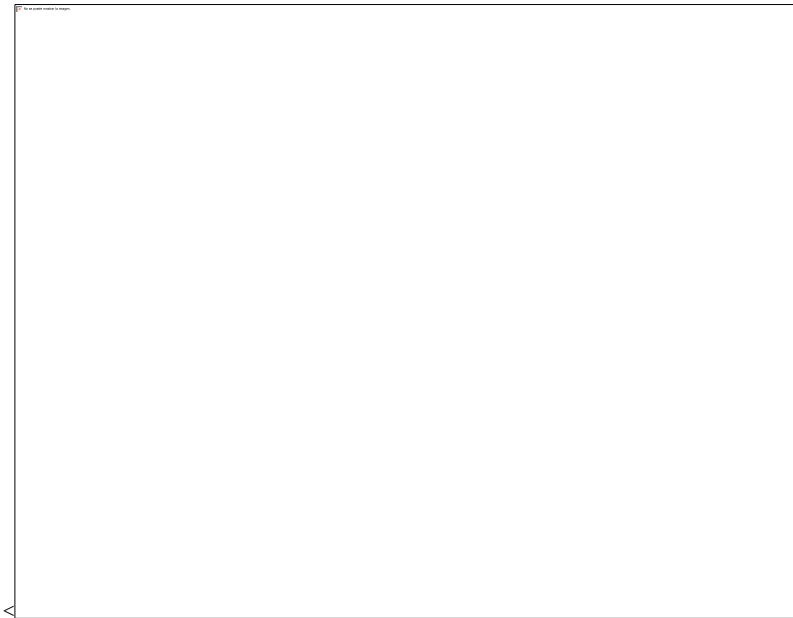
Debido a que es muy difícil contar con un dispositivo comparador para realizar las operaciones de división, debido a su alto costo, se buscan métodos alternativos para la realización de las mismas.

DIVISION CON RESTAURACION

Para comparar los valores del acumulador y el registro B, lo que puede hacerse es restar el divisor del dividendo y comprobar el signo del resultado.

- ★ Si el resultado es menor a cero ($AC - B$) el acumulador pierde el antiguo valor del dividendo y en ese caso es necesario restaurarlo, sumando al dividendo el divisor antes de hacer un nuevo intento de resta y se pone a cero el bit de menor peso del MC.
- ★ Si el resultado es mayor o igual a cero ($AC \geq B$), se pone en 1 el bit de menor peso del MC. Debido a que la resta que le seguía en la resta por sustracción-desplazamiento ya se realizó previamente.

Siempre se realiza el desplazamiento a izquierda, previo a la sustracción del dividendo con el divisor.



$$\begin{array}{l} \text{Si } \beta = 1010 \text{ (10)} \\ \quad \quad \quad \searrow \searrow \searrow \\ \beta/2 = 0101 \text{ (5)} \end{array}$$

DIVISION SIN RESTAURACION

Dado que la realización de la restauración consume tiempo de cálculo, tenemos la división sin restauración.

Sean α y β el contenido del acumulador y el divisor respectivamente antes de la fase de restauración (es decir, ya se hizo la resta y dio negativa), lo que se hace es $\alpha + \beta$ (restauración) seguida de una sustracción del divisor desplazado (que vale $\beta/2$):

$$(\alpha + \beta) - \frac{\beta}{2} = \alpha + \frac{\beta}{2}$$

Por lo tanto, esto quiere decir que si en la resta de comparación, el resultado < 0 , hago la restauración, el desplazamiento a la izquierda de AC+MC y la sustracción que le sigue TODO EN UN SOLO PASO, haciendo $\alpha + \beta/2$. Pero como no puedo desplazar el divisor a derecha (ya sea porque B no es un registro de desplazamiento o porque iría perdiendo el divisor de a poco), hace el equivalente, es decir desplazar a izquierda AC+MC y sumarle el divisor sin desplazar.

En el caso de que el resultado ≥ 0 , como no se restaura, entonces no se suma β .

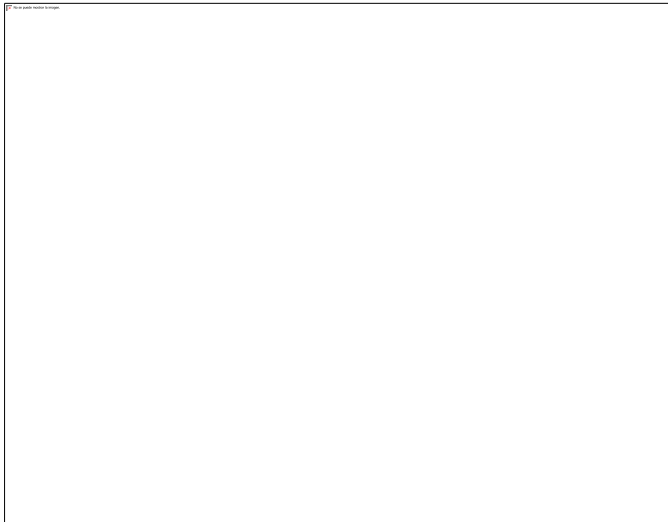
$$\alpha - \frac{\beta}{2}$$

En este caso, en vez de sumarle el divisor desplazado, lo resto. Por el mismo motivo que arriba, esto se implementa al revés, es decir se desplaza a izquierda AC+MC y se le resta el divisor sin desplazar.

¿Por qué $\beta/2$? Porque desplazar a la derecha β o dividirlo por 2 da lo mismo.

¿Por qué se le resta el divisor desplazado? Porque después de la suma de restauración se desplaza

AC+MC un lugar a la izquierda. Es lo mismo desplazar a izquierda AC+MC y restarle el divisor sin desplazar (como en el algoritmo con restauración) que restarle al AC+MC sin desplazar el divisor desplazado a derecha ($\beta/2$).



OVERFLOW O DESBORDAMIENTO

Puede ocurrir que el resultado de una suma sea mayor que el que podemos representar con la longitud de palabra que se está usando. En estos casos se dice que se excede la capacidad (de representación) del registro, lo que provoca la invalidez del resultado obtenido. Por ejemplo, si la palabra tiene 4 bits, podemos representar, en complemento a dos, de -8 a 7 , por lo que el resultado de $5 + 4 = 9$ no puede representarse.

Esta condición se denomina desbordamiento (overflow) y cuando ocurre, la ALU debe indicarlo para que no se intente utilizar el resultado obtenido.

Por ejemplo: $(-7) + (-6)$

$$\begin{array}{r} 1001 = -7 \\ + 1010 = -6 \\ \hline 10011 = \text{Desbordamiento} \end{array}$$

se descarta el acarreo

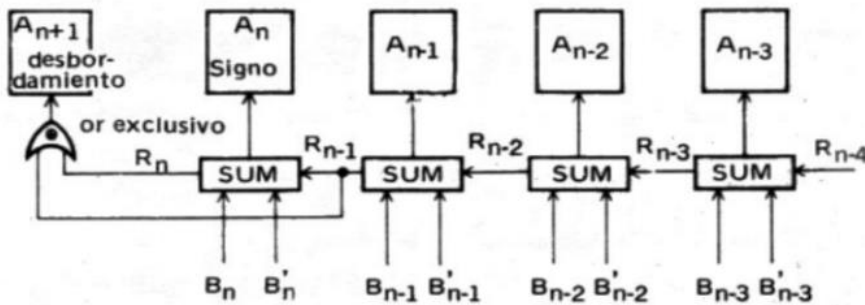
resultado no válido ya que se detectó desbordamiento (-13 excede el mínimo que podemos representar con 4 bits en complemento a 2, que es -8)

Regla para detectar desbordamiento:

Si luego de sumar dos números, el acarreo sobre el bit más significativo (el de signo) y el acarreo fuera de los n bits son diferentes, ocurre un desborde y el resultado no es correcto. Es decir, cuando no hay acarreo de orden $n - 1$ pero hay acarreo de orden n o al revés. Otra forma equivalente de detectar desborde es si al sumar dos números positivos (negativos) el resultado es negativo (positivo), ocurre un desborde y el resultado obtenido no es correcto.

Se debe destacar que el overflow sólo puede producirse con operandos del mismo signo, ya que en el caso contrario (un positivo y un negativo), el resultado obtenido siémpre “entrará” en el rango representable (en realidad, uno se resta del otro).

Supondremos que nuestra unidad aritmética procesa números representados por n bits, con el bit de signo situado en la primera posición a la izquierda. Asociaremos al acumulador un $(n + 1)$ -simo bit, que actuará de indicador de desbordamiento. De acuerdo a la regla de desbordamiento, este indicador debe posicionarse solamente en 2 casos para detectar desborde: cuando no hay acarreo de orden $n - 1$ pero si de orden n , o bien cuando hay acarreo de orden $n - 1$ pero no de orden n , lo que corresponde a un OR exclusivo entre ambos acarreos.



En general, las reglas para sumar y restar números enteros representados en complemento a 2 son:

- > Para sumar dos números enteros en representación en complementos a 2, se suman los n bits directamente.
- > Si el acarreo sobre el bit más significativo y el acarreo fuera de los n bits son diferentes, ocurre un desborde y el resultado no es correcto.
- > Si el acarreo sobre el bit más significativo y el acarreo fuera de los n bits son iguales, el resultado es correcto y queda en la representación de complementos a 2. Cualquier acarreo fuera de los n bits se deshecha.
- > Para hacer la resta de $A - B$ se hace la suma de $A + (-B)$. Si el número B es el más negativo, esta operación no puede realizarse ya que no es posible representar el número $-B$.

A continuación se muestran las 3 configuraciones posibles de signo de los operandos, y lo que sucede en cada caso cuando hay presencia y ausencia del acarreo procedente de la etapa $n - 1$:

	2 operandos > 0 El resultado debe ser positivo (si sumo dos positivos obtengo un positivo), de lo contrario hay desbordamiento	1 operando > 0; 1 operando < 0 No es posible que haya desbordamiento, el resultado va a ser positivo o negativo	2 operandos < 0 El resultado debe ser negativo (si sumo dos negativos obtengo un negativo), de lo contrario hay desbordamiento
operando 1	0	1 (o 0)	1
operando 2	0	0 (o 1)	1
resultado si no hay acarreo procedente de la etapa $n - 1$ 0 VÁLIDO ✓ resultado positivo esperado ✓ $R_{n-1} = R_n = 0$	1 VÁLIDO ✓ resultado negativo ✓ $R_{n-1} = R_n = 0$	±0 INVÁLIDO: DESBORDAMIENTO ✓ resultado positivo ✓ $R_{n-1} = 0$ y $R_n = 1$	
resultado si hay acarreo procedente de la etapa $n - 1$ 1 INVÁLIDO: DESBORDAMIENTO ✓ resultado negativo ✓ $R_{n-1} = 1$ y $R_n = 0$	±0 VÁLIDO ✓ resultado positivo ✓ $R_{n-1} = R_n = 1$	±1 VÁLIDO ✓ resultado negativo esperado ✓ $R_{n-1} = R_n = 1$	

ARITMETICA EN COMA FLOTANTE

Los números en coma flotante se representan en la forma $SM \cdot \alpha^E$, donde

S es el signo del número

M la mantisa

E el exponente

α generalmente vale 2 o 16 en aritmética binaria.

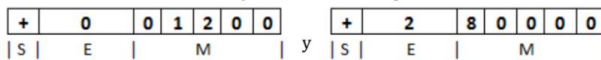
Las operaciones se llevan a cabo sobre **números normalizados**, es decir sobre números cuyo valor de exponente está ajustado para que la mantisa tenga el mayor número posible de dígitos significativos. Por lo tanto, un operador que trabaje con coma flotante debe, no solamente ejecutar la operación correspondiente (suma, resta, etc), sino también normalizar el resultado obtenido.

SUMA Y SUSTRACCION EN COMA FLOTANTE

Comenzamos con unos ejemplos para ver las dificultades que conlleva operar con coma flotante.

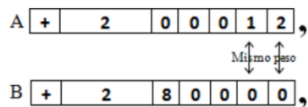
1. Alinear las Mantisas:

Para enfrentar los bits de igual peso. Como ejemplo sumamos los números $A = 12000 \times 10^{-1}$ y $B = 80000 \times 10^2$ que se escriben:



No se pueden sumar directamente las mantisas porque los exponentes no son iguales. Comparando los dos exponentes vemos que el primero es más chico. Lo que se hace entonces sobre el número con exponente más chico es:

- Se incrementa el exponente en una unidad
- Por cada incremento desplazamos la mantisa una posición a la derecha
- Se vuelven a comparar los exponentes, y si no son iguales, vuelvo a incrementar y desplazar. Si son iguales puedo sumar las mantisas



Incrementar exponente → DESD
Decrementar exponente → DESI

¿Por qué se hace esto?

Igualar los exponentes alinea las comas, dejando los dígitos del mismo peso uno debajo del otro. Al desplazar a la derecha la mantisa, lo que hacemos es correr la coma a la izquierda, lo que incrementa el exponente.

Antes: $12000, \times 10^{-1}$

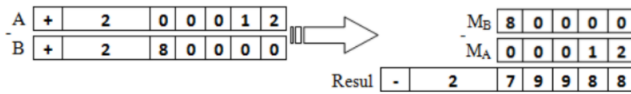
representado como $1200,0 \times 10^0$

Después: $12,000 \times 10^2$

El hecho de desplazar el operando que tiene menor exponente se debe a que un desplazamiento a derecha implica la pérdida de los dígitos menos significativos y, a lo sumo, el resultado se verá afectado por un redondeo. En cambio, **un desplazamiento a izquierda significaría la pérdida de los dígitos más significativos de la mantisa** y, por ende, el resultado será erróneo.

2. Ajustar los signos

Con la suma no hay problema, pero en la resta $A - B$ (en representación en signo y magnitud) se deberán comparar las mantisas de A y B para saber si la de B es mayor que la de A, en cuyo caso se deberá restar $M_B - M_A$ y al resultado darle signo negativo.



3. Normalización del resultado

Puede que el resultado de la operación no esté normalizado, por 2 razones:

(1) hay **desbordamiento** de capacidad de la suma, en cuyo caso debemos desplazar la mantisa una posición a la derecha, aumentando el exponente en una unidad

(2) la diferencia de las mantisas implica la formación de un **número n de ceros a la izquierda** del resultado, en cuyo caso debemos desplazar la mantisa n posiciones a la izquierda, restando n del exponente.

Caso 1: desbordamiento								Caso 2: muchos ceros a la izquierda							
1er operando	+	2	8	0	0	0	0	1er operando	+	2	8	0	0	0	0
2do operando	+	2	8	0	0	0	0	2do operando	-	2	7	9	9	9	9
resultado	1 6 0 0 0 0 0							resultado	0 0 0 0 0 1						
normalización	+	3	1	6	0	0	0	normalización	+	-2	1	0	0	0	0

FUNCIONAMIENTO DE UN OPERADOR EN COMA FLOTANTE

Los operandos tienen la siguiente configuración:

S	E	M
---	---	---

★ S = 1 bit = signo del operando (0 = positivo, 1 = negativo)

★ E = e bits = exponente.

Expresa una potencia de 2 y representa exponentes que van de -2^{e-1} a $2^{e-1} - 1$

. exponente decimal -2^{e-1} se representa por el binario $E = 0$

. exponente decimal $2^{e-1} - 1$ por el binario $E = 2^{e-1}$.

Para facilitar la comparación de exponentes, se elimina el signo del mismo agregándole un **exceso** igual a 2^{e-1} . Esto permite representar números positivos y negativos usando solamente números positivos.

$$\begin{array}{ccc} \pm E & + & 2^{e-1} \\ \text{exponente} & & \text{exceso} \end{array}$$

En general, si el exponente ocupa e bits, está generalmente representado por exceso de 2^{e-1} , es decir, que los exponentes negativos variantes entre -2^{e-1} y 0 son representados por los valores binarios de 0 a 2^{e-1} , mientras que los exponentes positivos de 0 a $2^{e-1} - 1$ lo son por los valores binarios de 2^{e-1} a $2^e - 1$.

Para explicar mejor esto, consideremos un campo de exponente de **4 bits**. Con él, puedo representar $2^4 = 16$ exponentes (8 negativos, el cero y 7 positivos):

Exponentes	Valores binarios
-8 = -2^{4-1}	0000 = 0
-7	0001
-6	0010
-5	0011
-4	0100
-3	0101
-2	0110
-1	0111
0	1000 = 2^{4-1}
1	1001
2	1010
3	1011
4	1100
5	1101
6	1110
7 = $2^{4-1} - 1$	1111 = $2^4 - 1$

EXPONENTES NEGATIVOS ENTRE -2^{e-1} Y 0 SE REPRESENTAN POR LOS VALORES BINARIOS DE 0 A 2^{e-1}
(En este caso los exponentes de -8 a 0 se representan por valores binarios de 0 a 8)

EXPONENTES POSITIVOS ENTRE 0 Y $2^{e-1} - 1$ SE REPRESENTAN POR LOS VALORES BINARIOS DE 2^{e-1} A $2^e - 1$
(En este caso los exponentes de 0 a 7 se representan por valores binarios de 8 a 15)

Los exponentes de la columna izquierda se representan por **exceso de 8** ($2^{e-1} = 2^{4-1}$) en la columna de la derecha. De esta manera, el exponente 4 (0100) se representa por 1100 (12) porque $0100 (4=\text{EXP}) + 1000 (8=\text{EXC}) = 1100 (12)$. El exponente verdadero se obtiene restando el exceso al exponente en exceso.

★ **M = m bits = mantisa.**

La mantisa de un operando negativo está representada por complemento a 2.

Un operando normalizado se caracteriza por el hecho de que el bit de signo y el de mayor peso de la mantisa tienen valores diferentes.

ALU ABACUS CON COMA FLOTANTE

El operador usado considera 2 registros A y B para los operandos y un sumador de $m + 1$ bits (mantisa y signo), que ejecuta las sustracciones por complementación previa del segundo operando. Se usa, por un lado, para comparar los exponentes por sustracción, dejando el resultado en el **registro CD** (montado como contador-descontador), y por el otro, para las operaciones con signo y mantisa, cuyo resultado va al **registro RD** (montado como registro de desplazamiento a derecha e izquierda).

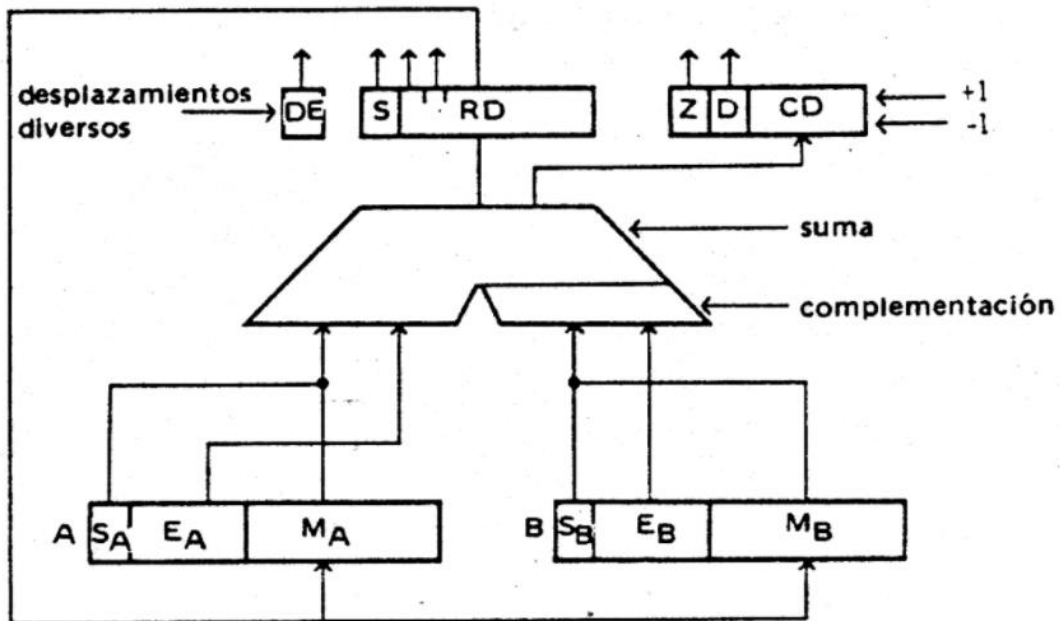


Fig. 23. Operador de suma-sustracción en coma flotante

La operación se puede dividir en 5 fases (3 si no hay que normalizar el resultado):

4. **COMPARACIÓN DE LOS EXPONENTES:** La comparación se hace por sustracción. Se resta $E_A - E_B$ sumando $E_A + (-E_B)$ y el resultado va a CD.

- Si la resta da nula, las mantisas están correctamente alineadas (ambos exponentes son iguales), por lo que saltamos la 2da fase de "Alineamiento de las mantisas" y pasamos a la 3ra, o sea "Suma- sustracción de las mantisas".
- Si no, el contenido de CD (diferencia entre los exponentes) permitirá contar los desplazamientos necesarios para alinear las mantisas en la 2da fase.

5. **ALINEAMIENTO DE LAS MANTISAS:** se pasa a esta fase porque uno de los exponentes es menor al otro,

y se trabaja sobre el operando con menor exponente.

- Si $E_A < E_B$, el resultado de la sustracción de los exponentes es negativo. El **complemento del contenido de CD me indica el número de desplazamientos por efectuar para alinear la mantisa**. El desplazamiento de la mantisa se hace en RD, por lo que se envía ($S_A M_A$) a RD, el cual se desplaza a derecha incrementando CD por cada desplazamiento. Esto ocurre hasta que $CD = 0$, momento en el cual se devuelve el contenido de RD al registro A y se pasa el mayor exponente (E_B) a CD.
- Si $E_A > E_B$, el desplazamiento a derecha afecta a S_B y M_B pero con la diferencia que CD se usa como descontador. Cuando $CD = 0$, se pasa el contenido de RD al registro B y se pasa el E_A a CD.

6. **SUMA-SUSTRACCIÓN DE LAS MANTISAS:** se opera entre los conjuntos signo-mantisa yuxtapuestos

(uno seguido del otro), sea ($S_A M_A$) $\pm$ ($S_B M_B$). La sustracción se obtiene sumando a $S_A M_A$ el complemento a la base de $S_B M_B$. El resultado (signo, mantisa y posible desborde) se lleva a RD.

7. **NORMALIZACIÓN EN EL CASO DE DESBORDAMIENTO:** se desplaza una posición a la derecha la

mantisa del resultado en RD, siendo reemplazado el bit de mayor peso por el desbordamiento. El signo queda inalterado y el registro CD se incrementa en 1.

8. **NORMALIZACIÓN EN EL CASO QUE LA MANTISA DEL RESULTADO TENGA DÍGITOS NO**

SIGNIFICATIVOS A SU IZQUIERDA: esto se detecta haciendo un test sobre el signo y el dígito de mayor peso de la mantisa.

- Si son distintos, el número ya estaba normalizado.
- Si no, se tienen que hacer desplazamientos a izquierda de la mantisa hasta que sean distintos y por cada desplazamiento se decrementa 1 de CD.

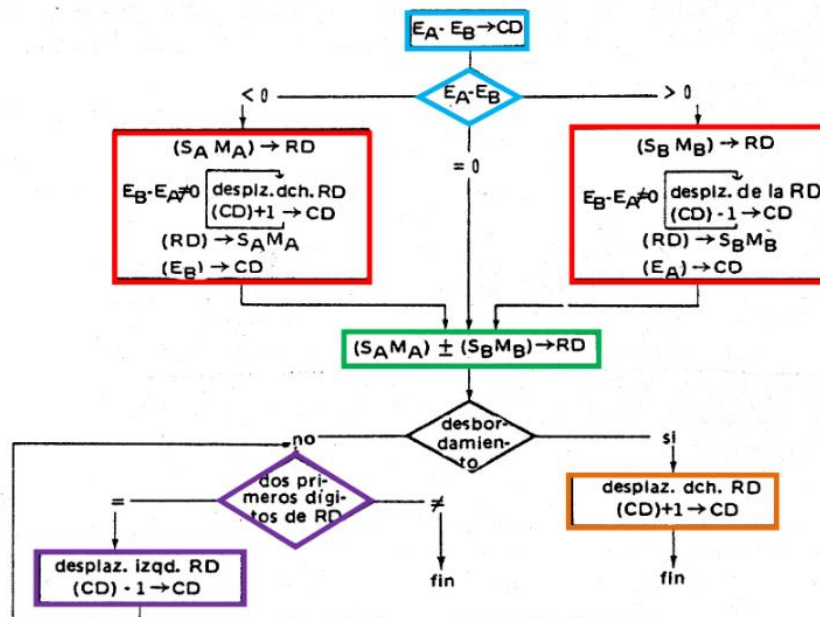


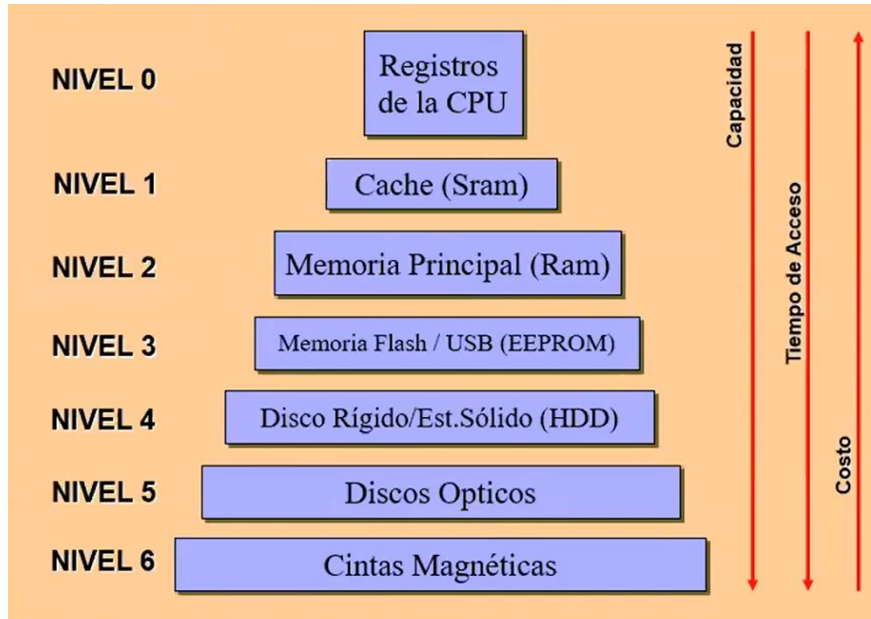
Fig. 24. Suma-sustracción flotante

MEMORIA

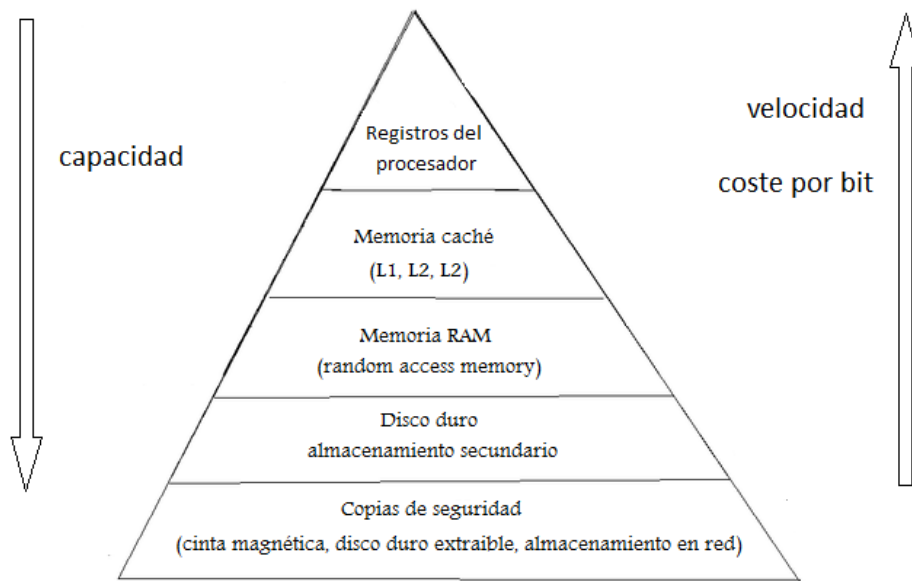
CARACTERISTICAS DE LAS MEMORIAS

- ★ **Volatilidad:** una memoria es volátil si la información se pierde al desconectarla de la fuente de alimentación eléctrica (memorias de semiconductores como DRAM y SRAM), y es no volátil en el caso contrario (como las memorias ROM o las memorias auxiliares de medios magnéticos y ópticos).
- ★ **Destructibilidad:**
 - **Lectura destructiva:** la información se pierde al ser leída y por lo tanto debe ser reescrita después de cada lectura (memorias de núcleos magnéticos, memorias DRAM)
- ★ **Modo de acceso:**
 - **Acceso aleatorio:** Se puede acceder a cualquier posición de forma directa y en el mismo tiempo, independientemente de su posición en la memoria, por lo que el tiempo de acceso es constante. No se requiere el acceso previo a otras unidades de información. Cada posición direccionable de memoria tiene un único mecanismo de acceso cableado físicamente. La información se ubica con una dirección decodificada por un decodificador asociado a la memoria, que permite la selección de sólo una unidad de información por vez.
 - **Acceso secuencial:** Para acceder a una posición de memoria es necesario acceder a todas las posiciones que le preceden. En este caso, el tiempo de acceso depende de la distancia entre la posición inicial y la unidad de información, por lo que es muy variable.
 - **Acceso asociativo:** Permite hacer una comparación de ciertas posiciones de bits dentro de una palabra buscando que coincidan con unos valores dados (rótulo o label), y hacer esto para todas las palabras simultáneamente. Una palabra es por tanto recuperada basándose en una porción de su contenido en lugar de su dirección, razón por la cual se las llama también “accesibles por contenido”. Es del tipo de acceso aleatorio, por lo que el tiempo de acceso es constante e independiente de la posición.
 - **Mixto o directo** (las unidades de disco son de acceso directo porque se accede directamente a la pista deseada y secuencialmente a la información mediante el giro del disco).
- ★ **Tiempo de acceso:** Para memorias de acceso aleatorio es el tiempo que tarda en realizarse una operación de escritura o de lectura, es decir, el tiempo que transcurre desde el instante en el que se presenta una dirección a la memoria hasta que el dato, o ha sido memorizado, o está disponible para su uso. Para memorias de otro tipo, el tiempo de acceso es el que se tarda en situar el mecanismo de lectura/escritura en la posición deseada.
- ★ **Caudal:** cantidad de informaciones transferidas por unidad de tiempo (según Meinadier se expresa en mega-informaciones o kilo-informaciones por segundo, por ejemplo “un mega-octeto por segundo”)
- ★ **Capacidad:** Se denomina capacidad de memoria a la cantidad de información que se puede almacenar en ella. La capacidad se puede expresar en bits, bytes o unidades que los agrupen:
 - 1 byte = 8 bits
 - 1 Kb = 1024 bytes = 2^{10} bytes
 - 1 Mb = 1024 Kb = 2^{20} bytes
 - 1 Gb = 1024 Mb = 2^{30} bytes
 - 1 Tb = 1024 Gb = 2^{40} bytes

JERARQUIA DE MEMORIAS



JERARQUÍA DE MEMORIA DEL COMPUTADOR



Un dispositivo de memoria ideal sería aquel que permita una gran capacidad de almacenamiento a bajo costo y con el menor tiempo de acceso. Sin embargo, en la realidad esto no es posible, ya que siempre que se elige un medio de almacenamiento, se limita al menos una de estas características.

Por ejemplo, una memoria rápida es más costosa y tiene menor capacidad. Esto significa que en todo el espectro de posibles tecnologías de memoria existen relaciones de compromiso entre costo, capacidad, y tiempo de acceso.

En el caso del diseño de una computadora se toman en cuenta todas estas consideraciones. Por eso, la memoria total de una computadora está organizada según una jerarquía de niveles de memoria, donde los datos solo se pueden copiar entre niveles adyacentes, cada nivel es accedido con mayor frecuencia que el nivel siguiente inferior y contiene parte de su información.

A medida que descendemos en la jerarquía:

- ★ disminuye el costo por bit
- ★ aumenta la capacidad
- ★ aumenta el tiempo de acceso (es decir que las memorias se hacen más lentas)

El objetivo que se persigue es que el usuario tenga la mayor cantidad de memoria posible con la tecnología más económica, pero con el tiempo de acceso ofrecido por la memoria más rápida. Para el usuario, es como si la computadora tuviera una memoria única, grande y rápida, cuando en realidad se tiene una variedad de tipos de memorias que operan en forma coordinada.

El propósito general de utilizar una jerarquía de memoria es obtener la velocidad de acceso promedio más alta posible, en tanto se minimiza el costo total de todo el sistema de memoria.

Para lograr esto se tiene en cuenta el **principio de localidad**, que dice que en general, es probable que la mayoría de los accesos futuros a la memoria principal, por parte del procesador, sean a posiciones accedidas recientemente (localidad temporal) o a posiciones cercanas a ellas (localidad espacial).

Basándose en lo anterior, **la estrategia que se sigue consiste en organizar los datos y los programas a través de la jerarquía de manera que las palabras de memoria necesarias se encuentren normalmente en la memoria más rápida**. Por ejemplo, la memoria caché guarda las posiciones de memoria accedidas recientemente, junto con otras posiciones cercanas, de manera que el procesador acceda mayormente a ella (que es más rápida) y no a memoria principal (más lenta). De la misma forma, la memoria principal guarda los programas y datos (del disco duro) que se están ejecutando, porque la lectura en disco es mucho más lenta. En todos los casos, influye el principio de localidad, de manera que el procesador tenga un tiempo de acceso a la cantidad de información que necesita leer o escribir lo más bajo posible y a un costo razonable.

Podemos decir entonces que la jerarquía de memorias combina la localidad temporal y espacial con lo que cada tecnología de memorias brinda en cuanto a velocidades de acceso, costo y capacidades de almacenamiento.

Mauro dice que: cada nivel de la jerarquía se corresponde también a un campo de aplicación (área sobre la que va a dar utilidad este grupo de memoria). Por ejemplo: si bien los SSD y HDD difieren en velocidad y costo ambos están pensados como “memoria secundaria”

- ★ En el 1er nivel (**NIVEL 0**) se encuentran los **registros internos del procesador**, que son el tipo de memoria más rápida, pequeña y costosa. Su capacidad es normalmente igual a una palabra del procesador (1 a 8 bytes) Un procesador suele contener unas cuantas docenas de tales registros, aunque algunas máquinas contienen cientos de ellos.

Son memorias de muy pocas palabras y caros de construir (cada punto de memoria posee 6 transistores y 2 capacitores) pero muy veloces. Forman parte de la CPU e interactúan continuamente con esta.

Su campo de aplicación es el hardware. En abacus serian los registros que interactúan con los buses: P, S, AC, M, I.

Constituyen el paso intermedio entre el procesador y la memoria

- ★ En el 2do nivel (nivel 1) está la **memoria caché**, son de pequeña capacidad, normalmente una pequeña fracción de la memoria principal, un tiempo de acceso muy pequeño. **Se colocan entre la CPU y la memoria central**. Su objetivo es guardar una cantidad limitada de información temporal que evite tener que consultar el órgano que lo genera. Normalmente es utilizada para mejorar el rendimiento de la CPU, evita que la CPU acapare todo el tiempo a la RAM, la libera de que tenga que estar continuamente leyendo y grabando, almacenando las informaciones más requeridas. Es como mínimo 5 veces más rápido que la RAM.

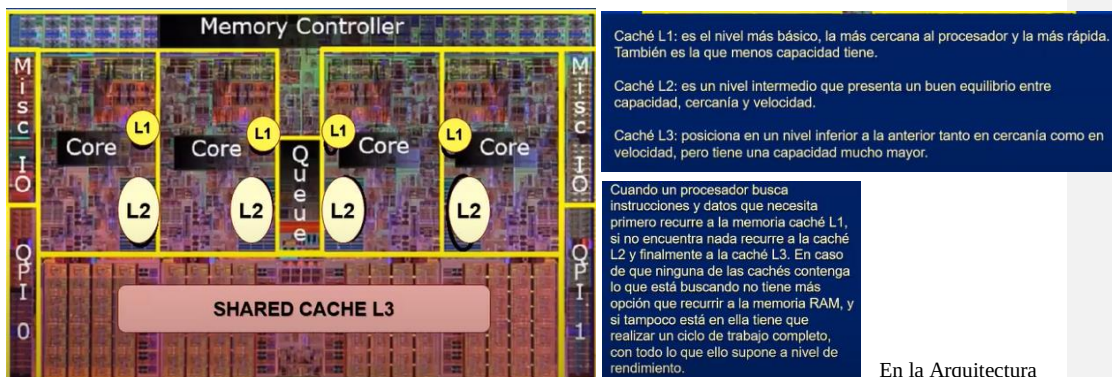
Generalmente no está presente como una sola memoria sino como 2 niveles de caché:

- la caché de **nivel 1 (L1)** dentro del procesador o circuito integrado y de tamaño reducido (guarda cálculos de la CPU y es la que interactúa con la ALU). **Caché on chip**.
- la caché externa **nivel 2 (L2)**, memoria cache en placa, de mayor tamaño (almacena direcciones de memoria, es la que se utiliza para almacenar información disponible en la RAM, y es estática). **Caché on board**. Como si fuese una RAM, peromuchos mas costosa, antiguamente se la colocaba en la motherboard, con el tiempo pierde el sentido desarrollarla así y se las coloca dentro del circuito. Existiendo hoy en día hasta un nivel 4 de cache

Es de alta velocidad, de tecnología SRAM y se emplea para compensar la diferencia de velocidad entre el tiempo de acceso a la memoria principal y el procesador, proporcionándole a éste las instrucciones y los datos utilizados con mayor frecuencia para ahorrarle acceder a memoria.

1. CPU consulta a la caché si tiene el registro necesario (y la caché no lo tiene).
2. CPU solicita el registro a la RAM.
3. La caché obtiene el archivo completo y lo resguarda porque piensa que puede necesitarlo a futuro.
4. Cuando la CPU consulte a la caché por ese mismo registro, ahora la caché si lo tiene.

La CPU solicita solo un registro a la vez, pero la caché almacena el archivo completo para no tener que solicitarlo a la RAM nuevamente, acelerando el procesamiento y mejorando el rendimiento de la CPU.
La CPU consulta a la memoria central a través de la caché.



En la Arquitectura

Harvard se cuenta con una cache para datos y una cache para instrucciones.

- ★ En el 3er nivel (**NIVEL 2**) se encuentra la **memoria principal**, que es el principal sistema de memoria interna de la computadora. Es donde residen los programas y datos. La CPU lee y escribe datos en el aunque con menor frecuencia que los niveles anteriores.
Está compuesta por capacitores, es un circuito sencillo. Tiene un tiempo de respuesta pequeño pero mayor que los niveles anteriores, y gran capacidad.
Es de tecnología DRAM, es más lenta que la caché, pero tiene mucha más capacidad de almacenamiento y es más barata. Al igual que la caché, se comunica directamente con el procesador, y contiene los programas y los datos que el mismo necesita en este momento. Se cree que su diseño físico ya ha llegado a un nivel atómico así que se buscan mejoras queriendo implementar un diseño multicapa 3d para aumentar su capacidad.
Las extensiones de memoria central son memorias de la misma naturaleza que la memoria central que amplían su capacidad de forma modular.

- ★ En el 4to nivel (**NIVEL 3**) se encuentran las **memorias Flash/ USB (EEPROM)**, que poseen una capacidad parecida a la memoria caché. Son de sobre todo lectura, poseen un borrado eléctrico por bytes, y son no volátiles. Están en este nivel porque requieren de otros tipos de memoria para funcionar (y en los niveles anteriores esto no ocurre). Son denominadas así por lo rápido que pueden reprogramarse, se encuentran en coste entre las EPROM y

las EEPROM. Son comúnmente utilizadas en computadoras pequeñas como por ejemplo Arduino, lavarropas o sistemas de automatización.

- ★ En el 5to nivel (**NIVEL 4**) se encuentran los **dispositivos de memoria secundaria** (HDD o SSD), son un tipo de almacenamiento masivo, permanente (no volátil). Residen en dispositivos externos al ordenador y brindan apoyo a la memoria central. Su campo de aplicación fueron los backups ya lamecanmiento de multimedia como películas.

Velocidad/Costo SSD > HDD. Se encuentran en el mismo nivel al pertenecer al mismo campo de aplicación, son un medio de almacenamiento masivo que da soporte a la arquitectura.

SSD: memoria flash no volátil basada en compuertas NAND. Retiene datos sin alimentación eléctrica, son ideales si se requiere un acceso rápido, pero no a persistencia de datos.

- ★ En el 6to nivel (**NIVEL 5**) se encuentran los **discos ópticos** (CD, DVD, Blue-Ray), están hechos de plásticos, rellenos de marcas que un rayo láser puede escribir y leer. Generalmente no pueden ser reescritas, poseen poca capacidad por unidad de memoria (sin embargo, son más baratas que las memorias anteriores). Mientras más marcas, mayor almacenamiento.
- ★ En el último nivel (**NIVEL 6**) se encuentran las **cintas magnéticas**, estas son generalmente utilizadas para realización de backups. Son contenedores plásticos con carretes móviles y cinta magnética sobre la cual se lee a través de inducción electromagnética. Poseen una gran capacidad por un precio muy bajo, sin embargo, el tiempo de acceso es mayor y no pueden ser reescritas.

Las características de las memorias de los niveles 1 y 2 llevan a una solución intermedia que considera las ventajas de cada una: utilizar mayor capacidad de DRAM y almacenar las zonas más frecuentemente accedidas en la memoria SRAM, para acelerar el procesamiento.

Se trata siempre, entre dos niveles adyacentes, de soluciones de compromiso entre costo y velocidad.

Sería deseable poder utilizar solo la memoria más rápida, pero al ser la más costosa se llega a un compromiso entre velocidad y costo, empleando más cantidad de memoria más lenta.

Tipo de memoria	Clase	Borrado	Mecanismos de escritura	Volatilidad
Memoria de acceso aleatorio (RAM)	Memoria de lectura/escritura	Eléctricamente por bytes	Eléctricamente	Volátil
Memoria de sólo lectura (ROM)	Memoria de sólo lectura	No posible	Mediante máscaras	No volátil
ROM programable (PROM)				
PROM borrable (EPROM)	Memoria de sobre-todo-lectura	Luz ultravioleta, chip completo	Eléctricamente	
Memoria FLASH		Eléctricamente, por bloques		
PROM borrable eléctricamente (EEPROM)		Eléctricamente, por bytes		

Memorias Estáticas y Dinámicas (Parhami)

Existen dos tipos fundamentales de memoria semiconductora:

- ★ Memoria de acceso aleatorio (RAM, Random Access Memory)
- ★ Memoria de solo lectura (ROM, Read Only Memory)

El uso de “RAM” para referirse a memorias de acceso aleatorio puede generar confusión, ya que ambas (ROM y RAM) son de acceso aleatorio.

Memorias RAM

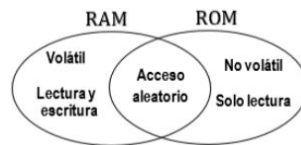
Son memorias de acceso aleatorio, se pueden leer y escribir cuantas veces sea necesario y son volátiles, por lo que necesitan estar siempre alimentadas para no perder la información que almacenan. Por eso, las RAM pueden utilizarse solo como almacenamiento temporal.

Las memorias SRAM y DRAM usualmente se usan para memorias caché rápidas y memoria principal más lenta, respectivamente.

DRAM → Memoria Dinámica → un transistor y capacitor → mas chica

SRAM → Memoria Estática → 4-6 transistores → biestables → circuitos electrónicos

Las **memorias de semiconductores** son aquellas que utilizan un biestable electrónico como punto de memoria. Son circuitos integrados que almacenan bits en celdas binarias constituidas por organizaciones de componentes electrónicos, como transistores y condensadores. Las memorias de semiconductores actuales son al mismo tiempo más rápidas, más densas y más baratas que sus antecesoras. *Tanto SRAM como DRAM son memorias volátiles.*

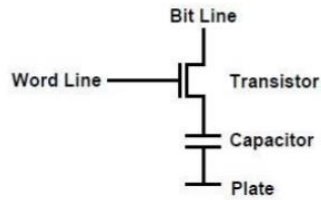


DRAM → memoria principal:

Es imposible construir un elemento biestable sólo con un transistor. Para permitir celdas de memoria de transistor sencillo, que conducen a la mayor densidad de almacenamiento posible en un chip, y a costo por bit muy bajo, la memoria de acceso aleatorio dinámica (DRAM) almacena datos como carga eléctrica en pequeños capacitores, a los que se accede mediante un transistor de paso MOS

- ★ La celda básica de memoria está formada por un transistor y un capacitor (también llamado condensador).
- ★ Los bits se almacenan como cargas eléctricas en los capacitores: la presencia o ausencia de carga se interpretan como el 1 o el 0 binarios.
- ★ El término dinámica hace referencia a esta tendencia a que la carga almacenada se va perdiendo con el transcurso del tiempo, aun cuando están conectadas a la fuente de suministro.

Cuando la línea de palabra se postula, el voltaje bajo (alto) en la línea de bit causa que el capacitor se descargue (cargue) y, por tanto, almacena 0 (1):



Source: ICE, "Memory 1997" 19941

Figure 7-1. DRAM Cell

Escritura:

1. Se abre transistor habilitando el paso de la línea de bit
2. Puerta de transistor:
 - a. Se pone al potencial de alimentación → carga un 1
 - b. Se pone a V(tierra) → carga un 0
3. Este voltaje llega al capacitor el que almacena el bit
4. La línea de palabra es conectada a tierra → para aislar carga del capacitor

Lectura:

1. Precargar la línea de bit a medio voltaje.
2. Luego este voltaje se jala ligeramente abajo o arriba hasta la postulación de la línea de palabra, dependiendo de si la celda almacena 0 o 1.
3. Este cambio de voltaje lo detecta un amplificador, que recupera un 0 o un 1 en concordancia.
4. La lectura destruye el contenido de la celda, se debe seguir inmediatamente con una operación de escritura para restaurar los valores originales (**regeneración**)

Regeneracion:

★ Dado que las cargas de los capacitores se van disipando con el tiempo, las DRAM requieren refresco periódicos cada ciertos milisegundos para reestablecer la carga de sus celdas y mantener memorizados los datos.

La regeneración sucede para todas las hileras de memoria mediante la lectura de cada hilera y su reescritura para restaurar la carga al valor original

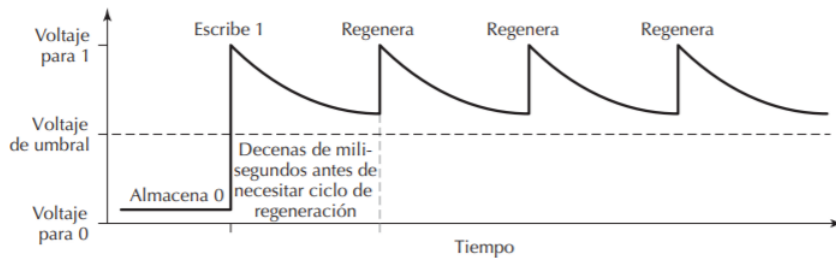


Figura 17.5 Variaciones en el voltaje a través del capacitor de una celda DRAM después de escribir un 1 y subsecuentes operaciones de regeneración.

Cada fila de células es regenerada en todos los ciclos. Por ejemplo, si la especificación de producto muestra “Ciclo de regeneración = 512 ciclos cada 8 ms” entonces hay 512 filas y cada fila individual debe ser regenerada cada 8 ms.

SRAM → cache:

- ★ Cada celda es un biestable (flip-flop), para un bit, diseñado a partir de 6 transistores.
- ★ Al no guardar los bits en capacitores como las DRAM, no necesitan ciclos de refresco y cada celda

permanece inalterable o “estáticamente” en un estado u otro en tanto se mantenga la alimentación eléctrica en la memoria o en tanto no se modifique mediante escritura.

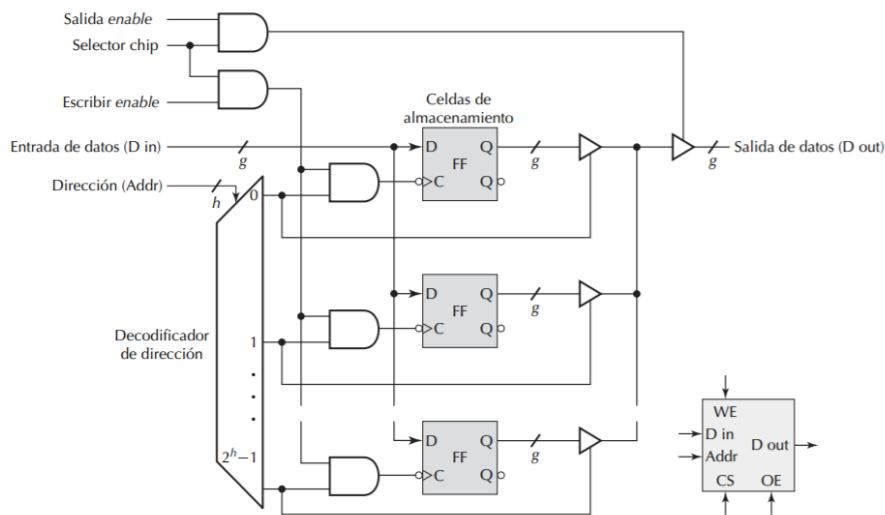


Figura 17.1 Estructura interna conceptual de un chip SRAM $2^h \times g$ y su representación abreviada.

SRAM frente a DRAM:

- ★ Ambas tienen en común que son volátiles, ante pérdida de potencia eléctrica hay pérdida de datos.
- ★ Una celda DRAM es más simple que una SRAM y en consecuencia más pequeña. Por tanto, las DRAM son más densas (celdas más pequeñas = más celdas por unidad de superficie), permitiendo almacenar más datos que una SRAM de tamaño similar, y son más económicas que estas últimas.

- ★ Una DRAM requiere de circuitería adicional para realizar el refresco, aunque en memorias grandes, el coste fijo de la circuitería de refresco se ve compensado por el menor coste de las celdas DRAM. Por su parte, la SRAM consume menos energía al no requerir operaciones de refresco contantes.
- ★ Las SRAM son más rápidas que las DRAM.
- ★ Las DRAM son mucho mas baratas, pero también mas lentas
- ★ Debido a todo lo anterior, las SRAM se usan para memorias cachés, y las DRAM para memoria principal.

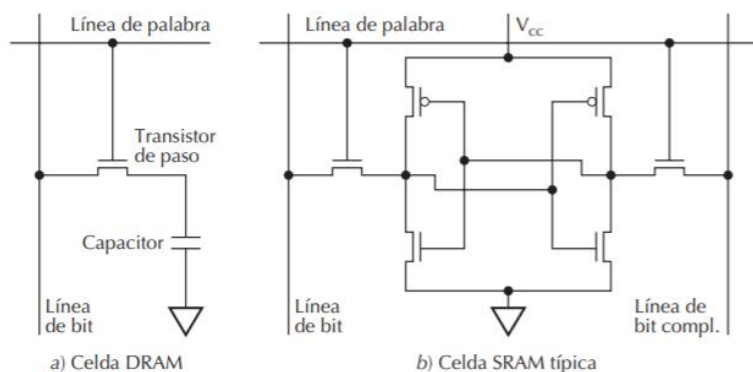


Figura 17.4 La celda DRAM de transistor sencillo, que es más simple que la celda SRAM, conduce a chips de memoria DRAM densos de alta capacidad.

Memorias de semiconductores con "random acces"			
Clase	Tipos	Características de re-escritura	Volatilidad
Memorias de lectura y escritura	DRAM SRAM VRAM	Cada byte direccionado puede ser re-escrito en igual tiempo que se tarda su lectura, cuantas veces se necesite	Son volátiles
Memorias para ser mayormente leída "Read Mostly Memory" (RMM)	EPROM EEPROM Flash ROM	Una vez que la memoria fue escrita, se puede re-escribir ("reprogramar") un número ilimitado o muy grande de veces, pero la escritura es lenta comparada con el tiempo de lectura de un byte..	No son volátiles
Memorias para ser sólo leídas	PROM ROM	Una vez que la memoria fue escrita, no se puede re-escribir	No son volátiles

MEMORIA CENTRAL

La memoria es cualquier dispositivo de almacenamiento de información que forma parte integral de la maquina y esta directamente controlada por ella. De esta manera no consideramos memoria a las cintas y discos magnéticos, pero si una vez que están montados en el dispositivo de lectura. Debe pertenecer a la maquina.

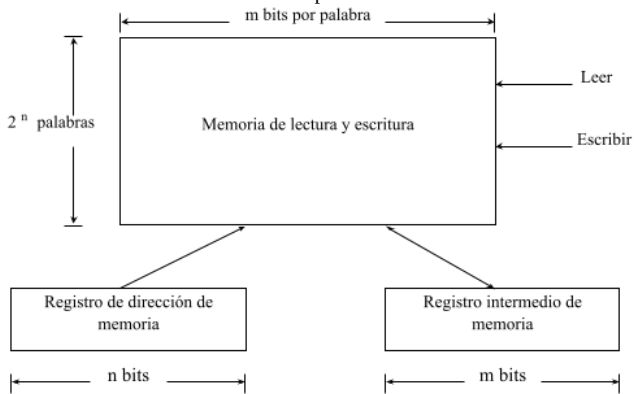
ORGANIZACIÓN GENERAL DE LA MEMORIA

Cuando nos referimos a la **memoria central** hablamos de la RAM.

La memoria está formada por un conjunto de células o registros, capaces de almacenar información (en general una palabra). La información guardada puede ser un dato o una instrucción, sin embargo, las instrucciones están en una zona y los datos en otra.

En cualquier memoria debe existir una **celda básica o punto de memoria**, formada por un elemento de almacenamiento y sus circuitos de control. Para poder usar esta celda en una memoria, deberá poder ser seleccionada mediante el registro de dirección de memoria (RDM o registro S), este registro está formado por n elementos binarios (generalmente flip-flops), en los que 2^n es el número de palabras que pueden ser almacenados en la memoria. Y se debe tener un método para controlar cuando las celdas seleccionadas deban ser leídas o escritas. Cada célula, deberá entonces, contener un número que la identifica, su dirección, que será habilitada por la unidad de control. Esta habilitación permite leer o escribir información contenida en una dirección determinada.

Además, toda la memoria debe estar conectada a un bus de datos y un bus de direcciones que permita vincularla con el resto de la arquitectura. 2 elevado al tamaño del bus S nos indica cuantas palabras pueden ser direccionadas y el tamaño del bus M nos indica el tamaño de la palabra de memoria



PUNTO DE MEMORIA

La información más elemental registrada es el bit; a cuyo soporte físico llamamos punto de memoria (almacena un bit). El punto de memoria debe poseer circuitos electrónicos (FF) de manera que pueda ser perfectamente definido e individualizado y constará, además del dispositivo de almacenamiento, de los de lectura y escritura.

Para las memorias actuales el punto de memoria siempre es un biestable

Los elementos de almacenamiento pueden ser:

- Electrónico – F. F.
 - Eléctrico – condensador (DRAM)
 - Magnético – Punto magnetizable (núcleo ferrita)
 - Deformaciones de superficie – Discos ópticos
- } Memoria Central

} Memoria Auxiliar

Lectura – Escritura

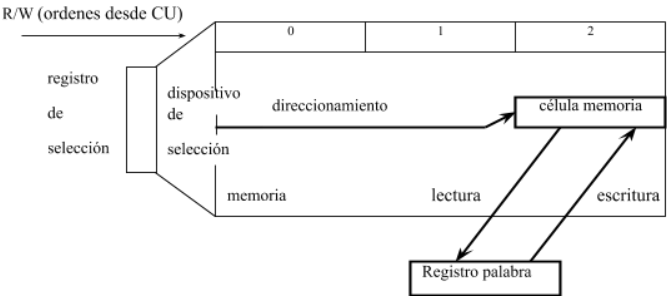
El dispositivo de selección de memoria analiza el contenido del registro de dirección y sensibiliza la célula implicada, bien para una lectura o para una escritura.

Si se tratase de una lectura, primero se limpia registro de palabra, y luego la información almacenada en la célula sensibilizada será transferida a este registro.

El registro de palabra de memoria (RPM) está asociado a la memoria central, tiene tantos elementos de almacenamiento binario como bits en cada palabra de memoria (m bits).

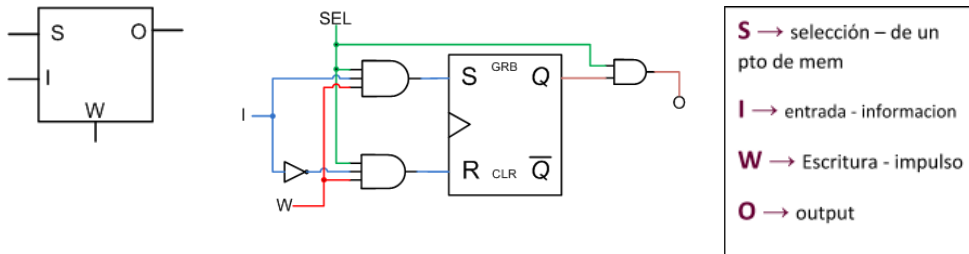
En el caso de una escritura, previamente habrá sido preciso cargar el RPM con la información que se quiera transferir a la célula de memoria en cuestión. Se dice a la memoria que se va a escribir, por medio de un 1 en la línea de escribir. La memoria, entonces almacenará el contenido del RPM, en la posición especificada en el registro de dirección de memoria (DIR).

La operación de lectura no destruye la información almacenada en la célula (lectura no destructiva, en el caso de memoria de semiconductores). La operación de escritura destruye la información almacenada, sustituyéndola por una nueva información.



ORGANIZACIONES DE MEMORIA

ORGANIZACIONES DE MEMORIA | 2 D



Se diseña sobre el plano, compuesta de n-palabras de longitud fija y un decodificador de direcciones que permite el acceso directo a cada una de sus palabras

El decodificador tiene una entrada por cada bit de la dirección que decodifica (las dos salidas de cada flipflop de arriba se consideran como una sola que se complementa). Cantidad de AND equivalente a la cantidad de palabras de memoria.

Tendra 2 elevado a la cantidad de entradas, líneas de salida, una para cada valor que pueda tomar el RS.

Ya sea que se quiera leer o escribir una palabra de memoria, lo que debe hacerse primero es enviar la dirección de dicha palabra al RS, para que el decodificador active la línea de salida correspondiente a esa palabra y se seleccionen las compuertas AND 1,2 y 3 de sus puntos de memoria.

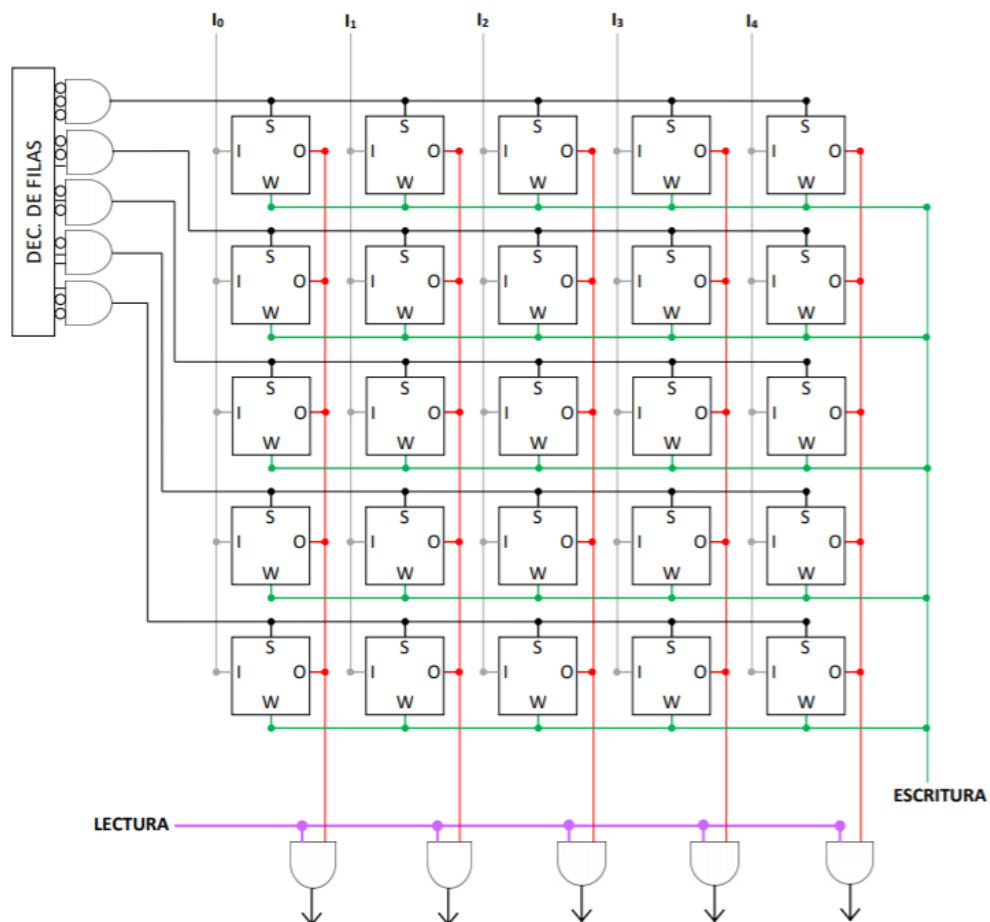
- ★ **Lectura:** se debe activar la línea LEER, para que los valores de los puntos de memoria de la palabra seleccionada previamente sean transferidos a las líneas de salida, que pueden ir, por ejemplo al registro de palabra de memoria (RPM). **Se usan los hilos O, output**
- ★ **Escritura:** La palabra a escribir en memoria está en las líneas de información I, que pueden ser, por ejemplo, salidas del RPM. Una vez seleccionada la palabra a escribir, la habilitación de la línea W (que llega a todos los puntos de memoria a la vez, pero sólo tendrá efecto sobre la palabra direccionada) provocará que los puntos de memoria seleccionados tengan sus 3 entradas activadas (selección + información + escritura), lo que causará el almacenamiento de la información en los flip-flop de la palabra. **Se usan los hilos I, input.**

→ Ventajas:

- Punto de memoria mas simple (por la lógica combinacional asociada es mas simple)

→ Desventajas:

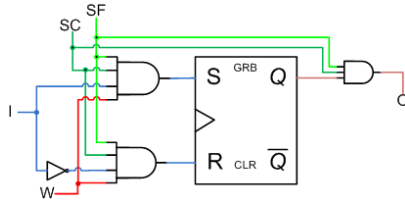
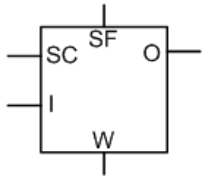
- El deco se vuelve cada vez mas complejo conforme aumente el tamaño de la memoria. Este tipo de decodificadores se construyen usando diodos (o transistores) para las compuertas AND. El número de diodos (o transistores) usados para cada compuerta AND es igual al número de entradas a cada compuerta.
- No aprovecha el diseño de circuitos multicapa.



Cantidad de palabras = PAL			
Bits por palabra = B			
Cantidad de puntos de memoria = PAL * B = PM			
Bits para direccionar las palabras = $\log_2 \text{PAL} = N$			
DECODIFICADOR	PUNTOS DE MEMORIA	LECTURA	ESCRITURA
AND	PAL	PM * 3	N
ENT	PAL * N	PM * 8	AND * 2

ORGANIZACIONES DE MEMORIA | 3D

Punto de memoria



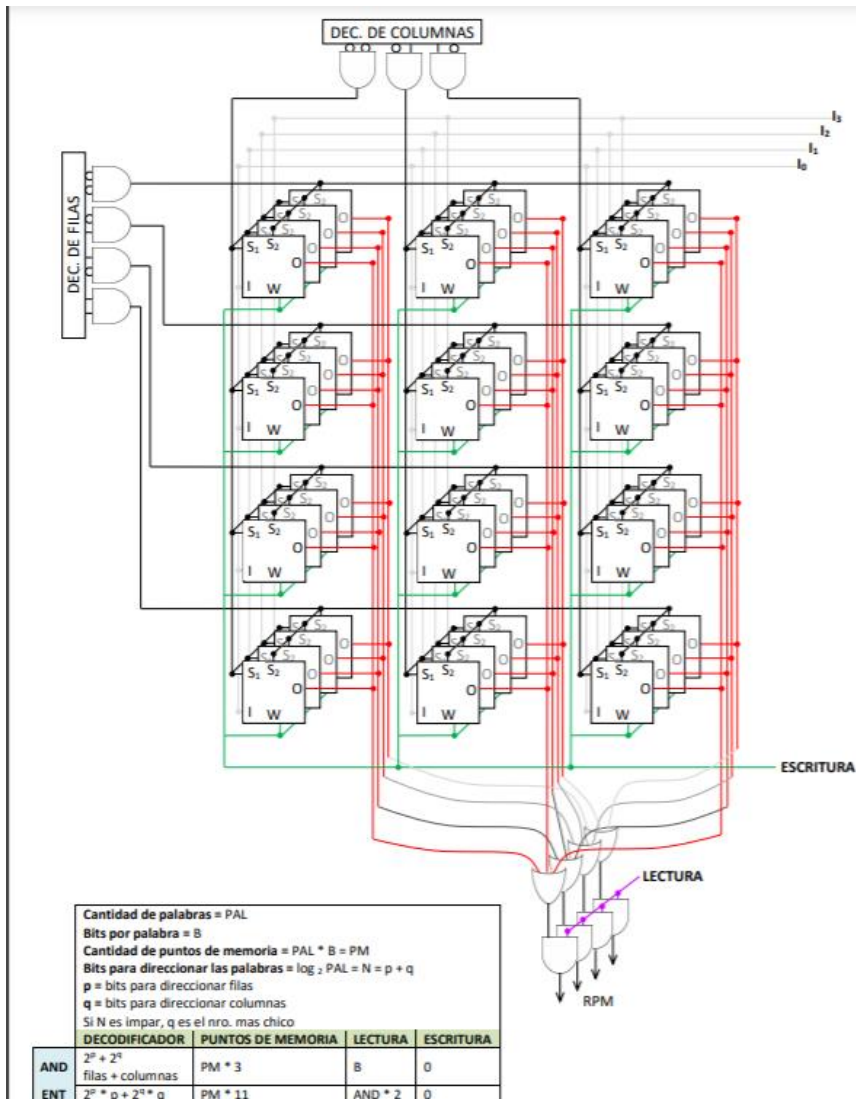
Se divide al decodificador y ahora hay uno para las columnas y uno para las filas. Por lo tanto debemos agregar una línea de entrada al punto de memoria, de manera que ahora habrá dos líneas de selección.

→ **Ventajas:**

- Implementación del diseño multicapa, lo que economiza espacio.
- Decodificador se divide, simplificándolo. Resultando en dos decodificadores simples, en vez de uno solo muy complejo.
- Mantiene esta simplicidad aun en memorias de gran tamaño.

→ **Desventajas:**

- Punto de memoria mas complejo y caro (por la lógica combinacional asociada, se agrega una entrada)



Planteamos una memoria compuesta de dos decodificadores uno para la ubicación columna y otro para las ubicaciones de fila de cada punto de memoria, es decir cada plano posee una matriz de puntos de memorias asociadas a dos decos. Deco X (llamamos al de filas) y Deco Y (columnas) que permite ubicar de manera individual a un solo punto de memoria . Para esto se agrega una nueva entrada al punto de memoria

A su vez la longitud de la palabra esta dada por la cantidad de capas o planos de puntos de memoria. Permitiendo definir a una palabra de manera espacial. La palabra se forma por aquellos puntos de memoria que en cada plano tienen igual valor de X e Y.

La capacidad de la memoria esta dada por *cantidad de filas * cantidad de columnas* .

Para la lectura agregamos una AND por columna. Y una OR por cada plano.

ORGANIZACIONES DE MEMORIA | 2 ½ D

Combina lo mejor de la organización 2D y 3D , con una lógica combinacional agregada para las operaciones de lectura y escritura

Punto de memoria de 2D

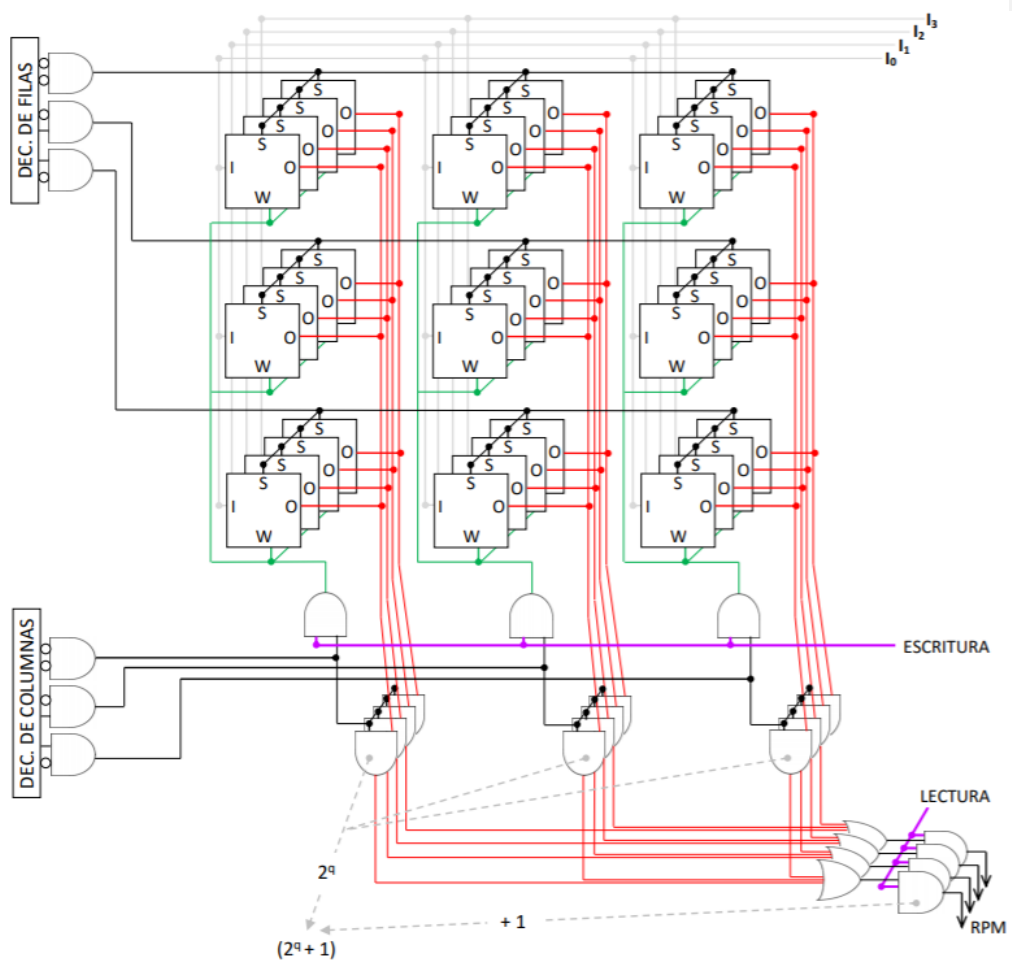
Organización (decodificador) de 3D

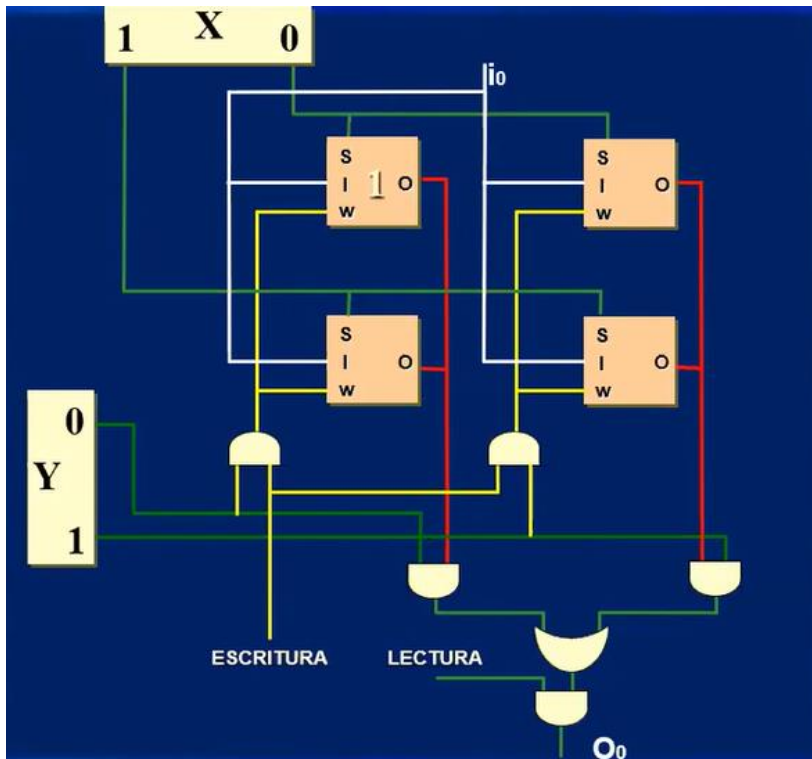
- ★ Deco X gobierna filas.
- ★ Deco Y gobierna columnas, pero a nivel del gobierno de la operación en curso. Es decir, en la columna una compuerta AND maneja la escritura y otra AND maneja la lectura. El proceso de selección surge de seleccionar una fila a través de X en S y la columna a través de Y pero sensibilizando W para la escritura u O para la lectura. Por lo tanto la habilitación para la escritura no se sitúa a nivel de los biestable sino al de las líneas de selección de columna

Se necesita una and para la **Y** para gobernar escritura de columna y una para gobernar lectura de columna.

Todas las and que miran para abajo serán la salida de la lectura.

- ➔ **Ventaja:** mas económica que la 3D pero igual de eficiente
- ➔ **Desventaja:** lógica combinacional asociada para mantener que haya una sola línea de selección en el punto de memoria se agrega a cada columna una and para la lectura y una and para escritura. Es decir la selección de la columna no llega al biestable





MEMORIA DE NUCLEO MAGNETICO

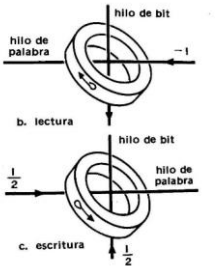
Anillo de ferrita

Material que contiene partículas de óxido de hierro. Se puede magnetizar conservando este estado durante bastante tiempo. Tiene forma de anillo porque mejora su característica magnética.

En contraste con un FF semiconductor que solo necesita una cantidad fca (por ej voltaje), para su operación un núcleo magnético emplea tres cantidades físicas (cte, flujo magnético y voltaje). La señal es un pulso de corriente que pasa a través del núcleo.

La información binaria se representa por la dirección del flujo magnético dentro del núcleo.

La propiedad física que hace que un núcleo magnético sea adecuado para el almacenamiento binario es su circuito de histéresis. Por eso, una magnetización contraria al sentido de las agujas del reloj, flujo positivo, se usa para representar un 1; y la otra un 0.



Se eslabonan tres alambres a cada núcleo.

- ★ Alambre de palabra: se excita por un impulsor de palabra, pasa por los núcleos de una palabra (una fila).
- ★ Alambre de bit: se excita por un impulsor de bit, que pasa por los núcleos de las mismas posiciones de bit (una columna).
- ★ Alambre sensor: se aplica una amplificador sensor que forma el pulso del voltaje cuando se lee un 1 y rechaza la pequeña perturbación cuando recibe un 0

Proceso de lectura

Es destructiva ya que para averiguar el estado del anillo es necesario aplicar una corriente que lo desmagnetice (la corriente es generada por un impulsor DR) dejando a los núcleos en estado 0, por lo tanto, el primer ciclo se dedica a desmagnetizar la palabra y el segundo deberá remagnetizar aquellos puntos que tenían un 1.

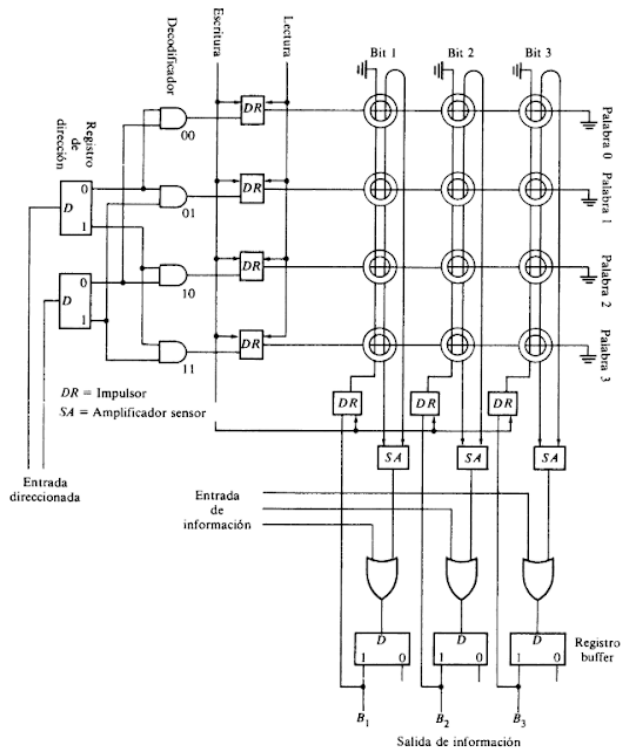
Primero se aplica una corriente en dirección negativa, si el núcleo tenía un 1 la corriente invierte la dirección y el cambio en el flujo produce un cambio de voltaje en el alambre sensor y si está en estado 0 lo mantiene en el mismo. La información de salida pasa por un amplificador de señal (SA) cuyas salidas establecen los flip-flops correspondientes en el registro buffer.

Un pulso de corriente del impulsor de palabra se aplica a los núcleos de la palabra seleccionada por el decodificador lo que causa que todos los núcleos pasen al estado 0 sin importar su estado previo. El pulso de voltaje en un alambre sensor de núcleos con un 1 previo se amplifica en el SA y establece el FF correspondiente en el registro buffer.

Escritura

Consiste en inducir un campo magnético sobre el anillo para que este quede con una carga remanente que indica un uno.

El registro buffer mantiene la información por almacenarse en la palabra especificada, se supone que todos los núcleos en la palabra seleccionada están en estado cero, se genera en forma simultánea un pulso de corriente en el impulsor de palabra seleccionado por el deco y en el impulsor del bit, cuyo FF del registro buffer contiene un 1. Ambas corrientes están en la dirección positiva pero su magnitud es solo la mitad de la necesaria para cambiar el flujo al estado 1. **Esta media corriente es demasiado pequeña por sí misma para cambiar el estado de magnetización, pero las dos corrientes si son suficientes para cambiar el estado.**

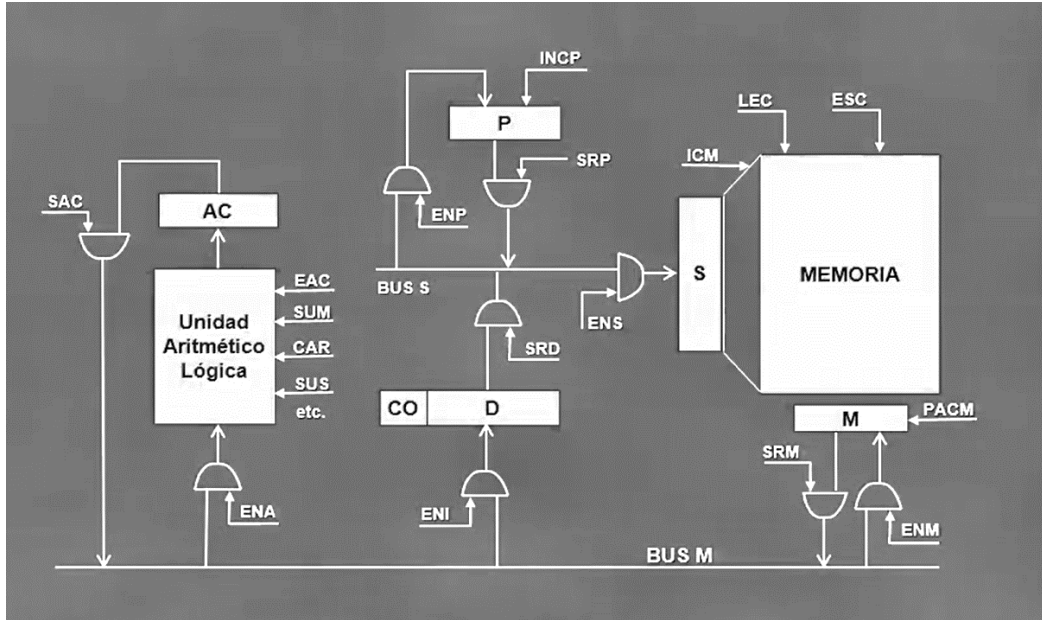


Problema de implementación

El hecho de enhebrar cada punto con un hilo de lectura y uno de escritura produce un circuito muy complejo y entramado. Porque para una memoria de un Kb o sea 1024 palabras de 8 bits se requieren 8192 anillos y 16384 hilos. Entonces surge el concepto 2D que soluciona este inconveniente. La técnica de transferencia de registro con previa puesta a 0 es para memoria núcleo de ferrita. Y la transferencia o carga forzada es para memoria de semiconductores

UNIDAD DE CONTROL

Como vimos la Unidad Central Proceso, incluye la Unidad de Control y la Unidad Aritmética Lógica.



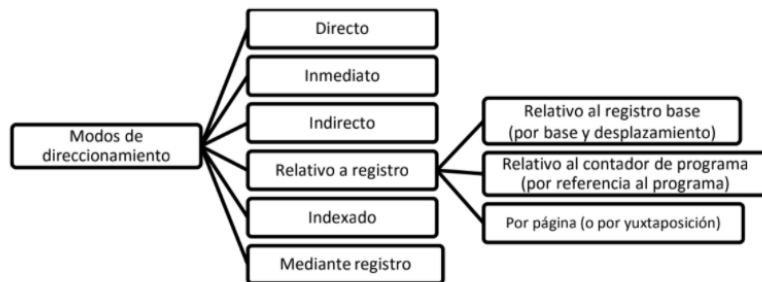
DIRECCIONAMIENTO

Son distintas maneras que provee una arquitectura para acceder a una palabra de memoria en un ciclo de lectura.

Todo diseño de memoria no está completo si no cuenta con alguna estrategia de selección de palabra que permita ubicarla para leerla o escribirla, basada en una condición específica.

Las diferentes técnicas de direccionamiento equivalen a una transformación de la parte de dirección de la instrucción en la dirección que se transferirá finalmente al registro de selección en memoria para obtenerla información deseada. Llamamos a esta última dirección efectiva.

La instrucción en curso a través de un campo el tipo de procesamiento que debe sufrir el contenido de la zona de dirección (D) viene especificado generalmente por el campo de condiciones de direccionamiento (CD). El campo CD (condición de direccionamiento) establece una estrategia según el código que selecciona o ubica una palabra, después veremos si se leerá el contenido de esta dirección o se almacenará algo en ella.



El campo CD puede tomar tres valores, indicando cada una:

- Direccionamiento indirecto
- Direccionamiento directo
- Direccionamiento inmediato

El resto no se indican el campo CD, sino solamente a usar un registro índice como ejemplo en indexado.

Direccionamiento Directo

Permite seleccionar una palabra de memoria cuya dirección sea coincidente con el valor binario del campo DIR de una instrucción.

El campo DIR de la instrucción contiene la dirección efectiva del operando. Exige un solo ciclo de memoria para obtener el operando.

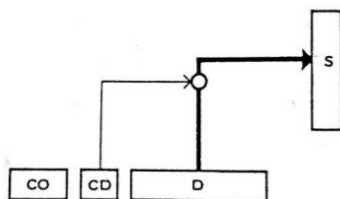
Mencionaremos las ventajas y desventajas de cada direccionamiento, pero comparar las ventajas entre direccionamientos es el ilógico, cada modo es una estrategia dependiendo el programa.

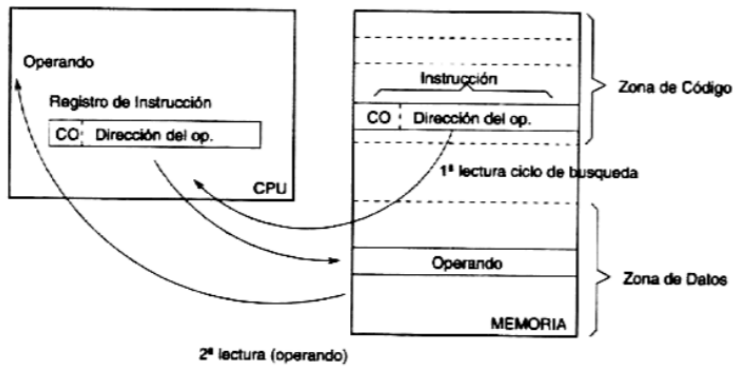
→ **Ventaja:** Su sencillez, porque no necesita ningún cálculo previo para obtener la dirección efectiva.

→ **Desventaja:** El tamaño de la dirección efectiva esta limitado por la cantidad de bits del campo DIR.

Si éste fuera el único modo de direccionamiento, la zona de dirección debería tener suficiente longitud como para poder direccionar toda la memoria, alargando demasiado el registro de instrucción.

(DIR) → S | 1 solo acceso a memoria





Direccionamiento Indirecto

El campo DIR indica la dirección de la dirección efectiva

La parte de dirección de la instrucción contiene, no la dirección de la información pedida, sino la dirección de una palabra de memoria donde se encontrará la dirección efectiva del operando.

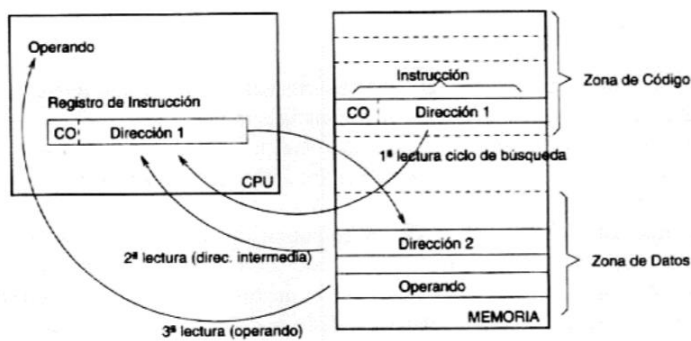
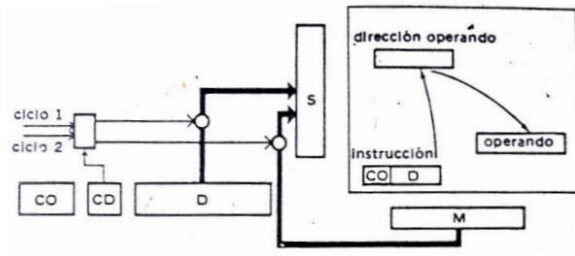
→ Ventaja:

- **Permite acceder a zonas mucho mas altas de la memroia**
- Ya no existe limitación en el conjunto de direcciones accesibles según el tamaño del campo DIR. El problema del direccionamiento directo era que la longitud del campo de dirección de la instrucción es normalmente menor al número de bits necesarios para direccionar toda la memoria, por lo que se limitaba el rango de direcciones accesibles.
- Con direccionamiento indirecto, la primera palabra buscada puede contener una dirección que ocupe todo el ancho de la palabra (que será enviada directamente al registro S), y que me permita acceder más allá del rango posible solo con la zona D (de menor longitud).
- Permite el manejo de estructura de datos como arreglos a través de punteros. Almacenando en DIR los punteros y pudiendo así encadenar datos. Un recorrido con esta técnica ya no es físico, sino lógico.

→ Desventaja:

- La ejecución de la instrucción requiere dos referencias a memoria para obtener el operando: una para obtener su dirección, y otra para obtener su valor.

((DIR)) | Por lo menos dos accesos a memoria



En ABACUS, el direccionamiento indirecto utiliza la señal de nivel **TMS**, transferencia entre buses, que lleva el contenido del BusM al Bus S. Esta señal se activa en el segundo ciclo, solo es usada para este direccionamiento.

Direccionamiento Inmediato

No es propiamente un direccionamiento, puesto que el campo DIR no contendrá una dirección de memoria, sino que el mismo ya almacena el operando.

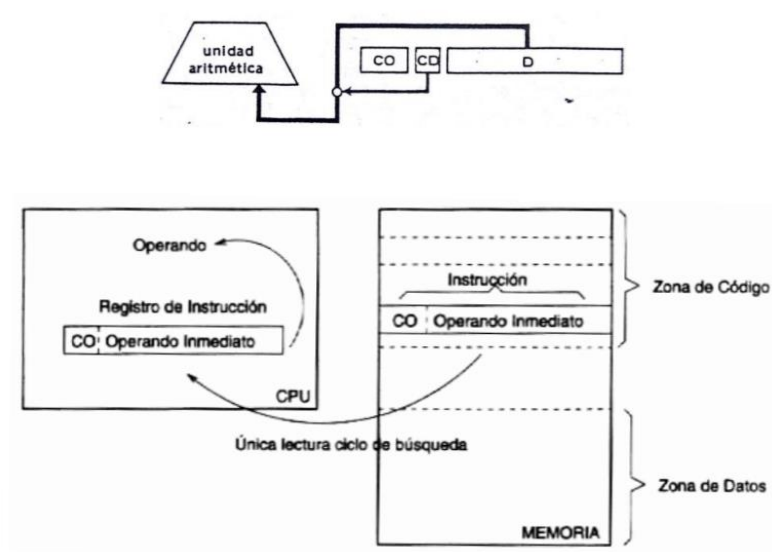
Por esta razón no consume ciclo de memoria para obtener el operando.

→ Ventaja:

- Una vez cargada la instrucción, no se requiere un nuevo acceso a memoria para obtener el operando, ahorrándose de esta manera un ciclo de memoria en la ejecución de la instrucción.

→ **Desventaja:** El rango de valores de un operando inmediato queda limitado por el tamaño del campo empleado por la instrucción para almacenarlo, que en la mayoría de los repertorios de instrucciones es pequeño comparado con la longitud de la palabra.

(DIR) → ALU | ningún acceso a memoria



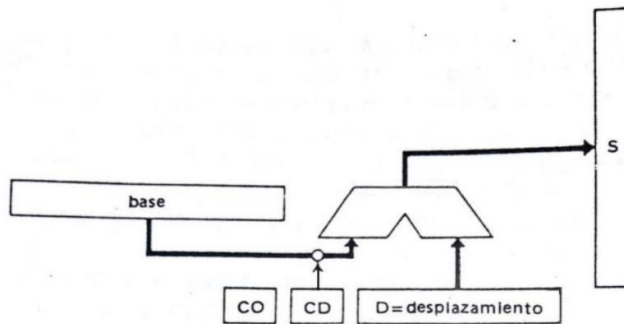
Direccionamiento Relativo

Veremos tres tipos de direccionamiento relativo donde el campo DIR siempre contiene *desplazamientos*.

Direccionamiento Relativo al Registro Base (por base y desplazamiento)

Un registro de la máquina, llamado registro base contiene la dirección de referencia (primera dirección de un programa o de una zona de datos, por ejemplo). A la información que guarda la parte de dirección de la instrucción se la llama desplazamiento.

La dirección efectiva es la suma de la base y el desplazamiento. Dado que las computadoras pueden disponer de varios registros que actúen como base, la instrucción debe tener, además del desplazamiento, un campo para seleccionar el registro base a usar, además de operaciones para inicializar el valor del registro base o puede ser un valor fijo establecido en la memoria .



→ **Ventaja:**

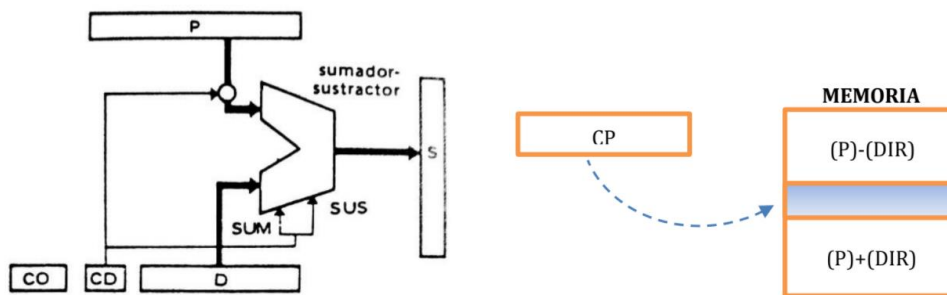
- Al cargar un valor en el registro base, luego solo tenemos que cargar desplazamientos en el campo DIR. Facilitando la programación de bajo nivel en lenguaje de maquina.

→ **Desventaja:**

- El calculo consume un tiempo adicional al tiempo de la lectura del operando

Direccionamiento por referencia a programa

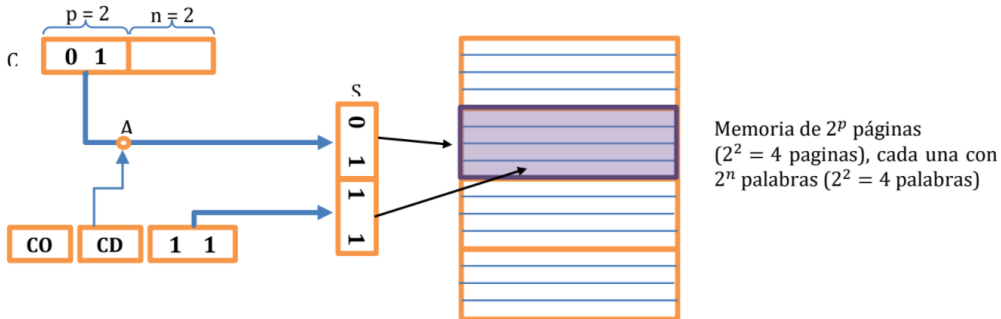
Usa como puntero al registro contador de programa, cuyo contenido sirve como la dirección de referencia. Con este sistema es posible generalmente direccionar secuencialmente dos zonas de memoria (por encima y por debajo de la instrucción en curso), según que la parte de dirección de la instrucción se suma o se resta con el contenido del contador de programa. **lectura secuencial del operando.**



Direccionamiento por Pagina (o por yuxtaposición)

Se considera a la memoria dividida en 2^p zonas de 2^n palabras cada una llamadas páginas. En general, la parte de dirección de la instrucción contiene n bits, por lo que no permite direccionar más palabras que las que contiene una página (que son 2^n palabras).

La dirección efectiva se obtiene por yuxtaposición (concatenación) del número de página, dado por los primeros p bits del CP, y de la parte de dirección de la instrucción (n bits), que me da la palabra dentro de esa página. El campo DIR dará la dirección de la palabra dentro de la página seleccionada por P.



Si la puerta A esta cerrada, hay direccionamiento directo en la primera página (00) y si está abierta, hay direccionamiento dentro de la página dada por los primeros p bits del CP. En este caso, el direccionamiento por página puede considerarse como un direccionamiento por referencia al principio de la página en curso (es decir, como el CP dice la instrucción en curso, sus primeros p bits dirán la página de dicha instrucción en curso).

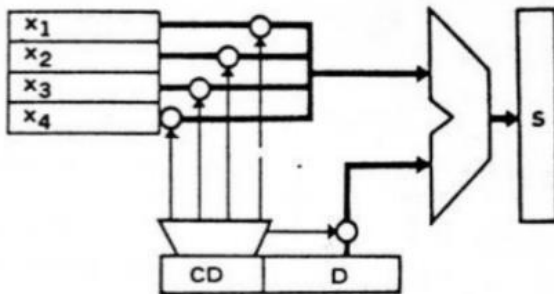
Esta técnica promueve el manejo de multiples procesos a través de sistemas operativos, ya que cada pagina puede manejar un proceso diferente, el que será atendido por la CPU en función a técnicas propias del sistema operativo. **Fundamental para que los sistemas operativos puedan implementar multiprogramación**

Direccionamiento Indexado

El mecanismo de direccionamiento es exactamente el mismo que para base-desplazamiento, con la diferencia significativa de que el valor del registro índice se suele modificar con frecuencia en la ejecución de un programa, mientras que el base no. La dirección efectiva se obtiene sumando la parte de dirección de la instrucción con el contenido del registro índice.

Debemos entonces indicar no solo la condición de direccionamiento sino también el numero del índice. Además se precisara de operaciones que inicializan el índice y que decrementan o incrementan su valor

Se usa habitualmente como mecanismo para recorrer estructuras de datos tipo vector o tabla. La dirección especificada en la instrucción es la de la primera posición del vector y a ella se le suma el valor del índice inicialmente puesto a cero e incrementado o decrementado de manera sucesiva dentro de un bucle. Cuando se llega al final de bucle, se compara el índice con el número de elementos del vector y se salta al principio del bucle si quedan elementos por procesar.



RESUMEN: Direccionamiento

Modos de Direccionamiento: Estrategias que brinda una arquitectura para acceder a una palabra de memoria

★ **Directo**

DIR efectiva = (DIR) | (DIR) → S
1 acceso a memoria

★ **Indirecto**

DIR efectiva = ((DIR)). Usa señal TMS
2 accesos a memoria

★ **Inmediato**

Operando esta en campo DIR | (DIR) → ALU
No hay acceso a memoria

★ **Base y Desplazamiento**

DIR efectiva = (Registro Base) ± (DIR)
El Dir indica el desplazamiento.

★ **Por referencia a programa**

Idem Base y desplazamiento
Dir efectiva = (CP) ± (DIR)

★ **Yuxtaposición**

Yuxtaponer → concatenar → & | **DIR efectiva = bits mayor peso CP & (DIR)**
CP dividido en P bits de mayor peso y n bits restantes
P bits → numero de pagina
N bits → DIR dentro de la pagina

★ **Indexado**

RUTA DE DATOS

Es el conjunto de órganos de la máquina que interviene para transferir memorizar y procesar las informaciones(instrucciones, direcciones y operandos) procedentes de la Memoria Central.

Elementos de la Ruta de Datos:

- Buses de interconexión: para la transferencia de información
- Registros y memorias. Para su almacenamiento
- Unidades Funcionales: para su procesamiento

Las Señales de gobierno están distribuidas por toda la ruta autorizando las diferentes operaciones. Se las llama **microórdenes**, y estas son generadas por el secuenciador

EL SECUENCIAMIENTO DE LAS INSTRUCCIONES

SECUENCIADOR CENTRAL

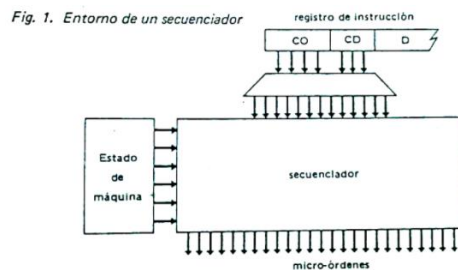
El secuenciador central de una computadora es el órgano que **genera las microórdenes, encargadas de materializar la instrucción en curso**, estas son distribuidas a lo largo de la ruta de datos para activarla y gobernar sus diversos elementos constituyentes.

Las microórdenes se convierten en señales de gobierno para manejar la ruta de datos

Entradas y salidas del secuenciador

Las **informaciones de salida** del secuenciador son las **microórdenes** que se distribuyen por la ruta de datos según diferentes cronogramas(en el caso de secuenciador cableado).

Las **informaciones de entrada** al secuenciador son suministradas, por un lado, por la **instrucción** (código de operación, condiciones de direccionamiento y las direcciones de registros), por otra parte, por el **estado de la máquina**, que agrupa un cierto número de informaciones como son indicadores de error, demandas de interrupción, indicadores del estado de la memoria y de las unidades funcionales, código de condición de la ALU, etc.



Secuenciador síncrono y asíncrono

El secuenciamiento de las microórdenes en el tiempo puede llevarse a cabo de dos maneras distintas:

- ★ **Forma síncrona:** el secuenciador conoce los tiempos de respuesta de los diferentes órganos gobernados. Entonces provoca los retardos apropiados entre las diferentes órdenes sucesivas de manera que las operaciones tengan tiempo de ejecutarse. **Las microórdenes van generalmente sincronizadas con los impulsos del reloj.** Por

ejemplo, si una operación necesita poco más de 2 batidos de reloj, el secuenciador esperará al tercer batido después del lanzamiento de la operación para enviar la siguiente microórden.

- ★ **Forma asíncrona:** el secuenciador recibe de los diferentes órganos señales indicando que han terminado las operaciones ordenadas y que se liberan. El secuenciador solamente lanzará una nueva operación después de haber sido avisado de que las operaciones precedentes fueron ejecutadas completamente y de haberse asegurado que el órgano que debe realizarla está libre. Por ejemplo, usará el resultado de una multiplicación solo después de que el operador de multiplicación le haya enviado una señal validando dicho resultado.

Secuenciadores cableados y secuenciadores microprogramados

Existen dos formas fundamentales de abordar el diseño de una unidad de control:

- ★ **Diseño cableado:** En una implementación cableada, la unidad de control es esencialmente un circuito combinacional donde sus señales lógicas de entrada se transforman en un conjunto de señales lógicas de salida, que son las señales de control o microórdenes. Desarrolla el cronograma a través de circuitos.
- ★ **Diseño microprogramado:** corresponden a la introducción en la unidad de control de una memoria con un pequeño programa, llamado microprograma, para cada instrucción. El desarrollo de este microprograma genera las microórdenes que gobiernan la ejecución de la instrucción. El cronograma está almacenado en una memoria.

SECUENCIADOR CABLEADO

Principio del secuenciamiento

El objetivo es generar microórdenes distanciadas en el tiempo por retardos apropiados. Por ejemplo, el inicio de la instrucción de almacenamiento en Abacus, suponiendo que la dirección de la instrucción se encuentre sobre el bus S será efectuado por la cadena de retardos:

Los retardos serán ajustados para acomodarse a los tiempos efectivos de las operaciones o las subidas de nivel correspondientes. Las señales no impulsionales permanecerán activas hasta anulación (por ejemplo, SRM será anulada, después del retardo θ_1 , por la señal $\overline{SRM}$).

En una máquina asíncrona, tendremos el siguiente esquema, en donde las compuertas AND indican simbólicamente que no se pueden generar las siguientes microórdenes hasta que las operaciones anteriormente ordenadas no hayan terminado.

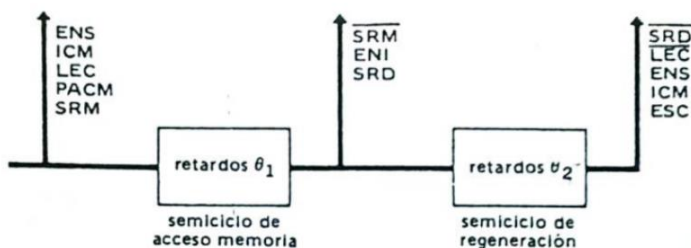


Fig. 3. Búsqueda en memoria (solución síncrona)

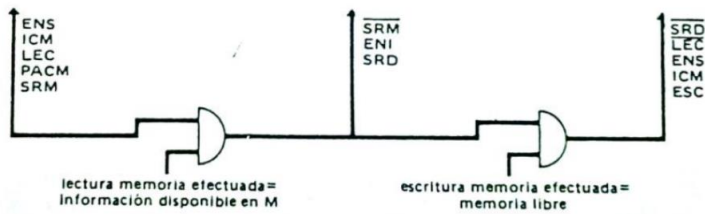


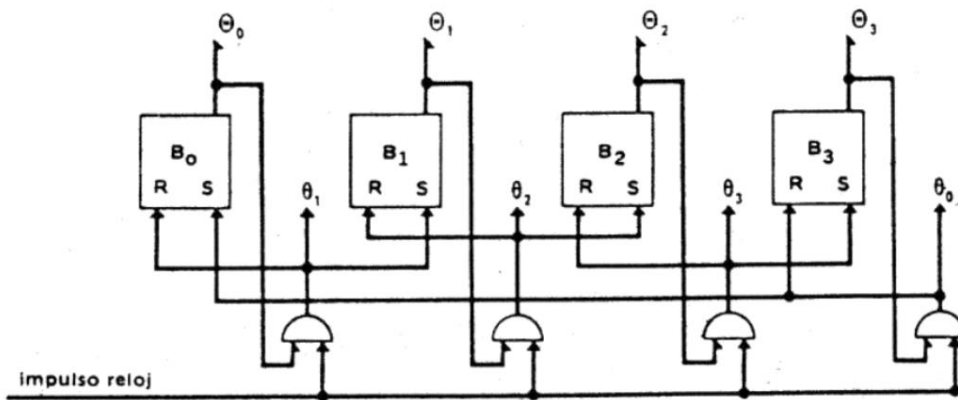
Fig. 4. Búsqueda en memoria (solución asincrónica)

El distribuidor de fases

En una máquina síncrona, las microórdenes se sincronizan con señales temporales regularmente espaciadas, provenientes de un mismo reloj. Ante cada impulso de reloj debe ser generado un cierto número de microórdenes.

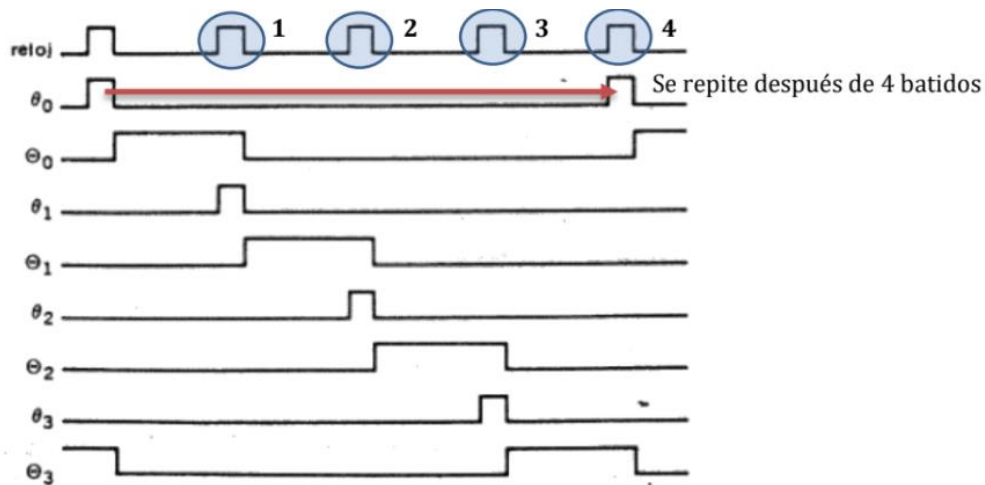
Si las señales de reloj son todas idénticas, ¿cómo el secuenciador se sitúa en el tiempo? es decir por ejemplo ¿cómo sabe en qué fase del ciclo de instrucción se encuentra? Esta es la función del distribuidor de fases, que **partiendo de los impulsos periódicos del reloj, produce señales impulsionales y de nivel.**

El distribuidor de fases es un circuito secuencial parte del secuenciador el cual convierte las microórdenes en señales impulsiones y de nivel en base a las ecuaciones lógicas que definen cada microinstrucción.

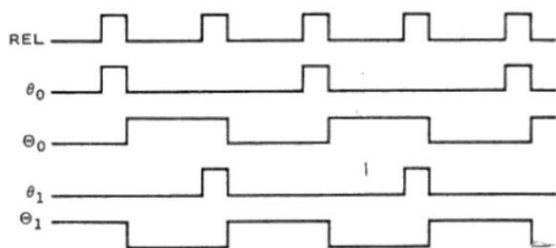


- En un instante determinado, solamente uno de los biestables B0, B1, B2, B3 está posicionado a 1.
- El próximo impulso de reloj devolverá este biestable a 0 y posicionará el siguiente biestable a 1.
- Los niveles lógicos $\phi_0, \phi_1, \phi_2, \phi_3$, serán ciertos uno a continuación del otro, pero solo uno en un momento dado.
- Los impulsos $\theta_0, \theta_1, \theta_2, \theta_3$, sincronizados con las señales de reloj, se producirán cada cuatro batidos.

El cronograma del distribuidor de 4 fases de arriba es el siguiente:



En el caso de Abacus, como el ciclo de memoria equivale a dos batidos de reloj, empleamos un distribuidor de dos fases, cuyo cronograma es el siguiente:



El distribuidor de 2 fases de Abacus me permite ubicarme temporalmente en **1 solo ciclo de memoria**, ya que en el siguiente se repiten las señales θ_0 , θ_0 , θ_1 , θ_1 , pero no me dice si estoy en ciclo de instrucción o de operando. Lo que me indica en cuál de los dos estoy serán los **biestables de estado** (I de *instrucción* y O de *operando*), como se ve más adelante. El distribuidor de 4 fases de arriba **no necesitaría** de dichos biestables.

Relación DF y secuenciador → Genera una única pareja de señales impulsionales y de nivel para cada fase necesaria y definida de las instrucciones de maquina en sus cronogramas.

Diseñado para dar soporte al mayor n° de fases requeridos por la arquitectura. Indicando así en que fase del ciclo se encuentra. Es tarea luego del secuenciador derivar estas señales a cada compuerta de gobierno de la ruta de datos en cumplimiento al cronograma de estado de la instrucción en curso.

Señales de impulso → tita

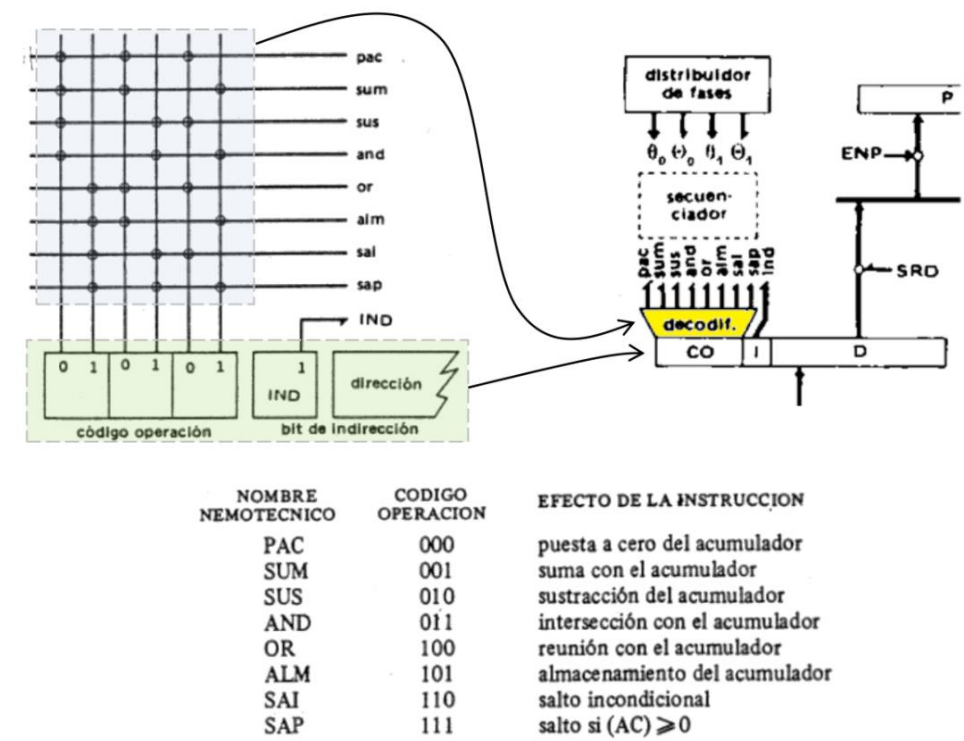
Señales de nivel → phi

Decodificación de la instrucción

La instrucción consta de diferentes campos de información que son de interés para el secuenciador, como son el código de operación, las condiciones de direccionamiento, la dirección de registros o de unidades periféricas, etc. Algunos campos,

como el CO deben ser decodificados, aunque otros ya se presentan en esa forma, como por ejemplo las condiciones de direccionamiento cuando cada bit tiene su significado.

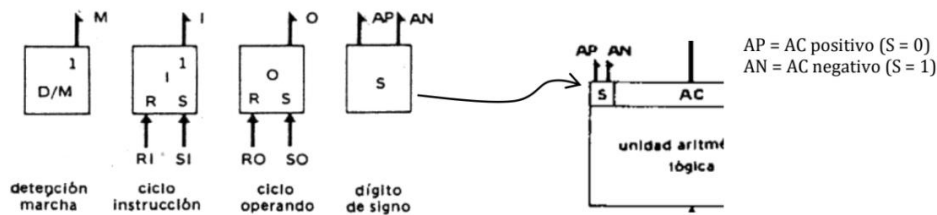
El decodificador de la instrucción se encarga de realizar la decodificación del CO. Para esto puede usarse una matriz de diodos o de transistores. Por ejemplo, para Abacus, dotado de las operaciones indicadas en la siguiente tabla y capaz de direccionamiento indirecto, se necesitaría el decodificador de la siguiente figura:



Las señales salidas de la matriz permanecen posicionadas todo el tiempo que la instrucción se encuentra en el registro de instrucción.

Biestables de estado

Los hay de dos clases:



- Los primeros memorizan el estado de la máquina. Son posicionados por las instrucciones y los resultados de operaciones, o también por acontecimientos exteriores, petición de ciclo de memoria, interrupción, etc. Sus salidas figuran entre las informaciones de entrada al secuenciador.
- Los segundos constituyen parte integrante del secuenciador. Es éste quien los posiciona y quien utiliza su salida. Esencialmente sirven de marcas temporales. En cierto modo, los biestables del distribuidor de fases podrían incluirse dentro de esta categoría.

Abacus consta de los cuatro biestables representados en la siguiente figura:

Ecuaciones Lógicas y Cronogramas

En los cronogramas se define instrucción por instrucción las microórdenes a ejecutar en cada momento para materializar a dicha instrucción.

Luego para las ecuaciones lógicas, tomamos una a una cada microorden, buscando las eventualidades en las que debe ser activadas, mirando los cronogramas de todas las instrucciones. A partir de esto se deduce una expresión lógica que relaciona estas eventualidades, que no son otra cosa que entradas del secuenciador.

Trazado de cronogramas

Hasta acá, solamente definimos las informaciones de **ENTRADA** al secuenciador (biestables de estado, señales del distribuidor de fases, código de operación decodificado + bit de indirección).

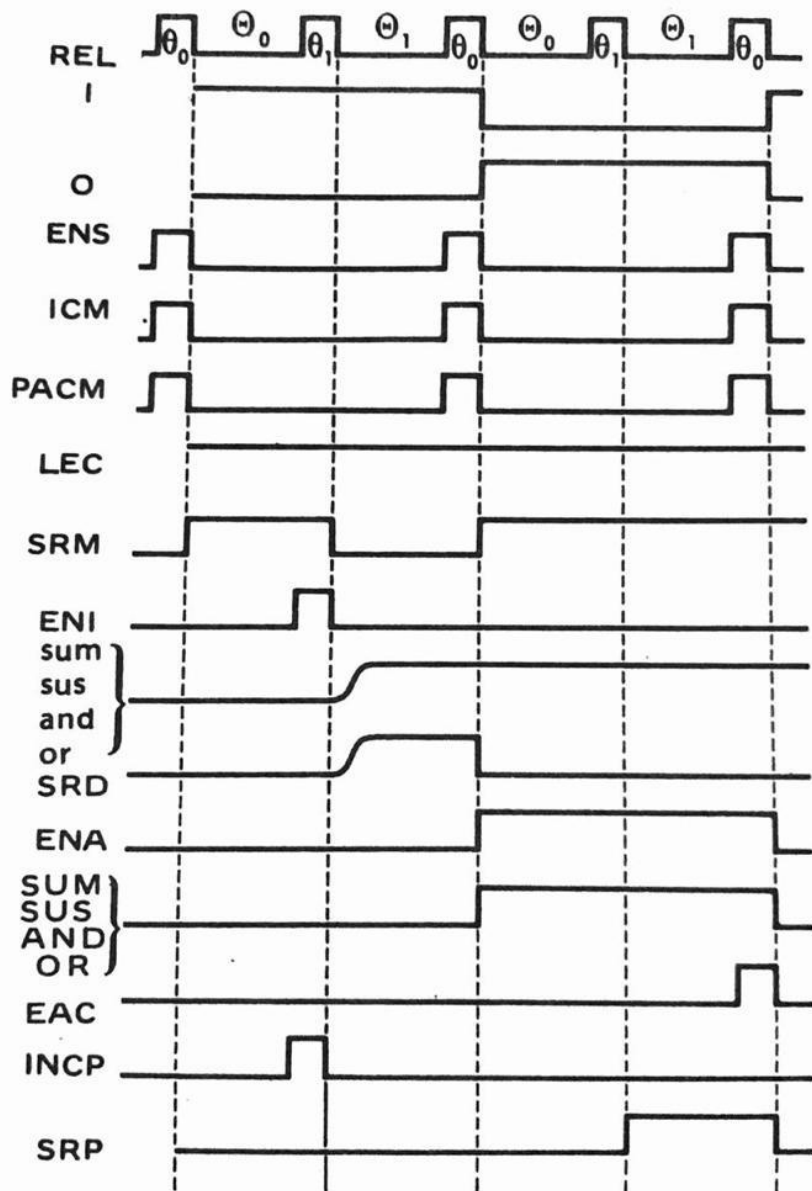
Ahora debemos tratar las **SALIDAS** del secuenciador, que son las microórdenes (SRM, ENI, ENS, etc.)

- ➔ Una **primera etapa** consiste en trazar el cronograma de cada instrucción, y de este cronograma se deducir las microórdenes que deben activarse a cada batido de reloj. Esto nos ayuda a ver las microórdenes que deben activarse y en qué momento.
- ➔ Una **segunda etapa** consiste en relacionar la generación de las microórdenes de los cronogramas anteriores, no sólo con los batidos de reloj, sino también con las distintas informaciones de entrada y estado del secuenciador, dando lugar a las ecuaciones lógicas de la máquina, cuyos circuitos asociados definen completamente el secuenciador.

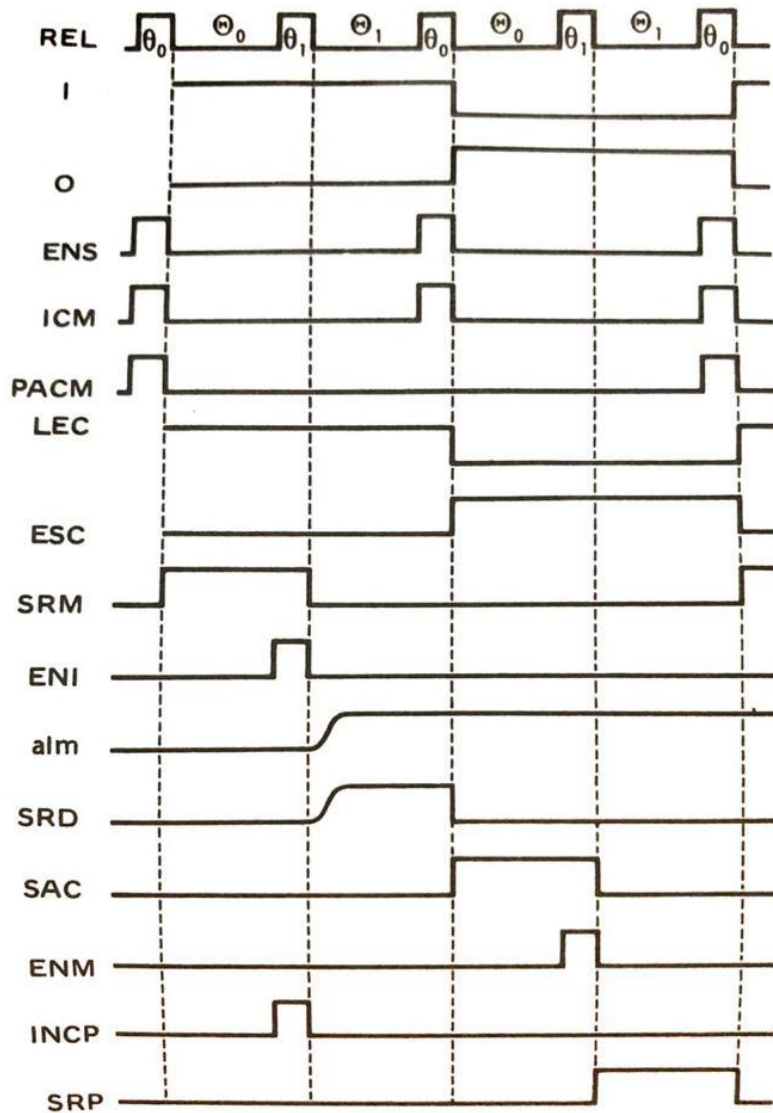
En lo siguiente hacemos lo anterior tomando como ejemplo a Abacus, que está provisto de un distribuidor de 2 fases, del decodificador de instrucciones y los biestables de estado definidos anteriormente. Se supondrá que el funcionamiento del reloj está condicionado por la señal de detención-marcha, y que la puesta en marcha inicializa el distribuidor de fases.

El objetivo final es obtener las ecuaciones lógicas del secuenciador, que por cada microórden de la ruta de datos, definen la combinación de entradas: bits de estado (I, O, AN y AP), CO decodificados (sum, sus, and, or, alm, sap, sai, pac) más bit de indirección, y señales del distribuidor de fases ($\theta 0$, $\theta 0$, $\theta 1$, $\theta 1$) que debe cumplirse para activar esa microórden.

CRONOGRAMA SUM/SUS/AND/OR



CRONOGRAMA DE ALMACENAMIENTO



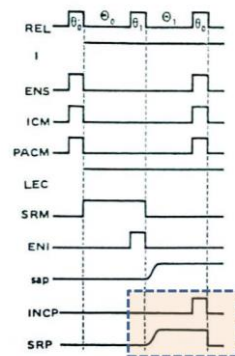
Las instrucciones sum/sus/and/or realizan estas operaciones entre el contenido del acumulador y el operando que se encuentra alojado en memoria en la dirección del campo DIR de la instrucción.

La instrucción de almacenamiento almacena el contenido del acumulador en la dirección de memoria indicada por el campo DIR de la instrucción

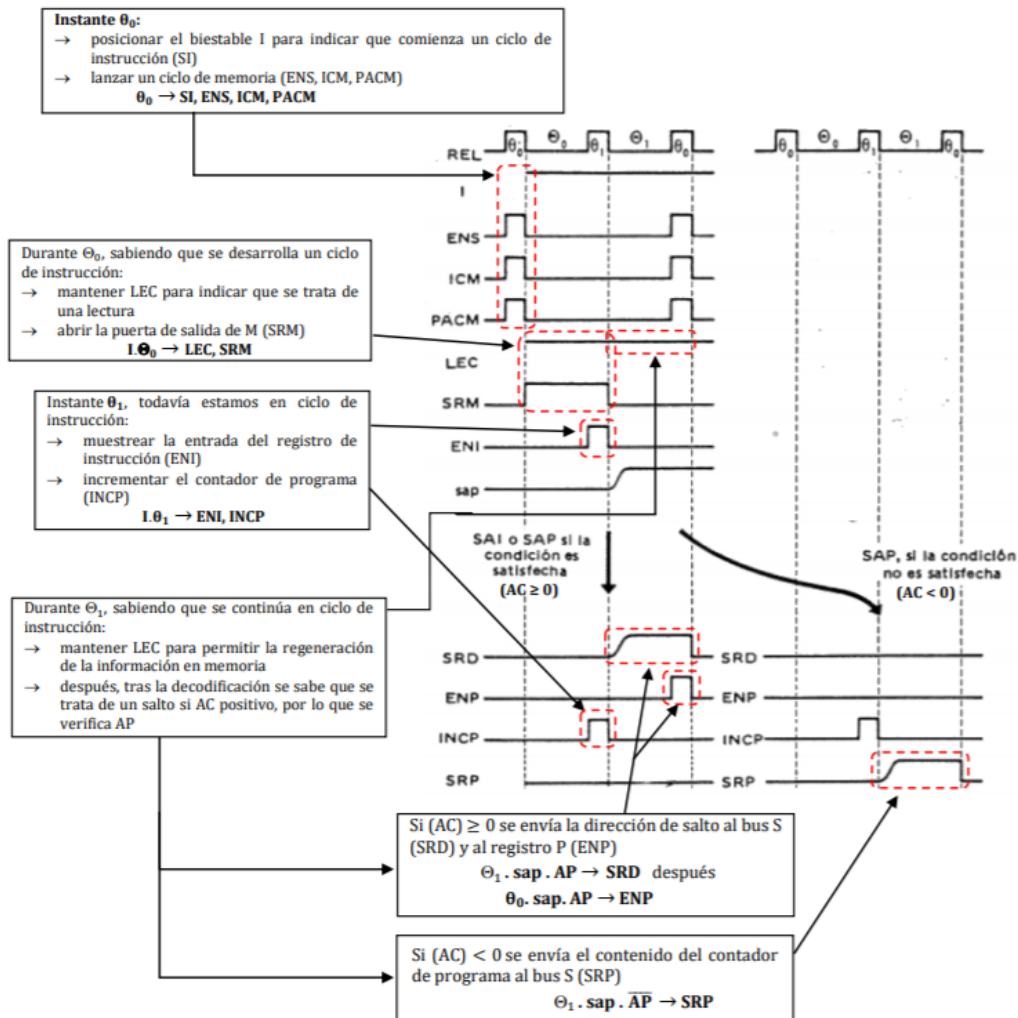
Observación (Meinadier pag 178): debemos observar que en todas las instrucciones se incrementa **sistemáticamente** el contador ordinal cuando estamos en ciclo de instrucción en el instante θ_1 (que se traduce en la ecuación lógica $INCP = I \cdot \theta_1$), **incluso** en el caso de SAP, más arriba, cuando se cumple $(AC) \geq 0$ (en ese caso se incrementa P al pedo porque luego la dirección de la nueva instrucción es reemplazada por la dirección de salto).

Lo último pasa porque en θ_1 aun no se sabe si la instrucción se tratará de un salto, ya que en ese instante recién se muestrea la instrucción presente en el bus M en el registro de instrucción mediante ENI para decodificarse.

La razón de Meinadier para hacer INCP en ese instante y no más tarde es porque en las instrucciones de un solo ciclo (como PAC o SAP en el caso que no se salte) sería **demasiado tarde** hacer INCP luego de θ_1 , porque SRP, que se ejecuta luego de θ_1 en estos casos, llevaría al bus S la dirección de la instrucción actual, y no de la siguiente. Es decir, pasaría lo siguiente:



CRONOGRAMA SAP (salto si acumulador positivo) y SAI (salto incondicional)



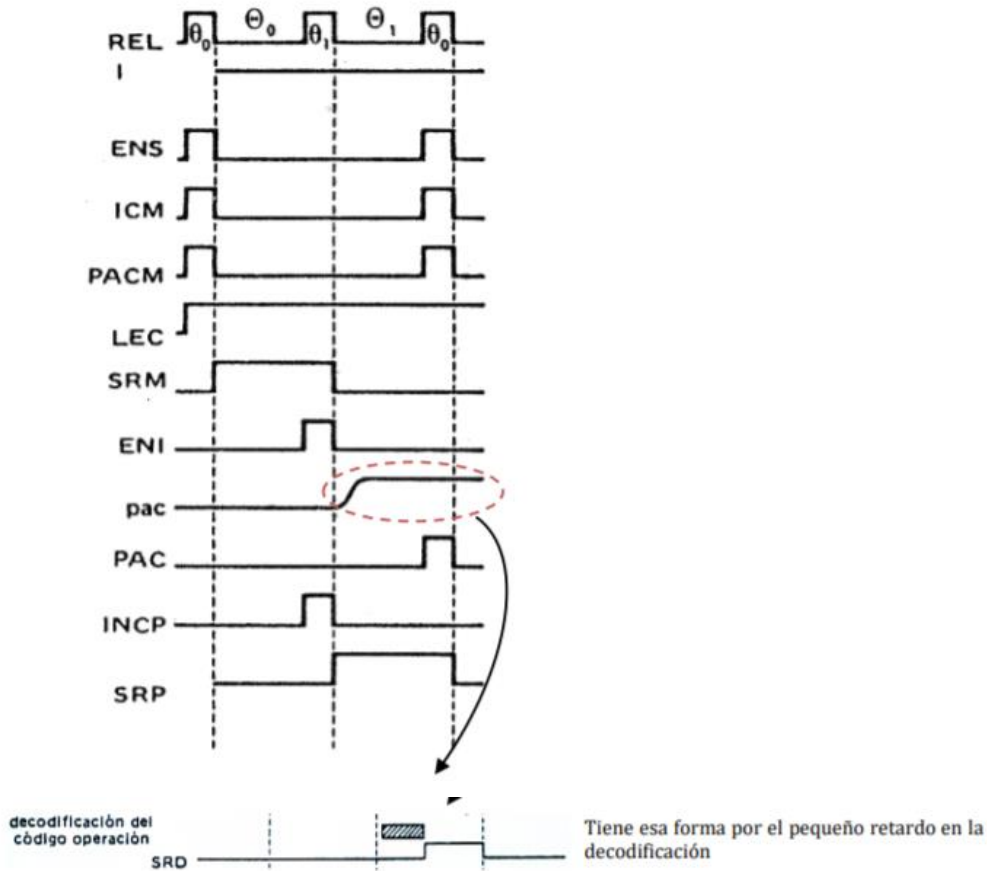
SAP cuando la condición se cumple lo que hace es interrumpir la secuencia normal del programa, la próxima instrucción no será la que indique el contador de programa, sino la dirección indicada en el campo DIR de la instrucción.

El cronograma para SAP con condición satisfecha es el mismo cronograma que SAI y estos toman un solo ciclo de memoria, es decir dos batidos de clock. Si la condición no se cumple si ocupa los dos ciclos.

En el cronograma, las microórdenes ENS, PACM, ICM, LEC, SRM, ENI, SRD, ENP, INCP y SRP son las que deben activarse para ejecutar la instrucción SAP (de lo que habla la “primera etapa” más arriba).

CRONOGRAMA PUESTA A CERO

Cronograma de la puesta a cero del acumulador Cr



CRONOGRAMA CICLO INDIRECTO

Es el único que ocupa 3 ciclos de memoria → 6 batidos de clock

1er ciclo: CICLO BUSQUEDA INSTRUCCIÓN – I = 1 | O = 0 –

Dir a la dirección de la palabra donde esta alojada, no la dirección efectiva sino la dirección del operando

2do ciclo: CICLO DE INDIRECCION – I = 0 | O = 0 –

- Se mantiene todo el ciclo la señal IND
- A penas se lee la dirección del operando se lo deriva al BUS S (INDIRECCION) - por lo tanto en tita4 tambien hay ENS – ICM – PACM
- Se mantiene habilitada la señal SRM (M) → I

- Se usa señal TMS, para que la info en el BUS M vaya directo al RS

3er ciclo: CICLO BUSQUEDA O ALMACENAMIENTO DE OPERANDO – I = 0 | O = 1 –

ECUACIONES LOGICAS

En las ecuaciones lógicas, pasa al revés que con los cronogramas, donde definíamos, instrucción por instrucción, las microórdenes a ejecutar en cada momento. Ahora lo que hacemos es tomar una a una cada microórden, buscar todas las eventualidades en que debe ser activada (mirando en los cronogramas de todas las instrucciones) y deducir la expresión lógica que relaciona estas eventualidades, que no son otra cosa que entradas al secuenciador.

ENS (entrada a RS) – ICM (inicio ciclo de memoria) – PACM (puesta a cero RPM):

$$ENS = ICM = PACM = \theta_0$$

Deben activarse siempre en θ_0, θ_2 y θ_4 , pero debido a que ABACUS trabaja con un Distribuidor de Fase de dps fases, se deduce que deben estar activos en el instante θ_0 de TODOS los cronogramas, tanto en fase de Instrucción como de Operando

ENI (entrada al registro instrucción) – INCP (incremento contador de programa):

$$ENI = INCP = \theta_1 \cdot I$$

I → biestable ciclo de instrucción

Son señales de tipo impulsional que solo deben generarse en el ciclo de instrucción.

En realidad no sucede INCP en θ_1 sino en θ_3 , pero al usar un DF de dos fases no existe θ_3 , por eso es necesario indicar .I

SUM – SUS – AND – OR (operaciones de la ALU)

Salida del deco en el ciclo de operando

Son señales de nivel que deben sostenerse a lp largo de $\phi_0 + \phi_1$ y para cada operación en particular dentro del ciclo de operando. La expresión $\phi_0 + \phi_1$ se elimina ya que la operación dura las dos fases completas durante las cuales esta activo el biestable O

$$SUM = sum \cdot O \mid SUS = sus \cdot O \mid AND = and \cdot O \mid OR = or \cdot O$$

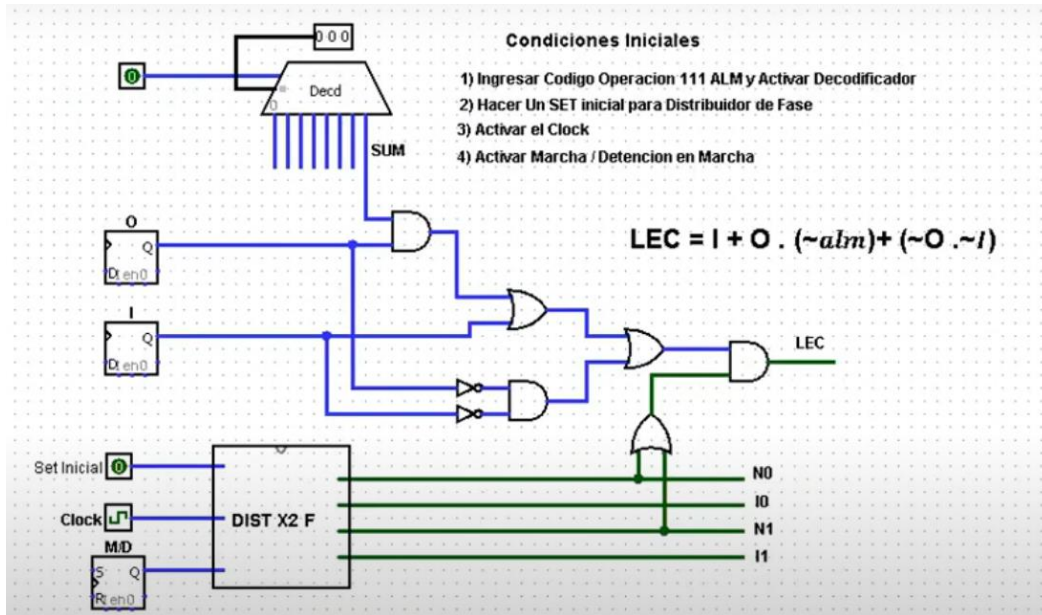
LEC (lectura)

La señal de lectura es de nivel y ,usualmente, se debe dar todo el espacio de tiempo entre el ciclo de Instrucción y de operando

- ★ Siempre se da en el ciclo búsqueda de instrucción
- ★ Se da en el ciclo de operando también con excepcion del cronograma de almacenamiento (donde en el ciclo O se hace una escritura → solo se da en operaciones del tipo IBO
- ★ También se mantiene activa durante el ciclo de indirección (I = O = 0)

$$LEC = I + O \cdot \overline{alm} + \overline{I} \cdot \overline{O}$$

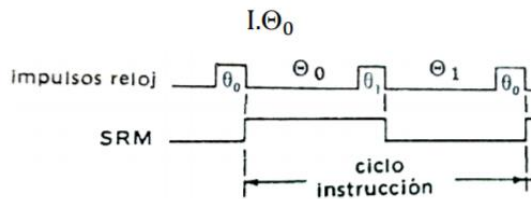
Ejemplo LEC para suma:



Para hacer los circuitos uno las dos señales de nivel en una OR y lo llevo a una AND con la salida de la señal SRM (salida del contenido del RPM al BUS M):

SRM es una señal de nivel que se da en 3 situaciones diferentes:

1. Se leyó una instrucción y se la quiere llevar al RI. En ciclo de instrucción durante el intervalo $\phi_0 \rightarrow \phi_0 \cdot I$



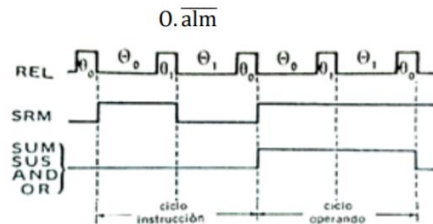
2. En ciclo de operando de las instrucciones con búsqueda de operando (SUM, SUS, OR, AND) durante todo el tiempo que dura la operación en la ALU, para mantener latente al operando en el BUS M ya que las operaciones en la ALU se hacen entre el BUS y el AC:

$$O \cdot (\theta_0 + \theta_1) \cdot (\text{sum} + \text{sus} + \text{or} + \text{and})$$

Pero como $(\theta_0 + \theta_1)$ cubre el intervalo en que O está activado, valdrá siempre 1, por lo que no se lo expresa.

$$O \cdot (\text{sum} + \text{sus} + \text{or} + \text{and})$$

Como las instrucciones implicadas son todas aquellas que necesitan un ciclo de búsqueda operando, no incluye la operación ALM, (en realidad tiene ciclo de operando, pero no se usa SRM) por lo que tendremos:



3. En el direccionamiento indirecto, en el ciclo de búsqueda de la dirección efectiva del operando y mientras dura la señal Θ_1

$$IND \cdot \Phi_1$$

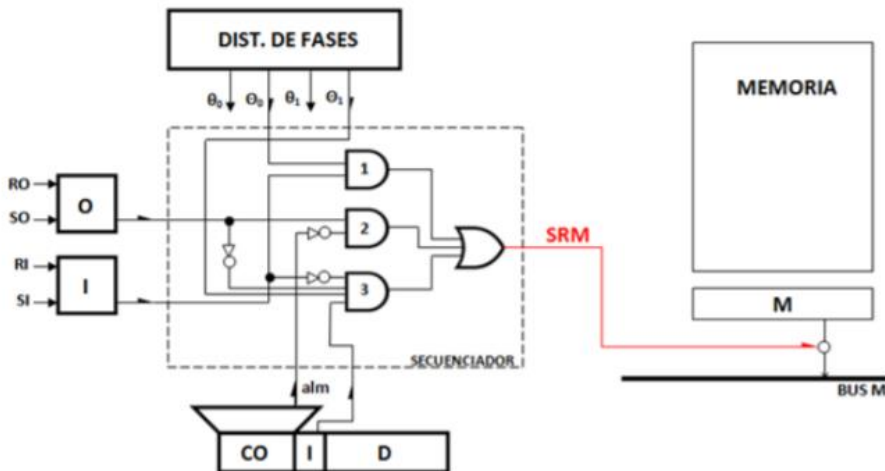
Pero IND puede estar activo también durante ciclo de instrucción o de operando. Para asegurar que sea el Θ_1 de ciclo de indirección, usamos el hecho de que, durante ese lapso, tanto I como O son cero:

$$IND \cdot \Phi_1 \cdot (\overline{O} \cdot \overline{I})$$

Como son los únicos casos en que se activará la señal SRM, es decir en las microoperaciones $(M) \rightarrow I$ (caso 1), $(M) \rightarrow ALU$ (caso 2), $(M) \rightarrow S$ (caso 3), la ecuación lógica completa será:

$$SRM = I \cdot \Phi_1 + O \cdot \overline{alm} + IND \cdot \Phi_1 \cdot (\overline{I} \cdot \overline{O})$$

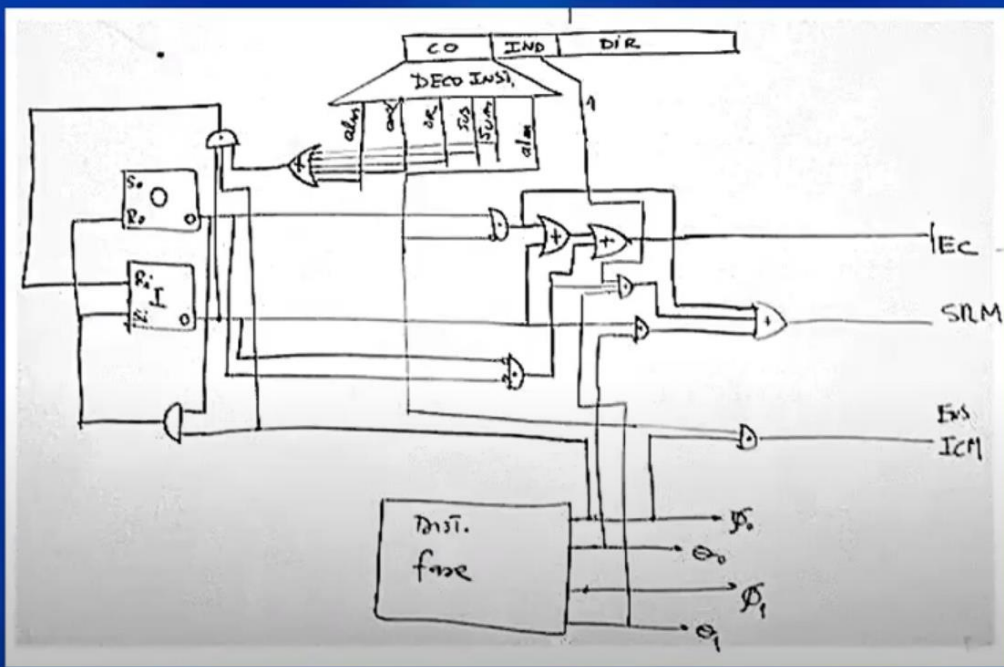
El circuito asociado a la ecuación lógica de la microórden SRM dentro del secuenciador se puede apreciar en el siguiente dibujo:



$$LEC = I + O \cdot \overline{alm} + (\bar{O} \cdot \bar{I})$$

$$SRM = \theta_0 \cdot I + O \cdot \overline{alm} + IND \cdot \theta_1 \cdot (\bar{I} \cdot \bar{O})$$

$$ENS = \phi_0 \quad ICM = \phi_0$$



TMS (Contenido del Bus M al RS)

Se da solamente en el cronograma de direccionamiento directo, en su segundo ciclo, donde se busca al operando a partir de la direccion almacenada en el RPM. Este ciclo esta indicado por los siguientes valores de los biestables de estado $I = O = 0$

$$TMS = IND \cdot \phi_1 \cdot \bar{I} \cdot \bar{O}$$

SRD (Salida campo DIR del RI)

Cuando se quiere llevar al RS la dirección contenida por el campo D.

- ★ En el caso de sum/sus/and/or contiene la dirección efectiva del operando
- ★ En el almacenamiento la dirección donde almacenar el contenido de la ALU
- ★ En sai la dirección a donde saltar
- ★ En sap con condición satisfecha, es decir que el acumulador sea mayor a cero, también con tiene la dirección de la siguiente instrucción del programa a ejecutar

Es durante el ciclo de instrucción, pero una vez que la instrucción ya se encuentre en el RI, por lo tanto será en la segunda fase, ϕ_1 ,

$$SRD = I \cdot \Phi_1 \cdot (sum + sus + and + or + alm + sai + sap \cdot AP)$$

SRP (Salida del CP al BUS S)

Es una señal de nivel que se da al final de los cronogramas como preparación de la próxima instrucción, en Φ_1 . Al final del ciclo de instrucción en el caso de **pac** y **sap con condición insatisfecha**, ya que solo usan un ciclo de memoria, y sino al final del ciclo de operando para las demás operaciones

$$SRP = I \cdot \Phi_1 \cdot (sap \cdot AN + pac) + O \cdot \Phi_1$$

ENA (Entrada a la ALU) – EAC (Entrada al acumulador)

La entrada a la ALU se dará en las operaciones sum/sus/and/or en el ciclo de operando, durante todo $\Phi_0 + \Phi_1$, y se debe mantener hasta que se termine de hacerse la operación

Como $\Phi_0 + \Phi_1$ es igual a 1 se lo omite

Y se reduce las operaciones a no alm

$$ENA = O \cdot \overline{alm}$$

En el ultimo batido del clock se debe habilitar la señal impulsional EAC, Para almacenar en el acumulador el resultado del calculo realizado

$$EAC = O \cdot \overline{alm} \cdot \theta_0$$

ENM (Entrada al RPM)

Se debe dar cuando se quiere almacenar, donde se necesita en el ciclo de operando que el contenido del bus m (que era el contenido de AC) ingrese al RPM para poder hacer la escritura. Esto es en el primer b

$$ENM = O \cdot alm \cdot \theta_1$$

SAC (salida del acumulador al Bus M)

ECUACIONES LOGICAS PARA BIESTABLES DE ESTADO I (instrucción) y O (operando)

Como el mismo secuenciador posiciona los biestables I y O, tendremos:

- ★ I debe ser posicionado a 1 al final del ciclo de operando:

$$SI = \theta_0 \cdot O$$

- ★ I debe ser puesto a cero al final del ciclo de instrucción de aquellas instrucciones que necesitan ciclo de operando (IBO + almacenamiento):

$$RI = \theta_0 \cdot I \cdot (sum + sus + and + or + alm)$$

Cuando esta en tita cero, el biestable I deberá ser reseteado ya que se encuentran en el inicio de la fase de resolución del operando. Esto no se da para **pac, sap o sai o IES (instrucciones de entrada- salida)**.

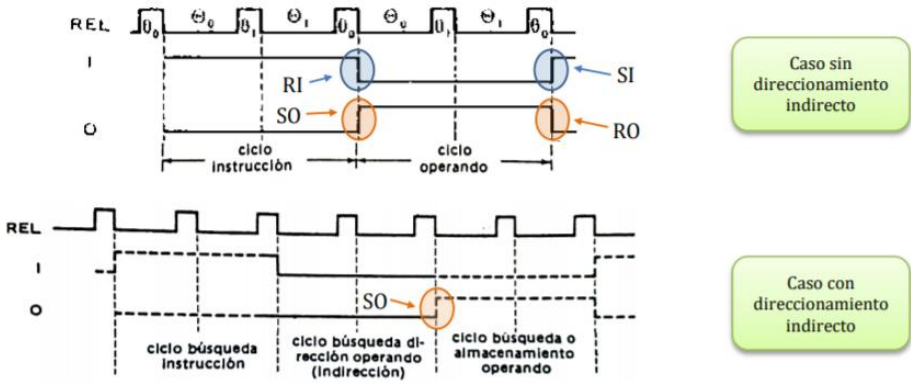
- ★ O debe ser puesto a 1 al final de un ciclo de instrucción cuando la instrucción requiere ciclo de operando sin direccionamiento indirecto, o luego de la búsqueda de la dirección efectiva del operando en caso de direccionamiento indirecto:

$$SO = \overline{IND} \cdot \theta_0 \cdot I \cdot (sum + sus + and + or + alm) + IND \cdot \theta_0 \cdot \overline{I} \cdot \overline{O}$$

No se da en el ciclo Indirecto en el mismo lugar que sum sus or and alm. Se da en indirecto en el ultimo ciclo del cronograma (al final de TMS para pasar al ciclo de operando)

★ O debe ser puesto a cero al final del ciclo de operando:

$$RO = \theta_0 . O$$



LISTADO DE ECUACIONES LOGICAS:

ENS	$= \theta_0$	
ICM	$= \theta_0$	
PACM	$= \theta_0$	
LEC	$= I + O \cdot \overline{alm} + (\bar{I} \cdot \bar{O})$	
ESC	$= O \cdot alm$	
SRM	$= I \cdot \Theta_0 + O \cdot \overline{alm} + IND \cdot \Theta_1 \cdot (\bar{I} \cdot \bar{O})$	
TMS	$= IND \cdot \bar{I} \cdot \bar{O} \cdot \Theta_1$	En caso de que $(AC) \geq 0$ se salta a la dirección en D
ENI	$= \theta_1 \cdot I$	
SRD	$= \Theta_1 \cdot I \cdot (sum + sus + and + or + alm + \boxed{sap \cdot AP} + sai)$	
SRP	$= \Theta_1 \cdot I \cdot (pac + \boxed{sap \cdot AN}) + \Theta_1 \cdot O$	
INCP	$= \theta_1 \cdot I$	
ENP	$= \theta_0 \cdot (sap \cdot AP + sai)$	En caso de $(AC) \leq 0$ se sigue con la dirección en P
SUM	$= sum \cdot O$	
SUS	$= sus \cdot O$	
AND	$= and \cdot O$	
OR	$= or \cdot O$	
ENA	$= O \cdot \overline{alm}$	
EAC	$= O \cdot \overline{alm} \cdot \theta_0$	
PAC	$= pac \cdot \theta_0$	
SAC	$= O \cdot \Theta_0 \cdot alm$	
ENM	$= O \cdot \theta_1 \cdot alm$	
SI	$= \theta_0 \cdot O$	
RI	$= \theta_0 \cdot I \cdot (sum + sus + or + and + alm)$	
SO	$= \overline{IND} \cdot \theta_0 \cdot I \cdot (sum + sus + or + and + alm) + IND \cdot \theta_0 \cdot \bar{I} \cdot \bar{O}$	
RO	$= \theta_0 \cdot O$	

Ecuación lógica LEC

Básicamente, el secuenciador de una computadora síncrona es un circuito secuencial que produce las secuencias de microórdenes necesarias para la ejecución de las instrucciones, sincronizándolas con los impulsos procedentes del reloj. Se lo puede fraccionar en 2 subconjuntos:

- ★ por un lado, unos **elementos de memoria** para poder marcar las diferentes fases del desarrollo de las instrucciones (distribuidor de fases y biestables de estado posicionados por el secuenciador)
- ★ por el otro el **circuito combinacional** correspondiente a las ecuaciones lógicas.

En la realidad, la concepción del secuenciador no es tan sencilla, sobre todo cuando el conjunto de instrucciones es grande y cuando gobierna operaciones secuenciales complejas. Ello nos lleva a pensar **en 2 niveles de control**: un **nivel central** que distribuye microórdenes globales a los distintos operadores y un **nivel local**, asociado a cada operador, para el gobierno de sus operaciones internas.

SECUENCIADOR MICROPROGRAMADO

En los complejos procesadores modernos, el número de ecuaciones lógicas necesarias para definir la unidad de control es muy grande, implementar un circuito combinacional que satisfaga todas estas ecuaciones es muy difícil. Además, el diseño es relativamente inflexible, ya que por ejemplo, es difícil alterar el cableado si se desea añadir una nueva instrucción máquina.

Una alternativa a la unidad de control cableada es la **unidad de control microprogramada**. Este tipo de unidad **contiene una memoria**, llamada memoria de control o ROM de control. **A cada instrucción máquina le corresponde, en dicha memoria, un microprograma, compuesto por microinstrucciones**, cuyo desarrollo genera las microórdenes que gobiernan la ejecución de dicha instrucción.

Básicamente la microprogramación consiste en reemplazar el secuenciador cableado por uno programado, lo que hace que añadir o cambiar un microprograma sea mucho más fácil que alterar el cableado.

El modelo de Wilkes

A la entrada del dispositivo de control tenemos el código de operación de la instrucción por ejecutar, el cual es considerado como la dirección de la 1ra microinstrucción del microprograma asociado a la instrucción.

Tal dirección se envía al registro SMC (Selección de Memoria de Control) que, decodificado, permite activar un hilo de palabra horizontal sobre el cual está grabada la microinstrucción bajo la forma siguiente: los 1 están representados por un acoplamiento (marcado con un punto) con el correspondiente hilo de bit vertical, los 0 por la ausencia de acoplamiento. A la salida, la microinstrucción comprende 2 elementos:

- ★ por un lado, el **código de microinstrucción**, que da directamente las microórdenes a distribuir por la ruta de datos;
- ★ por el otro, la **dirección de la próxima microinstrucción** (DC o Dirección de Control) que será transferida, al siguiente batido de reloj, a SMC, etc.

El microprograma concluirá con una última microinstrucción, la cual cargará en SMC el nuevo código de operación. Por último, para permitir saltos condicionales, una fila debe contener dos partes de direcciones, controladas por un biestable que es posicionado por las microinstrucciones, y que, dependiendo de su valor, activará una u otra bifurcación.

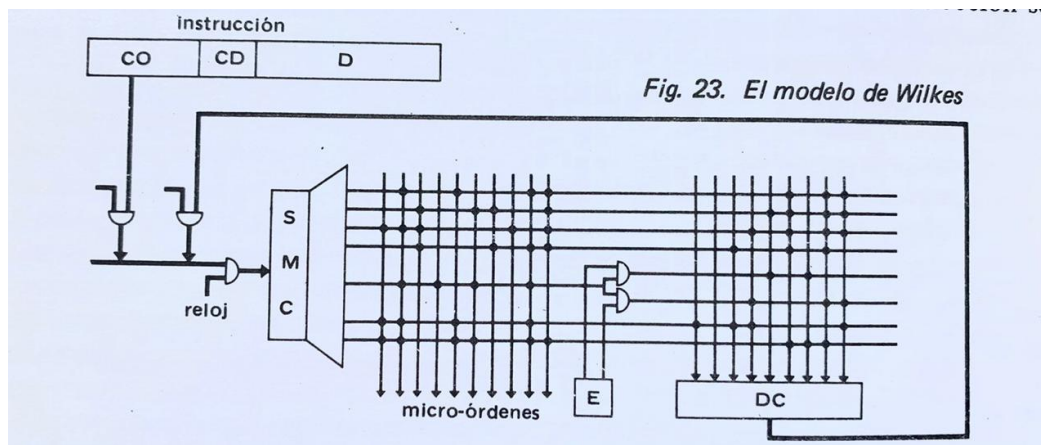


Imagen del meinadier (fig. 23)

Ventajas	Desventajas
Es mas económica , porque no se gasta en compuertas	Mas lenta , con respecto a la cableada (Para una instrucción que requiere cuatro fases, la memoria de control deberá ser 4 veces más veloz que la RAM)
Mínimo espacio , porque el dispositivo en si se reduce a una memoria de control	Tamaño de la palabra de control . Si no esta codificada puede ser muy larga
Ahorro de consumo , porque no hay circuitos, la memoria no necesita alimentación continua	
Es mas flexible que la cableada, se puede cambiar o agregar una instrucción facilmente	
Mas sencillo para programar	

Formato de las microinstrucciones

Se ha visto en el modelo de Wilkes que la microinstrucción se dividía en dos partes: una que permitía generar las microórdenes, otra que definía la dirección de la próxima microinstrucción. En esta parte nos enfocamos únicamente en la primera parte de la microinstrucción.

Pueden concebirse dos métodos extremos para pasar de la configuración binaria de la microinstrucción a las diversas señales de control o microórdenes correspondientes:

- ★ El primero consiste en hacer corresponder cada bit de la microinstrucción con una microórden, lo que se conoce como microinstrucción horizontal. Esta fue la propuesta original de Wilkes.
 - **Ventajas:** No se necesita ninguna lógica de decodificación, cada bit genera directamente una señal de control individual. Permite generar varias microórdenes simultáneas.
 - **Desventajas:** Requiere gran cantidad de bits y por lo tanto más memoria. Es más difícil de programar, ya que el microprogramador maneja individualmente cada microórden.
- ★ Inversamente, el segundo método consiste codificar el conjunto de posibles microórdenes con el número mínimo de bits. La microinstrucción se puede organizar como un conjunto de campos, que decodificados activan una o más microórdenes. La codificación se hace de modo que cada campo codifique aquellas microórdenes relacionadas al mismo tipo de función o recurso, de manera que puedan ocurrir simultáneamente acciones de

diferentes campos, pero no las de un mismo campo a la vez. Por ejemplo “apertura de puertas de diversos registros a un mismo bus”; cada código del campo representa un registro diferente, pero solo uno puede enviarse al mismo tiempo al bus, o “señales de gobierno de la ALU”. Este tipo de formato se denomina microinstrucción vertical.

- ➔ **Ventajas:** Ocupan menos bits y por lo tanto menos memoria de control. Son más fáciles de programar.
- ➔ **Desventajas:** Es necesaria la decodificación de los campos y cierto retardo temporal debido a esto.

Codificación tipo instrucción

Como su nombre indica, este tipo de codificación da a la micro-instrucción una estructura semejante a la de una instrucción, con código de operación y dirección de operando (en registro o en memoria local).

La decodificación de las micro-instrucciones para obtener las correspondientes microórdenes es relativamente compleja, siendo compensado por la pequeña longitud de las micro-instrucción. Tal tipo de codificación se adopta a menudo cuando se desea poner la microprogramación al alcance del programador en lenguaje de máquina.

Codificación por campos

Este tipo se acerca mucho más al funcionamiento real del calculador. Se dividen las distintas microórdenes en grupos independientes de tal suerte que sea imposible que, en cada grupo, se den microórdenes simultáneas. Para la comodidad del programador cada grupo corresponde, además, a un determinado tipo de función: apertura de puertas de diversos registros a un mismo bus, gobierno de una unidad funcional, gobierno de una memoria local, etc. Se asocia un campo de la micro-instrucción a cada grupo así definido, con una longitud justa para codificar todas las microórdenes del grupo. La decodificación de la microinstrucción se ejecuta campo por campo. Respecto a la codificación tipo instrucción, la codificación por campos aporta la ventaja de una decodificación más directa y el inconveniente de una microinstrucción más larga. Facilita a los programadores expertos en la estructura de la máquina una mejor optimización de los microprogramas.

Secuenciamiento de las microinstrucciones

Nos concentramos ahora sobre la segunda parte de la microinstrucción del modelo de Wilkes, aquella que define las informaciones para encadenar las microinstrucciones. Hay que tener en cuenta que cuando se ejecuta un microprograma, la dirección de la siguiente microinstrucción a ejecutar puede venir dada de 3 formas distintas:

- ★ Viene determinada por el registro de instrucción.
- ★ Es la siguiente dirección secuencial.
- ★ Es el destino de un salto.

Son posibles dos grandes opciones:

- ★ **Direccionamiento secuencial:** Una microinstrucción normal direcciona implícitamente la microinstrucción que se encuentra en la dirección próxima a la suya. El registro de selección de la memoria de control funciona entonces como un contador ordinal. Los saltos pueden realizarse especializando un bit de la microinstrucción como indicador afirmativo o negativo de salto y dejando el resto de la misma para albergar la dirección de la próxima microinstrucción por ejecutar y, cuando sea el caso, la condición de salto.
 - ➔ Ventajas: Simple de programar, no alarga la microinstrucción.
 - ➔ Desventajas: Un salto ocupa por sí solo una microinstrucción y desperdicia tiempo de ejecución, ya que no se generan microórdenes. Por lo tanto solo es conveniente este método cuando haya un porcentaje pequeño de microinstrucciones de salto.

★ **Direccionamiento explícito:** Se fracciona la parte de microinstrucción que define a la microinstrucción siguiente en 2 campos. El 1ro contiene de manera explícita los bits de mayor peso de la dirección. El 2do contiene, bien directamente los bits de menor peso de la dirección, si esta fuera conocida, o bien la dirección de un registro que los tenga almacenados, si fuera condicional.

Además, este último campo puede ir acompañado de un indicador cuyo valor defina si esos bits significan los de menor peso de la dirección o la dirección de un registro que los contiene.

- Ventajas: Minimiza el número total de microinstrucciones, porque algunas son comunes a varios microprogramas. No supone pérdida de tiempo en los saltos incondicionales.
- Desventajas: Es difícil de usar y aumenta la longitud de la microinstrucción.

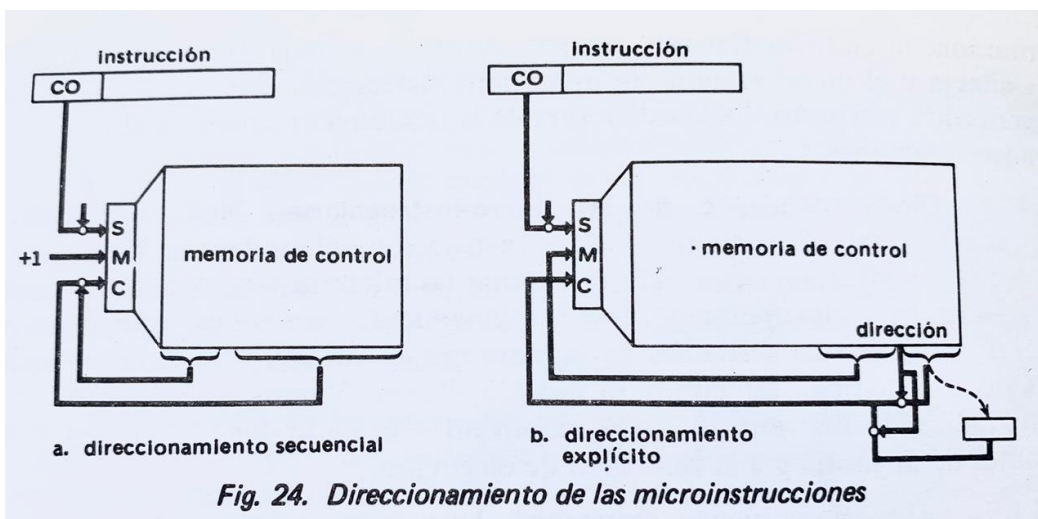


Figura 24 Meinadier

Unidad de control cableada vs. Microprogramada

Acompasamiento

El problema se plantea a dos niveles: el primero correspondiente al acompasamiento durante el desarrollo de una microinstrucción, el segundo al compasamiento de las demandas de sucesivas micro-instrucciones.

- ★ **Acompasamiento del desarrollo de una microinstrucción.** Cuando se emite simultáneamente el conjunto de las microórdenes generadas por cada microinstrucción, se habla de *microprogramación vertical*. En tal caso, el único compás existente es el de la demanda de las microinstrucciones. El método de codificación tipo instrucción y la técnica de microprogramación vertical se utilizan a menudo juntos.

Inversamente, es posible concebir que las microórdenes emanadas de los distintos campos de una microinstrucción sean emitidas en diferentes fases de su ejecución. Esta es la *microprogramación horizontal*. En dicha hipótesis la unidad de control puede estar provista de un distribuidor de fases como los secuenciadores cableados.

(Fig. 25 y 26)

- ★ **Acompasamiento de las demandas de las microinstrucciones.** El compás de demanda de las microinstrucciones puede ser fijo o variable. Se observa, en general, el uso de un compás variable en microprogramación vertical, bien porque el calculador gobierne por si mismo la llamada a la nueva microinstrucción en cuanto que las operaciones relativas a la anterior hayan terminado, bien porque el campo de la microinstrucción defina su duración, como se representa en la figura 26. A menudo se emplea el compás fijo de demanda en las máquinas con **microprogramación horizontal**, correspondiendo cada ciclo de memoria de control a un número fijo de batidos de reloj (Fig. 25)

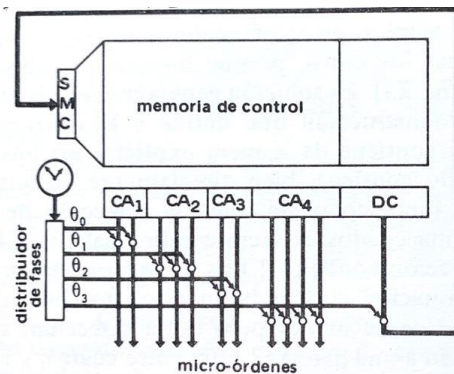


Fig. 25. Microprogramación horizontal y ritmo fijo

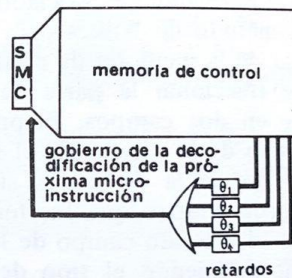


Fig. 26. Ritmo variable definido por la microinstrucción

CANALES

(Meinadier (leer completo esta parte) + notas d clase)

Toda arquitectura ejecuta un programa en curso, donde además de ocupar ciclos de memoria para este programa, en ciertos momentos requiere que se atiendan operaciones de entrada/salida. El programa necesita comunicarse con el exterior y atender a los dispositivos de E/S, lo hace a través de su controladora. La arquitectura no puede conocer a todos los periféricos que hay, conoce las controladoras de familias de periféricos de la misma clase, esta se comunica con sus controladoras que a su vez se comunican con los periféricos.

Los intercambios de información con el exterior requieren de 2 tipos de unidades entre la computadora o su memoria, por un lado, y los periféricos por otra.

- ★ **Las unidades de intercambio o canales** que permiten transferir información entre la computadora y cualquier unidad externa. Los canales más evolucionados se llaman procesadores de entrada-salida por diferenciarlos de la unidad central, llamada a veces procesador central.
- ★ Las unidades de control, o unidades de enlace o **controladores de periféricos**, que gobiernan uno o varios periféricos de una determinada clase obedeciendo a las órdenes de la computadora.
- ★ A estos dos tipos de unidades se suma el **sistema de interrupciones** que permite al medio exterior avisar a la computadora sobre acontecimientos exteriores, como por ejemplo la terminación de una operación de entrada-salida.

Hay distintos dispositivos de E/S, algunos requieren un dialogo permanente con la arquitectura, e interrumpen a la misma, estos serian el mouse, teclado o una pantalla táctil. Luego hay otros dispositivos que solo se solicitan cuando el programa indica que requiere comunicarse con el exterior, estos dispositivos en si mismo no interrumpen a la arquitectura, son por ejemplo dispositivos de almacenamiento de donde se quiere leer o escribir información o por ejemplo una impresora.

Esto termino resultando en una **competencia entre el programa en curso y los dispositivos E/S por el uso del procesador. Ya que ambas deben compartir un órgano de la arquitectura que es la memoria central**, es la única que tiene esta capacidad ya que la ALU no se comunica con el exterior y la UC solo se encarga de dar ordenes a la maquina.

El problema también resulta en que los dispositivos externos no trabajan con la misma frecuencia que la memoria, hay una gran diferencia en las velocidades entre la ejecución de instrucciones de un programa y los ritmos de transferencia, generalmente mucho mas lentos, impuestos por los periféricos. En el tiempo que lleva imprimir una línea ya se podrían haber ejecutado millones de instrucciones del programa en curso. Por eso se desarrollaron distintas técnicas para reducir los inconvenientes que causa esta asimetría, surgiendo así los **canales**, que son unidades dedicadas al tratamiento de operaciones de entrada/salida que automatizan estas tareas y liberan a la CPU para que pueda continuar con la ejecución del programa en curso.

CANAL → Dispositivo lógico que actúa de intermediario vinculan la memoria central con el conjunto de controladores de periféricos con el fin de automatizar de la mejor manera posible los procesos de transferencia de info desde la memoria a los periféricos y viceversa, administra los accesos de memoria desde o hacia los perifericos. Soluciona la asimetría entre los tiempos de ejecución de los periféricos y las instrucciones en curso.

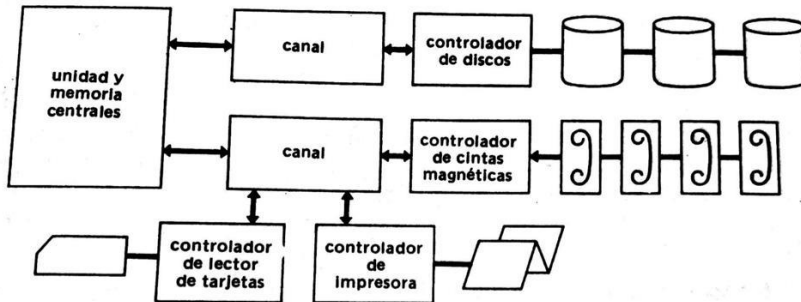


Fig. 1. Configuración ilustrativa de la función de los canales y de los controladores

SIMULTANEIDAD EN PROCESOS DE ENTRADA/SALIDA

Se trata de diseñar estrategias con el fin de lograr la mayor cantidad de simultaneidad en el uso de la CPU por parte del programa en curso y las transferencias con el exterior, reduciendo así la asimetría mencionada previamente.

Están ordenadas según como surgieron y mostrando así como la siguiente fue mejorando aspectos de la anterior.

Modo Bloqueado

Procesador se detiene ante una instrucción de transferencia, bloqueando el programa en curso, a la espera de que el dispositivo este listo, cuando se logra el control del periférico la transferencia se hace al ritmo de este

Se dice que una entrada-salida se realiza en modo bloqueado cuando el calculador ejecuta el programa de entrada-salida esperando, a cada nueva instrucción de transferencia, que el elemento periférico esté disponible.

La desventaja de este modo, es que el calculador se encontrará bloqueado durante el desarrollo completo de la entrada-salida, con la consecuencia de que el porcentaje del tiempo activo de la unidad central puede llegar a ser muy pequeño (del orden del 0,1 por 100 con las unidades periféricas más lentas).

Modo Prueba de Estado

Con objeto de introducir una cierta simultaneidad, parece lógico no ordenar la operación de transferencia de una palabra hasta que la unidad periférica esté presta a ejecutarla completamente. Para ello, se pensó incluir en el programa, con una periodicidad adecuada, puntos de prueba para conocer si la unidad periférica estaba disponible para realizar la operación. Esta técnica, muy enojosa para el programador, no puede hacer frente mas que a necesidades muy particulares.

Es similar a un salto condicional por estado de periférico. Se transfiere y se regresa al programa en curso no se lo bloquea, para así lograr un cierto paralelismo

Modo por Interrupción de Programa

Ya no es tarea del programador realizar pruebas de estado de los periféricos, sino el periférico el que interrumpe y avisa que se encuentra listo para usar.

La interrupción consiste en detener momentaneamente el programa en curso para permitir que el programa de transferencia sea ejecutado con prioridad. El modo por interrupción permite una cierta simultaneidad entre cálculos y entradas-salidas, retrasando el programa de cálculo una cantidad igual a la suma de los tiempos de ejecución del programa de interrupción.

Modo Automático por suspensión del programa o modo canal

En el modo anterior, la transferencia de una palabra implicaba el desarrollo de un programa de interrupción que dura de 10 a 20 ciclos de memoria, de los que sólo uno es empleado realmente para la transferencia. Este razonamiento ha conducido a crear *unidades automáticas de intercambio o canales*, capaces de tomar completamente a su cargo la transferencia de todo un bloque de informaciones.

De igual manera este modo es una especificación de la arquitectura para mejorar la transferencia, pero no llega a ser un canal propiamente dicho, pero si ya es una unidad dedicada a la transferencia. Trata de mover no solo una palabra sino un bloque de palabras, siempre que sean contiguas.

Cuando el canal necesita intercambiar una información con la memoria, no produce ya una interrupción de programa, sino simplemente una solicitud de servicio para que le sea concedido un ciclo de memoria, con objeto de realizar dicho intercambio. Puede suceder que el programa en curso no pida acceso al mismo bloque de memoria en el mismo instante, en cuyo caso la solicitud de servicio no afectará en manera alguna a su desarrollo. En caso contrario, se *suspenderá* el desarrollo del programa durante el ciclo de memoria solicitado por el canal. Por consiguiente, la simultaneidad entre programa y entradas-salidas es casi total.

Arq incorpora una instrucción que contiene los siguientes campos:

- ★ *Nº de palabras a transferir*
- ★ *Dirección de la primer palabra*
- ★ *Cual es el periférico*
- ★ *Condición de finalización*

La realización de una operación de entrada-salida se desarrolla de forma siguiente:

1. Es el programa quien inicializa la operación: las instrucciones proveen al canal de todas las informaciones necesarias a la operación: número y emplazamiento de los datos por transferir, dirección de la unidad de entrada-salida, condiciones bajo las cuales debe terminarse la transferencia, etc.; una instrucción especial lanza la transferencia y el programa puede continuar independientemente.
2. La transferencia se efectúa al ritmo del equipo periférico bajo control del canal, que roba ciclos al calculador
3. El canal avisa que la transferencia ha acabado generalmente mediante interrupción. El programa de interrupción podrá preguntar entonces al canal en qué condiciones se ha efectuado aquella.

En general, el programa que ha lanzado la operación de entrada-salida puede ejecutar a cada instante una instrucción de prueba entrada-salida, que le permite conocer en qué punto se encuentra la operación.

Se distinguen dos tipos de canales automáticos: los **canales simples o selectores** que no pueden gestionar más de una transferencia a la vez, y los **canales multiplados en el tiempo**, capaces de gestionar varias transferencias simultáneamente.

- ★ **La transferencia la inicia el programa**
- ★ **La transferencia se hace al ritmo del periférico**
- ★ **El canal roba ciclos al procesador**
- ★ **El canal avisa la finalización y sus condiciones**

Encadenamiento automático de las transferencias (programa canal):

Se basa en el mismo concepto de modo automatico donde esta técnica es la que se encarga de la transferencia y de saber donde empieza y termina un bloque de palabras, pero, en el modo canal la computadora está obligada a inicializar cada transferencia cargando las informaciones necesarias en los registros del canal.

Para aliviar a la computadora de esta tarea, se hace que el canal tenga su propio programa. La unidad central genera en la memoria central un programa de control de E/S o programa de canal, que contiene las diferentes transferencias sucesivas a realizar. Para lanzar dicho programa, la unidad central proporciona la dirección del mismo al canal, que se encargará de su ejecución completa.

Además, el programa de canal permite encadenar varias transferencias sin intervención de la unidad central, lo que se llama encadenamiento. Si considera que normalmente la información a transferir no esta almacenada de manera contigua en memoria, por lo tanto tiene la capacidad de trabajar con una técnica similar al uso de punteros, de manera que pueda automáticamente encadenar distintos bloques de palabras no contiguas y presentarlas a la controladora como si lo fuera.

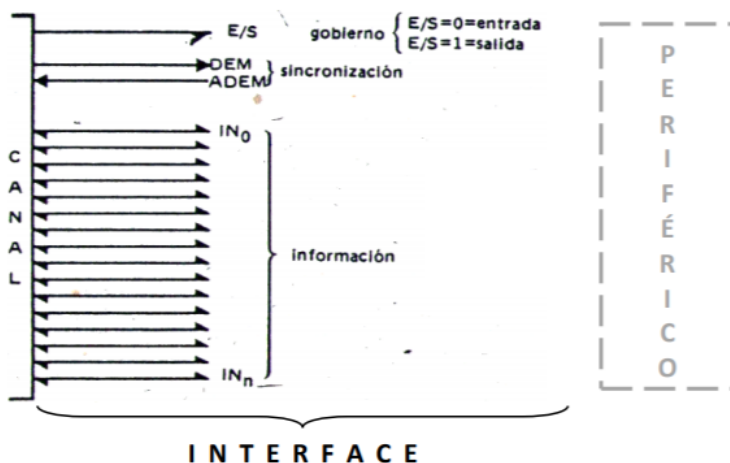
Interfaz

Debe ser posible conectar a un canal cualquier tipo de periférico, para esto sirve la interface, que es la “caja negra” que permite adaptar las señales del periférico a la computadora, y está formada físicamente por un cierto número de conectores, salidos del canal, en los cuales puede enchufarse el periférico.

Interfaz → **Circuito lógico que permite iniciar, sincronizar y transferir info del canal hacia una controladora**

Básicamente, la interface se compone de 2 tipos de hilos:

- ★ **Hilos de información**, que llevan en paralelo la información por transmitir (en forma de niveles)
- ★ **Hilos de gobierno y sincronización**, que permiten el diálogo entre canal y periférico.
 - **Señal de nivel E/S** → indica a la controladora si se quiere enviar o recibir info. Tiene que mantenerse activa todo lo que dure la transferencia
 - **Señal impulsional de sincronización DEM** → Solicitud de demanda de servicio del periférico (proviene de la arquitectura)
 - **Señal impulsional de sincronización ADEM** → Aceptación de la demanda de servicio (proviene del periférico)



Esquema de una interfaz paralela muy usada en los 80, trabajaba de bit a bit

Descripción de una salida de información (hacia el periférico)

El canal:

- ★ Posicionará la señal de nivel E/S para indicar que se trata de una salida de información;
- ★ Posicionará la información a salir en las líneas de información IN_0 a IN_n ;
- ★ Por último, enviará un impulso DEM de demanda de servicio, y quedará a la espera;

El periférico:

- ★ Reconocerá la demanda DEM y se fijará en el hilo de gobierno E/S para saber si la operación consiste en una entrada o en una salida de información;
- ★ Como E/S vale 1, sabrá que se trata de una salida y por lo tanto muestrearán mediante un impulso los hilos de información IN_0 a IN_n ;
- ★ Dicho impulso podrá ser usado también para decirle al canal que la demanda de salida fue reconocida y la información muestreada (enviándole el impulso bajo la forma ADEM de “acuerdo de demanda”).

Descripción de una entrada de información (desde el periférico)

El canal:

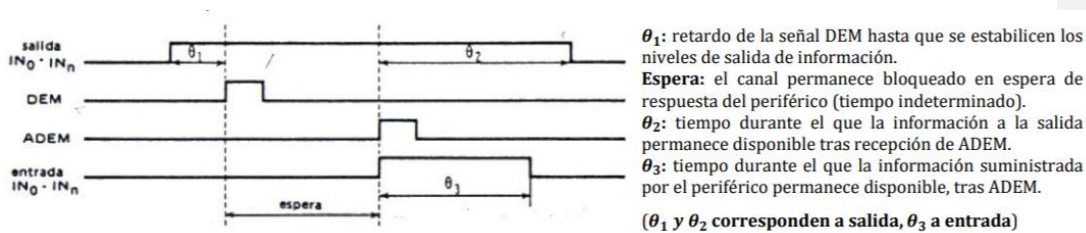
- ★ Pondrá el hilo E/S a 0 para indicar que se trata de una entrada;
- ★ Enviará el impulso DEM al periférico

El periférico:

- ★ Recibirá la solicitud DEM y sabrá que es una entrada (es decir que el debe enviar información) por el valor del hilo de gobierno E/S en cero;
- ★ Pondrá la información a enviar en los hilos IN_0 a IN_n ;
- ★ Enviará la señal de acuerdo ADEM.

El canal:

- ★ Recibe ADEM y lee de los hilos de información la información solicitada.



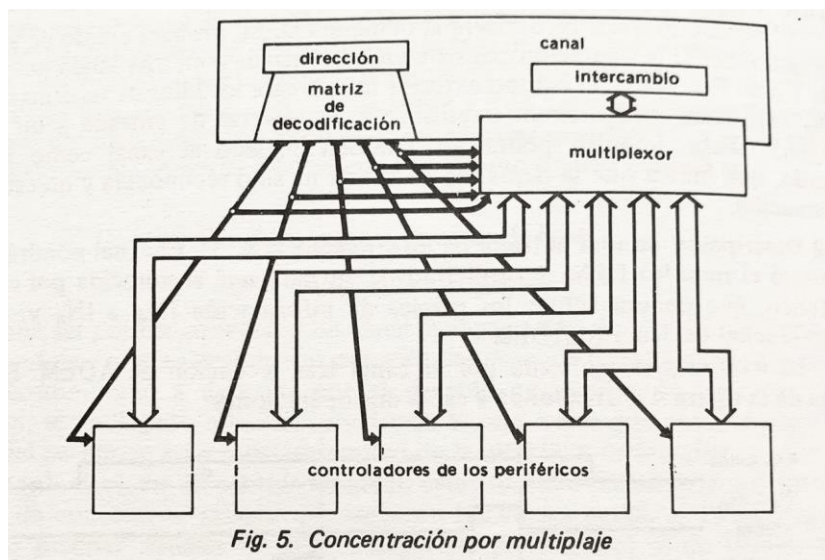
¿Qué técnicas hay para distribuir la información a través de la interfaz desde o hacia periféricos

Multiplaje

Multiplaje → Técnica de comunicación entre un canal y sus controladores

Permite concentrar varias vías de entrada en una sola o distribuir una vía de salida en varias vías. También permiten conectar varias unidades externas (periféricos) a un mismo canal. A veces se usa este método en los “canales simples”, es decir aquellos que no pueden gestionar más de una transferencia a la vez. De esta forma, el direccionamiento de las unidades externas (es decir la selección de la unidad con la que se trabaja en el momento de entre todas las conectadas) se hace en “estrella”, un hilo por unidad, después de decodificar la dirección de la unidad en el canal.

Dada la dirección del periférico, la *matriz de decodificación*, indica cual es el periférico que se quiere usar, sensibilizando una de las líneas verticales que selecciona a una, y solo una, controladora del grupo de controladora. Sensibiliza a la línea horizontal que le corresponde, la cual llega al *multiplexor* indicándole con cual controladora debe hacer la transferencia. Comunicandose así el multiplexor con la memoria a través de su registro de *intercambio*. El multiplexor



La dirección del periférico es decodificada por el canal el cual realiza 2 operaciones:

1. Selecciona con una señal de nivel una de las controladoras afectadas a la tarea
2. Establece un canal único de comunicación entre la MC y el controlador seleccionando la dirección del periférico asociada a la controladora

→ **Ventajas:**

El canal tiene la capacidad de poder rápidamente alternar la comunicación desde o hacia otra controladora, permite trabajar con múltiples controladoras.

Permite vincular el único recurso, al tener un único canal y única memoria, con varias controladoras para cambiar rápidamente. El canal se ubica de manera simultánea con varias controladoras. Flexibilidad en la conectabilidad, conectando varias entradas en una sola

→ **Desventajas:**

La complejidad física, se necesita de mucho cableado, se requiere de muchos buses, lo cual es costoso.

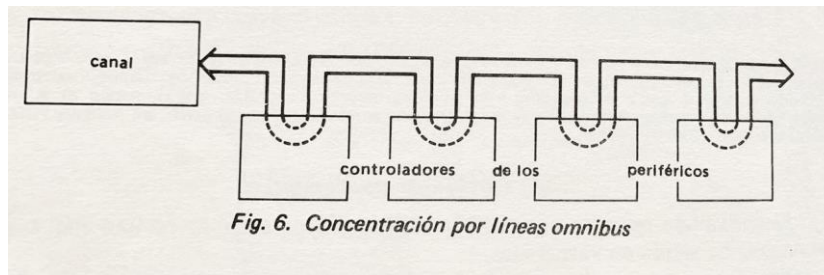
LINEAS OMNIBUS

Otra forma de conectar las unidades externas a un canal es usar una línea ómnibus que las una.

Si una unidad externa no resulta seleccionada, se comporta “transparentemente” frente a la información, es decir que la transmite sin alterarla.

El direccionamiento puede hacerse en “estrella” como se explica mas arriba. También es posible usar la propia línea ómnibus para el direccionamiento, en cuyo caso, un hilo especial indica que la información emitida por el canal es una dirección (no una información). Cada unidad externa atravesada por la línea ómnibus comprueba este hilo a cada transferencia, y si detecta que lo transmitido se trata de una dirección, compara la dirección transmitida con la suya propia. Si ambas coinciden, la unidad externa se pone en estado de diálogo con el canal, en cambio, si son distintas, transmite la dirección a la unidad siguiente.

La información a transferir va acompañada de un numero o puntero que indica cual es el periférico requerido pasa por cada controladora en el orden que están hasta coincidir con alguna.



→ Ventajas

Es mas económico que el multiplexaje, no se precisa de tanto cableado, se necesita de un solo bus.

→ Desventajas

Es mas lento dependiendo de en donde se encuentre la controladora requerida en la cola. ~~No hay simultaneidad, un solo paquete de información puede viajar por vez.~~

TECNICAS DE TRANSFERENCIA

La arquitectura debe presentar distintas estrategias de transferencia con el fin de controlar de forma mas eficiente las operaciones de E/S y su uso de la memoria del computador

Llamaremos transferencia elemental a la transferencia de una información (carácter o palabra) desde una posición de memoria central a una unidad externa (o viceversa, según se trate de una entrada o una salida). La ejecución de una transferencia supone que se conozcan las informaciones siguientes:

- información pidiendo el lanzamiento de la transferencia: DEM;
- indicación del sentido de la transferencia (entrada o salida): SEN;
- dirección de almacenamiento en memoria de la información transferida: DIR;

Las diferentes técnicas de ejecución de una transferencia elemental dependen de la manera en la que sean suministradas al ordenador estas tres informaciones. Distinguiremos los siguientes modos:

Transferencia Programada

El programa tiene una instrucción de transferencia, la cual brinda el sentido y la dirección, y esta se ejecuta cuando el programa llega a esa instrucción. Es el ordenador quien inicializa la demanda de transferencia (DEM). Las informaciones complementarias son suministradas por la instrucción, que puede adoptar dos formas:

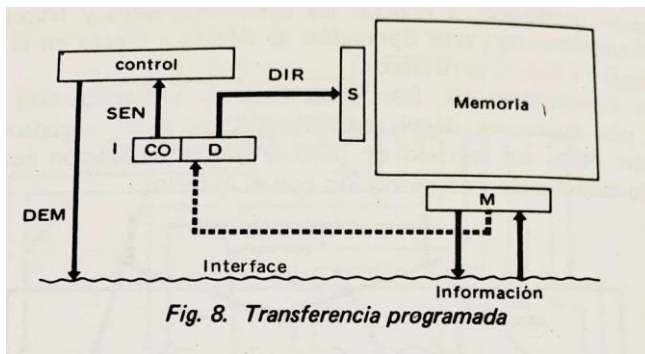
ENT DIR Entrar información del exterior y almacenarla en la dirección DIR.

SAL DIR Leer información almacenada en la dirección DIR y hacerla salir.

Usado en arquitecturas antiguas donde no había un modelo de canal, sino mas bien una interfaz

Consiste en una instrucción que se encuentra en el programa donde el código de operación le brinda a la UC el sentido de la transferencia

- ★ **DEM** → computadora – orden del programa en curso
- ★ **SEN** → instrucción – código de operación
- ★ **DIR** → instrucción – campo DIR

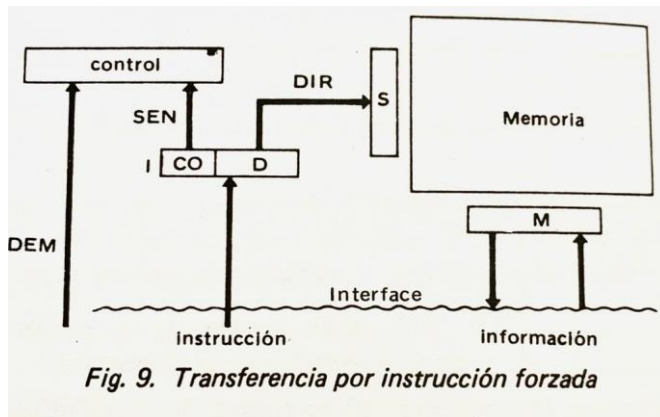


Transferencia por Instrucción Forzada

La transferencia por instrucción forzada es **inicializada por la unidad externa, que genera la instrucción de entrada-salida y fuerza su ejecución prioritaria en cuanto que la unidad central ha concluido la instrucción en curso**. Ni bien la ALU termina el calculo en curso, el procesador estará libre, y el programa queda suspendido durante el tiempo de ejecución de esta instrucción para darle un ciclo de memoria a esta instrucción. Ej: mousse. Se mete directo a la memoria no pasa por el RI y decodificar una instrucción.

Es forzada porque si hay una instrucción en curso donde la ALU esta haciendo el cálculo ahí ya aprovecha para meter esta instrucción E/S, no hace INCP el CP queda a espera de que se termine la transferencia

- ★ **DEM** → unidad externa
- ★ **SEN** → instrucción – código de operación
- ★ **DIR** → instrucción – campo DIR

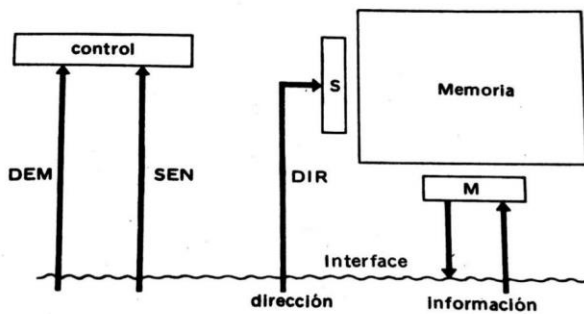


Transferencia por Robo de Ciclo

La transferencia por robo de ciclo se inicializa a petición de la unidad exterior, ya no hay una instrucción. La unidad central termina el ciclo de memoria en curso y concede el próximo ciclo a la unidad exterior, no tiene que esperar a que la instrucción en curso termine de ejecutarse solo que termine el ciclo actual de memoria. Esta indica si se trata de una lectura o de una escritura y proporciona la dirección de almacenamiento. **El programa en curso queda en suspenso durante un ciclo de memoria**, pero no se suspende por completo, en las anteriores no hay suspensión.

(DIR) → S | No-pasa por RI

- ★ DEM → unidad externa
- ★ SEN → unidad externa
- ★ DIR → unidad externa



Transferencia por Acceso Directo a Memoria

Se supone que la memoria posee dos vías de acceso. Una de las vías está reservada al ordenador, la otra a la unidad exterior.

Las peticiones de ciclo de memoria son dirigidas a la unidad de control de acceso a memoria, tanto si proceden de la unidad central como si de la unidad exterior. Esta unidad de control gestiona las prioridades de acceso de la unidad central y de la unidad externa, es el que establece si se puede lograr simultaneidad o no ante la petición de ciclo de memoria que puede provenir del exterior o de la UC.

Contemplado desde esta última, el proceso es el mismo que en el caso del robo de ciclo, con la única diferencia que la solicitud es dirigida al controlador de acceso a memoria y no a la unidad central. El programa en curso no resulta suspendido durante el ciclo de memoria solicitado, excepto cuando la unidad central hubiera pedido simultáneamente un ciclo de memoria.

Si en manera simultanea piden un ciclo la CPU y la unidad exterior, si se suspende el programa, y se da paso a la transferencia

Se puede leer o grabar una palabra del programa en curso y del exterior de manera simultanea

Generalmente, los sistemas de acceso directo a la memoria son interesantes si se dispone de varios bloques independientes de memoria. En la medida que el programa, de una parte, y las entradas-salidas, de otra, conciernan a bloques diferentes se acercaría el proceso a la simultaneidad total. Para aprovechar esta técnica el sistema operativo deberá acomodar en bloques

Si la unidad externa y la computadora eligen el mismo sector, se prioriza al programa en curso, solo se suspende si pide el ciclo de UC **al mismo tiempo**.

2 accesos distintos a memoria → M1 → ordenador | M2 → unidad externa

- ★ DEM → unidad externa
- ★ SEN → unidad externa
- ★ DIR → unidad externa

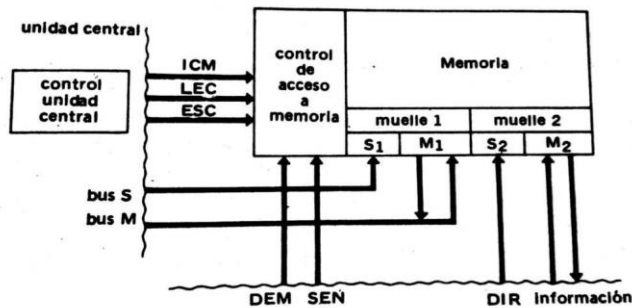


Fig. 11. Transferencia por acceso directo a la memoria

Técnica Empleada	Tiempo máximo de espera de la unidad exterior	Tiempo restado al programa en curso
Programada	Hasta el final de la instrucción en curso, mas inicio del programa solicitado de interrupción	Procesamiento de un programa de interrupción
Por instrucción forzada	Duración de la instrucción mas larga del repertorio	Una instrucción generalmente ejecutada en uno o dos ciclos de memoria
Por robo de ciclo	Un ciclo de memoria	Un ciclo de memoria
Por acceso directo	Un ciclo de memoria	Nada, o un ciclo de memoria

LOS CANALES:

Enlace Programado

Este no es propiamente un canal, sino un medio de acceso controlado por el programa, es una interfaz, organización de señales de gobierno para realizar transferencias programadas (precisan instrucciones en memoria alojadas en el programa). Permite transferir una sola palabra, por lo que este esquema se beneficia de las líneas ómnibus, no del multiplexaje

En efecto, cada transferencia elemental está gobernada por una instrucción del programa en curso.

Por el hecho de estar controlados por el programa, tales enlaces pueden utilizarse tanto en **modo bloqueado** como, mucho más a menudo, en **modo por interrupción de programa**. Casi siempre aparecen integrados en la unidad central.

Por ejemplo, para Abacus podríamos concebir un esquema parecido al representado en la figura.

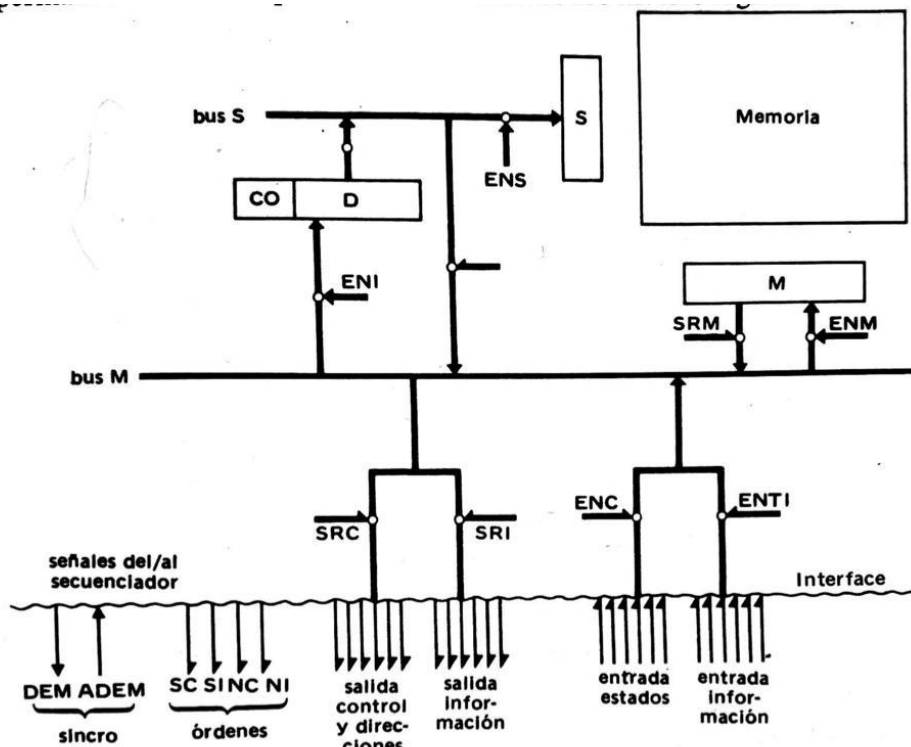
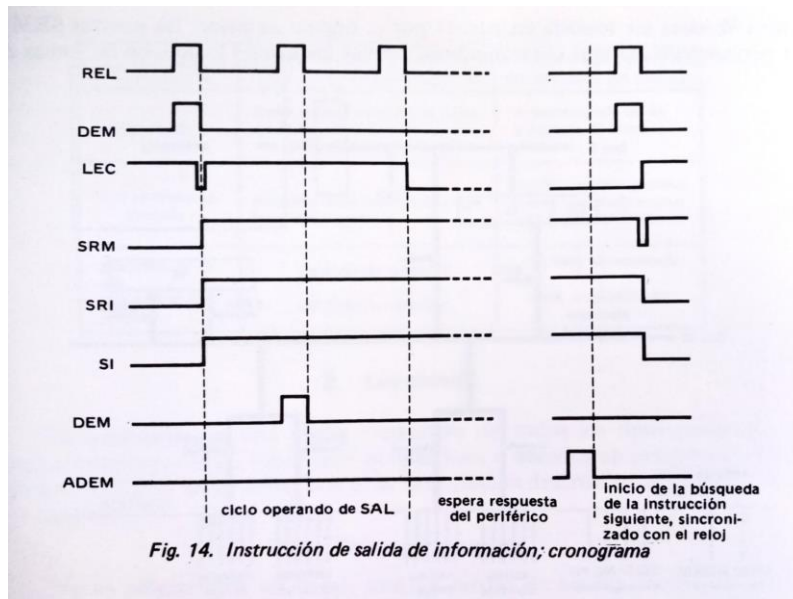


Fig. 12. Enlace programado de Abacus

SC: niveles lógicos posicionados sobre las líneas de salida de control; **SI:** niveles lógicos posicionados sobre las líneas de salida de información; **NC:** peticiones de entrada de estados; **NI:** peticiones de entrada de información.

Establece una técnica, interfaz lógica, para gobernar a un periférico ante la acción de una instrucción de E/S.

Trabaja sobre la premisa de 4 instrucciones de maquina dedicadas específicamente a comunicarse con el exterior, por lo tanto cada una de ellas contara con su cronograma (para la materia no es necesario saberlos pero si entender que al agregar instrucciones esto significa cablearlas en el secuenciador y por lo tanto que estas tengan su cronograma):



Son necesarias cuatro instrucciones de entrada-salida para realizar entradas-salidas en modo programado:

- ★ dos instrucciones de **gobierno y transferencias de información**, acompañadas de la dirección de palabra de memoria para leer o grabar:
 - **SAL M (SI)**, para extraer contenido de la dirección M hacia las líneas de salida de información de la interfase. Acompañado del campo DIR hace que la UC establezca las señales de gobierno necesarias para señalar al periférico
 - **ENT M (NI)**, para almacenar en la dirección M una información presente en las líneas de entrada de información de la interfase.
- ★ dos instrucciones de **control general de las operaciones de entrada-salida**:
 - **GCP** (gobierno del periférico SC), que saca la configuración binaria presente en la parte de dirección de la instrucción a las líneas de salida de control y dirección.
 - **PRE** (prueba de estado NC), que almacena en el acumulador el contenido de las líneas de entrada de estados. Acompañada de la dirección del periférico pregunta si esta libre o no

Las señales de interfase SI, NI y SC, NC corresponden respectivamente a estos cuatro tipos de operación.

GCP permite direccionar una unidad de control de periférico y especificarle la operación por ejecutar, PRE permite adquirir una información descriptiva del estado del periférico anteriormente direccionado. Primero se realiza la prueba de estado con PRE, luego de una prueba exitosa, es decir que el periférico esta disponible, se activa GCP para tomar gobierno del periférico.

En la operación de salida SAL, la información permanece memorizada en el registro M hasta ser tomada en cuenta por el órgano exterior: las puertas SRM y SRI permanecen abiertas para mantener activos los niveles lógicos en las líneas de salida de información y la señal SI permanece activa. El computador envía entonces al controlador del periférico previamente escogido, una señal impulsional DEM. Al recibirla este, supuesto disponible, comprueba los hilos de órdenes y, tras haber detectado que se trata de una salida, muestrea la información y transmite al computador una señal de acuerdo ADEM para liberarle.

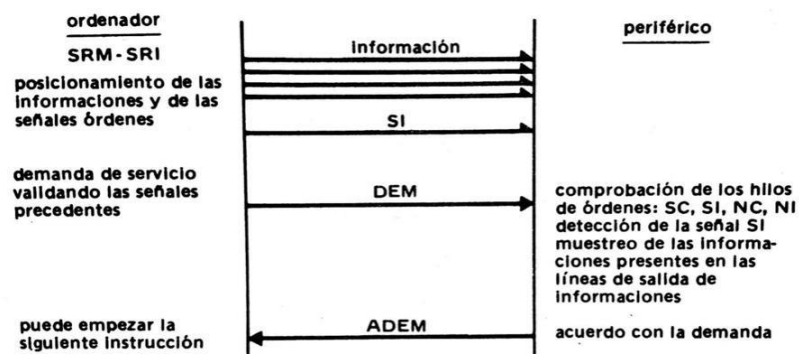
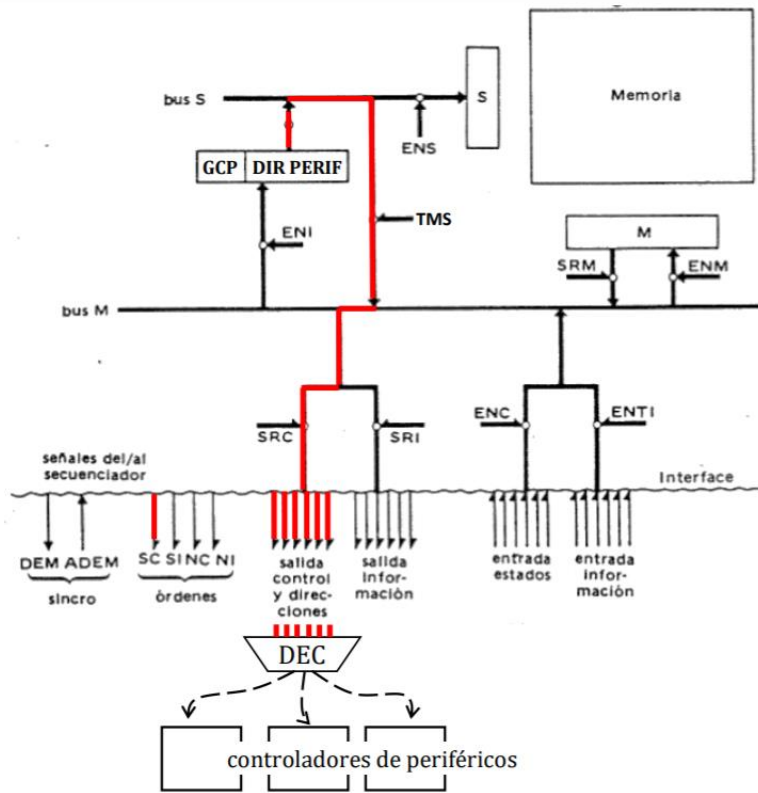
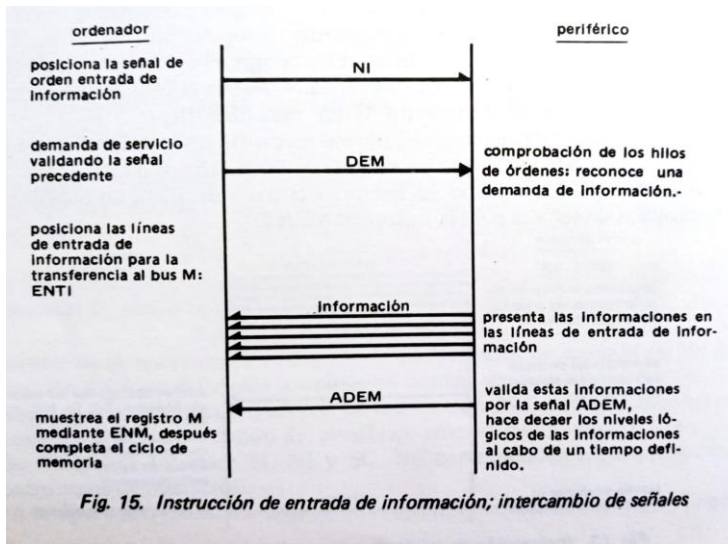


Fig. 13. Instrucción de salida de información; intercambio de señales



Además podemos ilustrar un posible sistema de intercambio de señales para la instrucción ENT.



Algunos calculadores poseen las instrucciones de *entrada directa* y *salida directa* que permiten la transferencia de una información entre el acumulador (o un registro aritmético designado por la instrucción) y el órgano periférico direccionado por ésta.

Modo Canal | Canal Automático

Ahora si ya presentamos un esquema de Canal, propiamente dicho, para Abacus.

Permite que el canal tome el gobierno de la transferencia de una o mas palabras contiguas de memoria

Los canales automáticos son capaces de gestionar el conjunto de una operación de entrada-salida sin intervención de la unidad central, excepto para inicializar y concluir dicha operación.

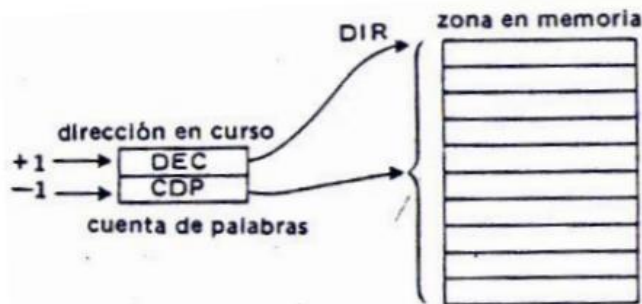
Las transferencias elementales pueden ser efectuadas, bien por **instrucción forzada**, bien por **robo de ciclo**, bien por **acceso directo a la memoria**. Se les llama **canales simples** cuando no gestionan más que una operación de entrada-salida a la vez.

Lo vamos a describir partiendo de que utiliza **robo de ciclo**. El programa no se suspende al no tener necesidad de crear una instrucción nueva, genera la dirección que necesita y cuando pueda roba un ciclo.

Desde este estricto punto de vista, las operación de entrada-salida queda enteramente definida por la unidad externa la cual debe indicar:

- ★ El sentido de la transferencia por realizar (entrada o salida)
- ★ La zona de almacenamiento en memoria de las informaciones por transferir.
- ★ Cantidad de palabras a transferir

Esta zona puede delimitarse mediante la dirección de su primera palabra (**DIR**) y su número de palabras (**CDP**)



La unidad central es quien inicializa la operación, por programa al llegar a una instrucción de E/S.

Selecciona el periférico (indicando eventualmente el sentido de la transferencia por efectuar) y carga las informaciones que definen a la zona de almacenamiento en memoria en dos registros del canal, a los que llamaremos **DEC (dirección en curso)** para la dirección de la zona y **CDP (cuenta de palabras)** para el número de palabras. Esta carga puede realizarse partiendo de instrucciones del tipo de salida directa en enlace programado.

Se lee la palabra y esta va al registro T o Tapón, donde puede permanecer el tiempo necesario hasta que el periférico este disponible, ya es incumbencia del canal su tratamiento permitiendo así que el programa en curso puede continuar con su ejecución.

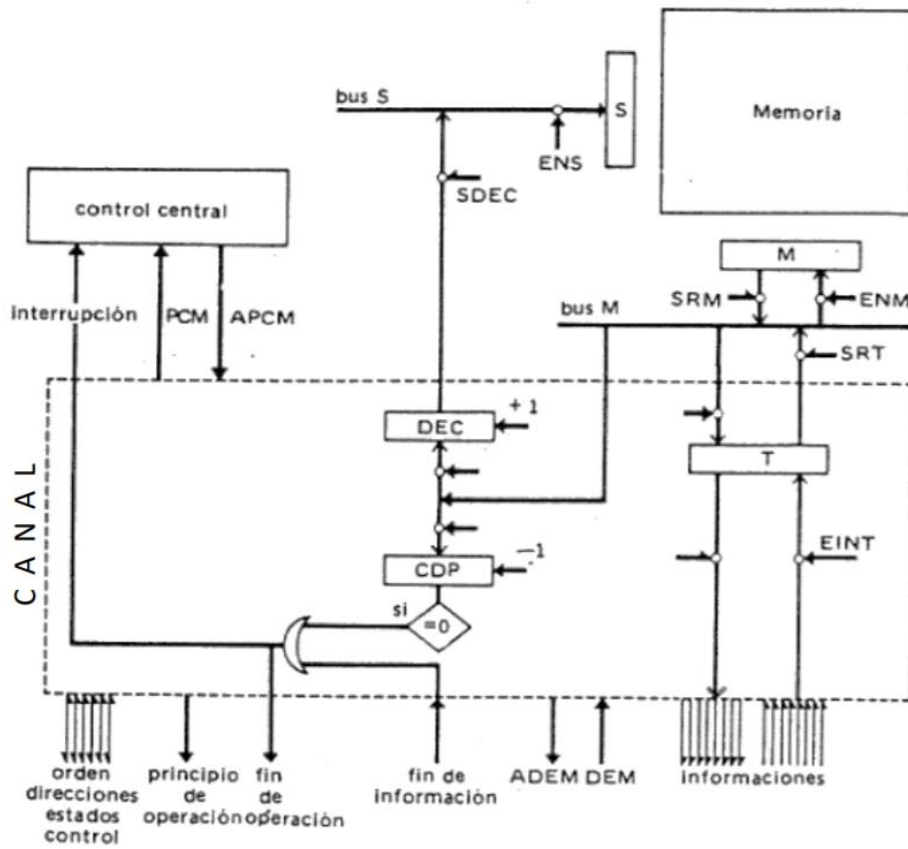
¿Por qué podría estar ocupado el periférico? No termino de gestionar la transferencia anterior, o porque es otro periférico(raro)

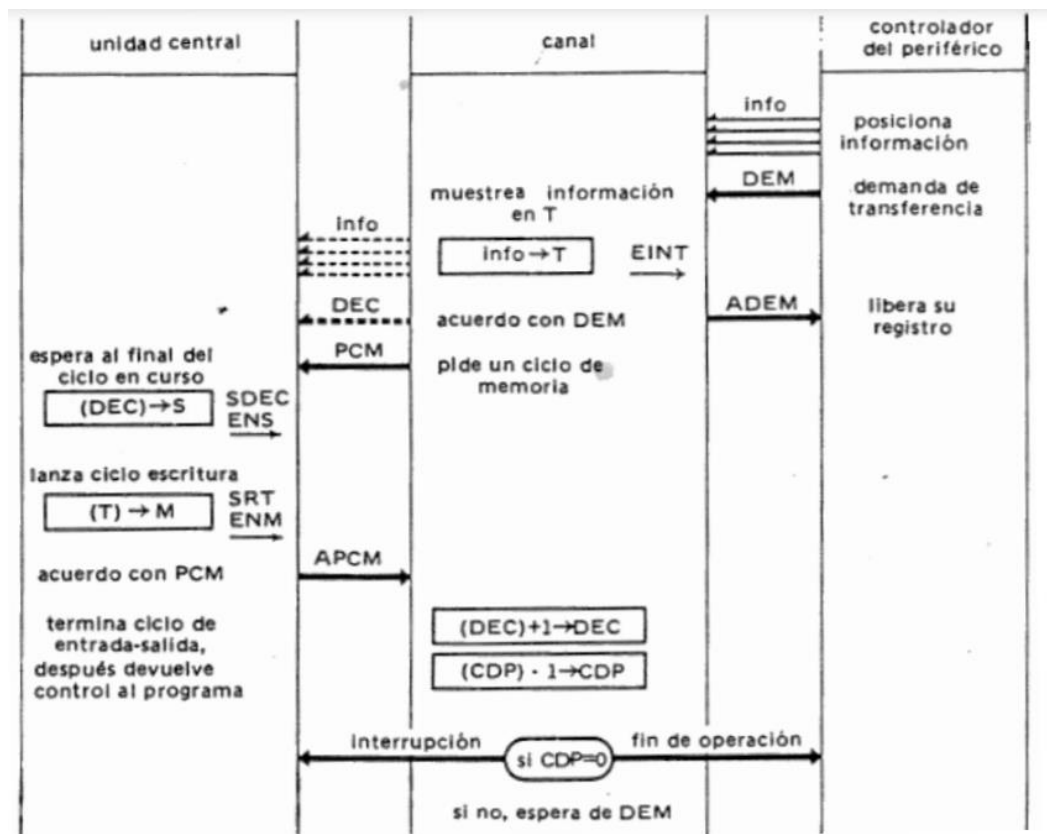
¿Por qué no se incluye una condición de fin? Porque termina cuando el CDP llega a 0 o el periférico por excepción da fin.

Una vez efectuada la inicialización, la operación de E/S se desarrolla ya al ritmo de la unidad periférica. Es a petición de ésta que el canal ejecuta una transferencia elemental, utilizando el contenido del registro DEC como dirección de almacenamiento en memoria, y después actualiza la dirección en curso, sumando 1 al registro DEC, y la cuenta de palabras, restando 1 al registro CDP. Cada vez que se incrementa DEC y se decrementa CDP hay una petición de ciclo de memoria, la unidad de control central acepta cuando pueda robar un ciclo.

La transferencia termina cuando el canal detecta que el registro contador de palabras llega a cero o cuando el controlador del periférico envía una señal de fin de información.

En ese momento genera el canal una interrupción para indicar a la unidad central que se ha concluido la operación E/S. Tal modo de funcionamiento, soportado por dos registros (CDP Y DEC), es designado generalmente como modo canal. A la correspondiente unidad de intercambio se le llama canal.





Encadenamiento de Datos

Permite pasar de una zona de memoria a otra sin necesidad de volver a inicializar una nueva operación de I/O sin implicar a la Unidad Central de proceso

Sin esta técnica, luego de la interrupción fin de transferencia de la primer zona se debería volver a generar un nuevo programa canal para recargar los registros DEC y CDEP para la segunda zona

Para lograr que el canal tome el control del encadenamiento se necesita agregar a la info DEC y CDP }, dos informaciones mas de **gobierno** conocidas como IT (interrupción de fin de transferencia, indica que termino ese bloque) y ED (Encadenamiento de datos)

Consiste en efectuar la operación de carga de dichos registros bajo el control del Canal con el objeto de pasar a de una zona de memoria a otra de manera automática, dentro de una única operación de E/S, y sin intervención de la Unidad Central

En el canal automático, para pasar de una zona de memoria a otra, la unidad central debería desarrollar, después de la interrupción de fin de transferencia correspondiente a la primera zona, un programa cuyo objetivo sería recargar los registros DEC y CDP con las características de la segunda zona y relanzar la operación de entrada o de salida. El

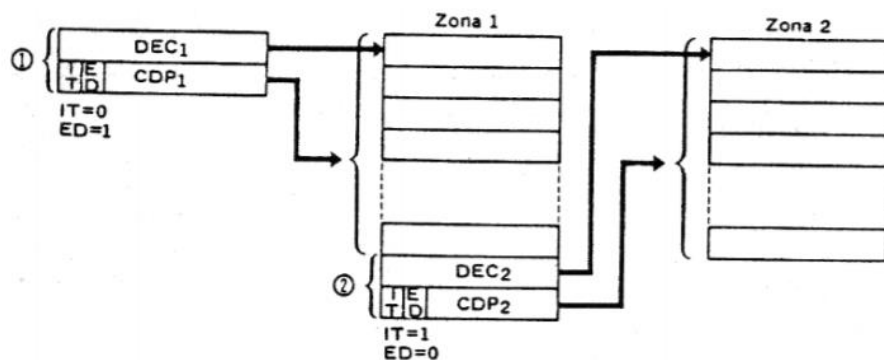
encadenamiento de datos consiste en efectuar esta operación de carga de dichos registros bajo control del canal, de tal suerte que se pase automáticamente de una zona a otra dentro de una operación E/S, sin implicar a la unidad central. Para conseguirlo es necesario añadir a las informaciones la “dirección” y “cuenta de palabras” dos informaciones binarias de gobierno:

IT:	=	0	no interrupción
	=	1	Interrupción
ED:	=	0	no encadenamiento
	=	1	Encadenamiento

Puede suponerse, por ejemplo, que las informaciones de control caben en dos palabras de máquina:

DEC		
IT	ED	CDP

Y que, si hubiera encadenamiento, las nuevas palabras de control las encontrarán al final de zona.



La operación E/S definida por las zonas de la figura se descompone según el siguiente esquema:

al nivel de la unidad central:

- cargar la doble palabra de control 1 en los registros de canal;
- iniciar la transferencia;

al nivel del canal:

- transferir la primera zona;
- cargar las palabras de control 2, puesto que el encadenamiento está pedido en 1;
- transferir la segunda zona;
- generar una interrupción pedida en 2.

Programa Canal

El canal de los computadores evolucionados es una unidad programable, capaz de leer decodificar y ejecutar un **programa de gobierno de entrada-salida** o **programa de canal**, registrado en memoria central, con los mismos derechos que la unidad central lee, decodifica y ejecuta un programa de procesamiento.

Vamos a describir la forma de un programa de canal en una unidad de intercambio compleja.

Con el fin de evitar confusiones, adoptaremos la siguiente terminología:

- la unidad central lee, decodifica, y ejecuta **instrucciones**;
- el canal lee, decodifica y ejecuta **instrucciones de canal**;
- el controlador de periférico recibe, decodifica y ejecuta **códigos de operación periférica**

El programa de canal permite a la vez el encadenamiento de datos y el de instrucciones de canal. El encadenamiento de instrucciones de canal logra el encadenamiento automático de operaciones E/S de diferentes registros (ej., lectura de varios registros de una misma cinta magnética).

Descripción de una operación de E/S:

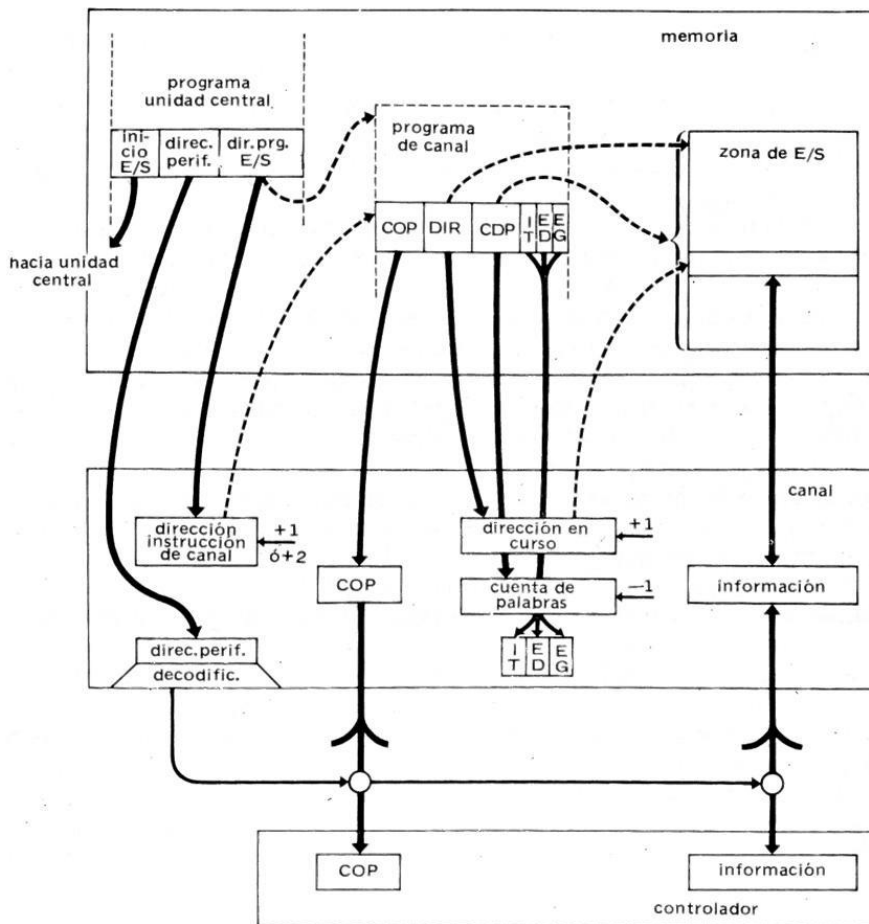


Fig. 21. El programa de canal

1. En el curso del programa de unidad central, se genera en memoria una lista de instrucciones de canal. Es el *programa de gobierno de entrada-salida*, también llamado **programa de canal**.
2. La unidad central ejecuta una instrucción de *inicio de entrada-salida* que proporciona al canal la dirección del órgano periférico interesado y la de la primera instrucción del programa canal.
Si el periférico designado está libre, se pone en actividad, si no, la unidad central es avisada de que el programa de canal no debe arrancar. La dirección de la primera instrucción de canal es enviada y almacenada en el contador de instrucciones del canal. A partir de ese instante, el programa de canal se desarrollará sin intervención de la unidad central.
3. El canal va a buscar en memoria la primera instrucción de canal, que consta de los siguientes elementos:

C.O. PERIF.	DIRECCIÓN 1ra. PALABRA	CUENTA DE PALABRAS	INDICADORES
-------------	------------------------	--------------------	-------------

El canal transmite al controlador del periférico el código de operación que le es destinado. En este momento queda inicializada y un proceso idéntico se desarrolla cada vez que el canal pide una instrucción de canal.

4. Las demandas de transferencia elemental de información, una vez inicializada la operación E/S, emanan del controlador al periférico, que trabaja a su propio ritmo. Al recibir una de estas demandas, el canal efectuará la transferencia pedida utilizando y actualizando las informaciones DEC y CDP.
5. Una transferencia se terminará, bien cuando sea nulo el contenido del CDP, bien al recibirse una señal de fin de transferencia procedente del controlador del periférico. El canal comprobará entonces los indicadores, en número de tres, para saber lo que ha de hacer:
 - ★ **Encadenamiento de datos:** si este indicador está puesto, el canal pasará a la siguiente instrucción de canal, ignorando el nuevo código de operación periférica.
 - ★ **Encadenamiento de gobierno:** si el indicador está puesto, el canal pasará a la siguiente instrucción de canal, utilizando el nuevo C.O.P.
 - ★ **Interrupción:** se generará una instrucción al final de la transferencia, si el indicador correspondiente está activado.

El programa de canal está controlado por un cierto número de instrucciones E/S, entre las cuales he aquí las más importantes: **inicio de entrada-salida**, **detención de entrada-salida**, **prueba de entrada-salida**. Las dos primeras instrucciones permiten inicializar o detener una operación E/S, señalando el periférico y la dirección de la primera instrucción de canal. La tercera se emplea para obtener informaciones acerca del estado actual de una operación de E/S, sin alterar el funcionamiento de ésta.

Las informaciones de estado, devueltas por el canal direccionado, comprenderán la dirección de la instrucción canal en curso y el contenido del contador de palabras (excepto para la instrucción de entrada-salida), así como ciertos indicadores, en particular:

- *interrupción solicitada y no acordada;*
- *estado del órgano periférico direccionado:* dispuesto, no operativo, ocupado;
- *estado del controlador de periférico:* dispuesto, no operativo, ocupado;
- *órgano periférico en espera* de una nueva operación;
- *error de paridad* producido en el intercambio con la memoria;
- *error de paridad* en la transferencia de datos al exterior.

- ★ **Prueba de órgano periférico:** permite obtener informaciones, más precisas que con la instrucción de prueba de E/S, acerca del estado de un órgano periférico. Tales informaciones, peculiares de los distintos periféricos, son suministradas por el controlador del periférico.
- ★ **Identificación de interrupción:** esta instrucción permite, tras recibirse una interrupción emitida por el canal, identificar al órgano provocador de la interrupción y conocer la causa de ella. La información de estado devuelta por el canal comprende, por una parte, la dirección del periférico causante de la interrupción, por otra parte, indicadores concernientes a la operación que ha dado lugar a la interrupción, en particular:
 - *desbordamiento de datos* (en el curso de una operación de entrada);
 - *longitud incorrecta*, si no existiera `ceeeeeeeeeeeeeeeeeeeeeeeeeee3+}` concordancia entre cuenta de palabras detectada por el canal y final de transferencia enviada por el periférico;
 - *contador de palabras nulo*;
 - *final de transferencia*;
 - *error de paridad* en el curso de la transferencia, etc.

Canal Multiplado en el tiempo

Básicamente, el canal multiplado en el tiempo es capaz de gestionar simultáneamente varias transferencias entre memoria y órganos periféricos. Cada una de estas transferencias es controlada por un programa de canal, como el descrito en el párrafo anterior. Por consiguiente existirá, asociado a cada controlador de periférico conectado al canal, un **subcanal**, para conservar las informaciones que debe utilizar el canal en el curso de la ejecución y de la finalización de una instrucción de canal.

IMAGEN

Entre estas informaciones, se encuentran: la dirección de la instrucción de canal en la memoria; la dirección en curso y la cuenta de palabras de la zona de entrada-salida; los indicadores de encadenamientos e interrupciones. Al transferir una información entre la memoria central y uno de los controladores de periféricos, son transmitidas las informaciones de control al canal por el subcanal asociado, y devueltas después de utilizadas (dirección en curso incrementada, contador de palabras decrementado). Así, todo sucede como si los circuitos del canal fueran comunes al conjunto de los subcanales, y éstos se redujeran a una memoria almacenando las informaciones específicas de una operación (fig. 22). Las transferencias de información están imbricadas en el tiempo, de tal suerte que un controlador no se liga al canal más que el tiempo necesario a la transferencia efectiva de una información (dirección, orden, dato o estado). El diálogo entre el canal y los controladores de periféricos se realiza a través de un bus y de un hilo de prioridad (fig. 23).

IMÁGEN

Se distinguen dos clases de diálogos entre canal y controladores de periféricos

- (1) Los diálogos inicializados por el canal para lanzar una operación de entrada-salida, o para comprobar el estado de una unidad periférica (en general, como consecuencia de una instrucción de entrada salida).
- (2) Los diálogos inicializados por un controlador de periférico. Generalmente se trata de solicitud de ejecución de una transferencia elemental de entrada o de salida, cuando la unidad periférica está dispuesta. Como pueden producirse varias solicitudes simultáneas, una prioridad cableada establecerá la selección de la solicitud a satisfacer.

Diálogo inicializado por el canal. El canal pone en los hilos de información del bus la dirección del periférico y valida esta dirección por una señal de selección SEL. Los controladores de periférico detectan la señal SEL, comparan la información del bus con su propia dirección y se muestran transparentes si ambas no coinciden. Cuando sí coinciden, el

controlador se reconoce y no transmite la información, sino que se conecta. Generalmente pone en los hilos de información su "estado" y lo acompaña por una señal de validación ESTD.

IMÁGEN

Diálogo inicializado por un controlador de periférico. Uno o varios controladores emiten sobre la línea ómnibus una señal de demanda de servicio DES. La selección del controlador de mayor prioridad, entre los que han generado una demanda, se hace por intermedio de la línea de prioridad que, saliendo del canal, se comunica con los distintos controladores según su orden de prioridad. En respuesta a la DES, el canal envía una señal PRI por medio de la línea de prioridad. Los controladores se comportarán transparentemente a la señal PRI, excepto han emitido una DES. El primer controlador atravesado que haya emitido una DES, no transmite la señal PRI, pondrá su dirección sobre los hilos de información del bus y la acompañará por una señal de validez DIR. El canal aprobará la dirección por una señal ADIR y el periférico será conectado. El canal cargará en sus registros las informaciones de intercambio específicas de dicho periférico (DEC y CDP). El controlador del mismo enviará una demanda de transferencia DT, acompañada por una señal indicando si se trata de una entrada o de una salida. En el primer caso, pondrá simultáneamente la información sobre el bus, en el segundo, esperará a la información procedente del canal. En general, es posible efectuar sucesivamente varias transferencias elementales del mismo sentido si el controlador repone cada vez la señal DT. A cada transferencia, el canal usa el contenido del registro DEC para la selección en la memoria, lo incrementa, decrementa el contenido del CDP y comprueba que éste no ha llegado a cero. El servicio del canal concluye cuando el controlador activa la señal FDES (fin de demanda de servicio). El canal reactualiza a partir de sus registros, las informaciones en el subcanal del órgano periférico direccionado. Si a continuación aún detecta el canal la señal DES, esto quiere decir que otro controlador tiene presentada igualmente una demanda de servicio, en cuyo caso el canal realiza una señal PRI a través del hilo de prioridad, etc.

IMÁGEN

Arquitecturas de computadoras

Instrucciones simples: Luego de ser decodificadas se ejecutan en un solo pulso de reloj. : por ejemplo suma o resta de números almacenados en registros de la CPU.

Instrucciones complejas: Normalmente microprogramadas, como movimientos de cadenas de caracteres de longitud extensa y variable en procesamiento de textos. Como primer resultado, estas instrucciones, tienen un puntero al microprograma de dicha instrucción compleja, requieren para su ejecución un numero de pulsos que dependerá de la secuencia de pasos de dicho microprograma. Cada paso ejecuta un microcodigo (de la memoria central del procesador) el cual activa los circuitos que intervienen en dicho paso.

CISC – RISC

CISC (Complex Instruction Set Computer)

El diseño de esta arquitectura está marcado por la tecnología existente en los años 60. Es un sistema orientado a ventanas, lo cual debe ser soportado por su sistema operativo, es decir que CISC también estuvo condicionado por su SO.

Su principal característica es el uso de un secuenciador microprogramado, donde cada instrucción tiene un microprograma almacenado en una memoria ROM interna llamada memoria de control.

La mayor desventaja es que las instrucciones consumen muchos ciclos de CPU, las operaciones complejas pierden tiempo en la búsqueda de los punteros para comenzar su ejecución, mientras que las simples no tienen este tiempo agregado. También que al ser tan grande y complejo el set, se dificultan hacer modificaciones y mejoras.

En la practica lograr elevadas velocidades de ejecución se torna difícil, requiere de chips con superficie elevada de silicio, y el repertorio de instrucciones dificultan la generación de códigos óptimos por parte de los compiladores

Características:

- ★ **La cantidad de instrucciones en lenguaje de maquina es lo máximo posible**

CISC posee un repertorio muy amplio de instrucciones muy complejas y potentes. Permite realizar en la arquitectura más de una operación, contempla por ejemplo multiplicación, división.

Un set de instrucciones CISC tiene alrededor de 210 instrucciones. Estas ocupan distinto número de bytes en memoria, por lo tanto, las instrucciones tienen un “**formato variable**”. Como sus instrucciones ocupan distinto número de pulsos de reloj no puede usarse la técnica del “pipeline” (explicada después).

- ★ **Las instrucciones llevan varios ciclos de instrucción**

- ★ **Tantos modos de direccionamiento y formatos de instrucción como sea posible**

Puede tener más de una docena de modos de direccionamiento. En CISC, pasa que el set de instrucciones admite múltiples modos de obtención del dato (direccionamiento) y, por lo tanto, un mismo verbo (por ejemplo, ADD) **tendrá en el set tantas instrucciones distintas como modos de direccionamiento acepte.**

- ★ **El juego de instrucciones es de longitud variable, logrando reducir la longitud de los programas.**

- ★ **Utiliza un secuenciador microprogramado, cada instrucción tiene un microprograma**

Necesariamente debe tener una **Memoria de Control con los microcódigos para ejecutar las instrucciones complejas**. Cada microprograma será almacenado en la memoria de control que es una ROM interna, al ejecutar estos programas se irán emitiendo las microórdenes correspondientes para materializar la instrucción en curso.

Depende del microcódigo para hacerle fente a la complejidad, a veces teniendo que relantizar los ciclos de reloj para tener suficiente tiempo para completar las instrucciones

¿Por qué utiliza secuenciador microprogramado? Al tener instrucciones muy complejas cablearlas incluirá una lógica muy compleja y costosa que además ocuparía mucho espacio físico. Es la base de la arquitectura CISC

- ★ **Sus ciclos de reloj son mucho más largos y extensos debido a la cantidad de instrucciones**
- ★ **Varias instrucciones pueden acceder a memoria que también permiten que accesos indirectos a memoria se hagan de una sola vez.**
- ★ **Posee registros de propósito específico**

En CISC se tiene **pocos registros internos**, lo que implica tener instrucciones para guardar en memoria el contenido de registros a fin de usarlos en otra cosa, y otras para restaurar esos contenidos. Esto a su vez lleva a un mayor tiempo de ejecución. Pero si cuenta con varios registros de propósito específico como por ejemplo índice o base, que den soporte a los modos de direccionamiento

- ★ **Soporta el acceso desalineado a memoria permitiendo utilizar al máximo su espacio.**
- ★ **Posee instrucciones muy completas que permiten facilitar la programación en el ensamblador.**
- ★ **Ayuda en la ejecución de lenguajes de alto nivel, al mejorar las instrucciones del ensamblador.**

Son procesadores que pueden ejecutar tanto instrucciones simples como instrucciones complejas.

Se busca lograr el llamado “salto semántico”, es decir, lograr que las sentencias en lenguajes de alto nivel y lenguaje de máquina sean fácilmente equiparables, haciendo así los lenguajes de alto nivel más fáciles de producir y los tiempos de compilación más reducidos. *Cada máquina tiene su propio lenguaje assembler.*

Disminuye el semantic gap entre lenguaje de máquina y lenguaje de alto nivel

Considera que la compilación de programas de alto nivel requiere muchas menos instrucciones de máquina y con tiempos más cortos. Entonces Muchas de las instrucciones que normalmente serían de alto nivel, en CISC están implementadas a nivel de HW.

- ★ **Dificultad para implementar el paralelismo entre instrucciones**

RISC (Reduced Instruction Set Computer)

Repertorio de instrucciones simples y lo más reducido posible, que se completan la mayoría en un ciclo de reloj. La gran disputa entre partidarios de cada arquitectura era que una arquitectura CISC, al tener un set tan grande y complejo, monopolizaba la arquitectura cada vez que debía realizar una instrucción, pero esto no pasa en RISC al ocupar sus instrucciones muy pocos ciclos de CPU. Pero entonces requerirá para la ejecución de una tarea muchas más instrucciones de bajo nivel para realizar la misma tarea que en CISC. Pero por esta RISC presenta mejoras **permitiendo el paralelismo en la ejecución de instrucciones**, mientras está realizando un cálculo ya puede ir ejecutando la siguiente instrucción, gracias a que no monopoliza muchos ciclos de CPU, esto es lo que se conoce como cauce segmentado o *pipeline*.

Posee menos transistores en el chip por lo cual es más económico que CISC, surge a través de observaciones de programación en máquinas CISC, lo cual condujo a esta nueva arquitectura barata capaz de minimizar ciclos de reloj. En este caso el control microprogramado, fue reemplazado por un **secuenciador cableado**, para hacer más fácil la canalización de instrucciones, pero volviendo así más difícil de tratar la complejidad del hardware. Esto además resulta en que si se quiere añadir una nueva instrucción no debe ser tan grande y no debe atrasar el ciclo de ejecución.

Su repertorio es utilizado convenientemente por compiladores optimizadores, estos analizan el código y ve posibilidad de reducirlo en función a no desperdiciar ciclos de CPU, genera código dependiendo de las instrucciones complejas que deba realizar. Cuando el compilador optimizador tiene una relación directa con las instrucciones puede fraccionar el set de instrucciones y al ser el de RISC muy pequeño tiene la posibilidad de mezclarlas convenientemente.

Procesa de 2 a 4 veces más rápido que CISC.

Permite una ejecución más rápida de las instrucciones, pero requiere compiladores más complejos.

La filosofía RISC establece una serie de principios de diseño que toda máquina debe tener:

★ **Una instrucción por ciclo, pocas instrucciones, simples = UC cableada:**

La falta de complejidad en la operación que realizan las instrucciones de los procesadores RISC permite que se ejecute una instrucción máquina cada ciclo máquina.

Pero se necesitan muchas más instrucciones para realizar la misma tarea que una instrucción CISC.

Hay que evaluar bien antes de agregar una instrucción, si no es imprescindible y no serán usadas frecuentemente no se agrega, deberán optimizar la arquitectura y no ralentizar el procesamiento de las demás instrucciones. Precisa que la RAM este cerca, como en el caso de celulares que la memoria está integrada.

★ **Utiliza un secuenciador cableado**

Debido a la sencillez de la arquitectura, e implementación de operaciones por hardware

Con un repertorio de pocas instrucciones, sencillas y de un ciclo, hay poca o ninguna necesidad de microcódigo, a diferencia de las máquinas CISC que maneja la complejidad de sus instrucciones máquina con una memoria de control de microprograma.

Esto posibilita el uso de la unidad cableada permitiendo que las instrucciones se ejecuten de manera más rápida que en una CISC ya que no se requiere del tiempo que le tomaba acceder a la memoria de control.

La ausencia de ROM de control permite aprovechar ese espacio en el procesador para otras funciones.

★ **Operaciones registro a registro, solo instrucciones load/store acceden a memoria.**

Las operaciones aritméticas son registro a registro, es decir sobre datos contenidos en registros internos, lo que permite que estas instrucciones tardan menos en ejecutarse al no tener que acceder a memoria para buscar los operandos. **Las únicas excepciones son las instrucciones LOAD y STORE**, a parte separadas, evitando así monopolizar la CPU con la LEC o ESC. para cargar un registro desde memoria y guardar en memoria el contenido de un registro, respectivamente. Esta forma de diseño simplifica el repertorio de instrucciones, y por tanto también la unidad de control.

En RISC la mayoría de las instrucciones opera sólo sobre datos en registros internos, sin tener que direccionar memoria, por lo que necesita solo 1 o 2 instrucciones para el mismo verbo.

★ **Muchos registros (*windowed o no-windowed*).**

Para soportar las operaciones registro a registro, se necesita un amplio banco de registros, de manera que los operandos accedidos frecuentemente puedan permanecer en registros por largos periodos de tiempo, sin necesidad de acceder a memoria. Las arquitecturas RISC tienen más registros que las CISC, en general un mínimo de 32 registros.

★ **Modos de direccionamiento sencillos.**

Los modos de direccionamiento complejos y que requieren varios accesos a memoria (como en CISC) tardan mucho en ejecutarse, atrasan la cadena de instrucciones y agrandan el set de instrucciones. Una máquina RISC tiene como máximo 4 o 5 modos de direccionamiento. Esto simplifica el repertorio de instrucciones (necesitamos 1 o 2 ADD en lugar de 25) y por lo tanto la unidad de control.

★ **Formato uniforme y sencillo de instrucción. Decodificación de instrucciones *hardwired***

Generalmente se usa un formato o unos pocos, la longitud de las instrucciones es fija y la misma para todas, las posiciones de los campos son fijas.

Todo lo anterior tiene varias ventajas: con campos fijos, la decodificación del código de operación y el acceso a los operandos en registros puede hacerse simultáneamente. Los formatos sencillos simplifican el decodificador y la unidad de control. Se optimiza la captación de instrucciones ya que se captan unidades de una palabra de longitud (no se tiene instrucciones de más de una palabra).

★ **Compiladores optimizadores**

Al planificar los recursos del procesador, solo se incluye uno si este está justificado por su frecuencia de uso y su inclusión no ententece a otros recursos más utilizados. La complejidad de esta arquitectura está en lograr este compilador optimizado

★ **Ejecución en conductos (pipeline).** Las instrucciones simples, de formato fijo y ciclo único

permiten que las diferentes etapas de los ciclos de ejecución para instrucciones múltiples se puedan realizar simultáneamente, de un modo más simple y eficaz. Esta característica es fundamental de RISC y permite lograr velocidades elevadas de ejecución.

★ **Arquitectura no destructiva de tres direcciones.** Los procesadores CISC destruyen la información que existe en alguno de los registros, como consecuencia de la ejecución normal de instrucciones; esto es debido a su arquitectura de dos direcciones, por la cual el resultado de una operación sobrescribe uno de los registros que contenía a los operandos.

Por contra, las instrucciones RISC, con tres direcciones, contienen los campos de los dos operandos y de su resultado. Por lo tanto, los operandos origen como el destino, son mantenidos en los registros tras haber sido completada la operación. Esta arquitectura "no destructiva" permite reutilizar los operandos optimizando la concurrencia.

Presenta ventajas relacionadas con la tecnología de fabricación VLSI (Very Long Scale Integration), ya que necesitan menos área para control y procesamiento de secuencia de instrucciones (mayor espacio para registros y cache).

★ **Soporte de lenguajes de alto nivel**

Debido al haber pocas instrucciones sobraría lugar en el chip para colocar elementos que den soporte a los lenguajes de alto nivel.

Comparación entre CISC y RISC

RISC	CISC
Énfasis en el SW (sencillez y rapidez en el pipeline)	Énfasis en el HW (velocidad<)
Unas cuantas instrucciones simples	Muchas instrucciones complejas
Instrucciones más chicas de longitud fija con tiempos de ejecución menores	Instrucciones más grandes de longitud variable con grandes tiempos de ejecución multi-reloj

Ciclos de reloj reducido	Ciclos de reloj más extensos
Secuenciador cableado	Secuenciador micro programado
Complejidad en el compilador	Complejidad en el microcódigo
Es más flexible y se pueden realizar cambios	Difícil de modificar
Acceso a memoria solo con instrucciones <i>load/store</i> . <i>Carga y almacenamiento son instrucciones independientes</i>	Muchas instrucciones pueden acceder a memoria. Carga y almacenamiento incorporados en la misma instrucción
Muchos registros	Pocos registros para almacenar los resultados
Bajo costo de producción	Alto costo de producción
Pocos modos de direccionamiento	Muchos Modos de direccionamiento

El hecho de tener actualmente mecanismos rápidos de acceso a memoria, buses de alta velocidad y compiladores especializados, hace que los ordenadores **RISC** obtengan mejores rendimientos.

Cualquier tarea necesita más instrucciones en **RISC** que en **CISC**, ya que en **RISC** las instrucciones son más elementales.

Posiblemente en breve los microprocesadores **RISC** sustituirán a los **CISC**. Pero los microprocesadores **CISC** tienen un mercado de software muy difundido, esto limitará la sustitución en lo inmediato

Con tecnologías de semiconductores comparables e igual frecuencia de reloj, un procesador **RISC** típico necesita un tiempo de procesamiento dos a cuatro veces mayor que la de un **CISC**, pero su estructura de hardware es tan simple, que se puede realizar en una fracción de la superficie ocupada por el circuito integrado de un procesador **CISC**.

Actualmente, se consideran las ventajas de ambas arquitecturas y por tal motivo la convergencia RISC-CISC es cada vez mas vista en los nuevos procesadores. Los procesadores CISC incluyen nuevas técnicas de Pipeline para acelerar el número de instrucciones por segundo. Los procesadores RISC incluye cada vez mas instrucciones para acelerar aún mas lo programas en HW.

La mayoría de los sistemas CISC de alto rendimiento implementan un sistema que convierte dichas instrucciones complejas en varias instrucciones simples del tipo RISC, llamadas generalmente microinstrucciones. Algunos ejemplos de ellos son: Motorola 68000, Ziglog Z80 y toda la familia Intel x86 usada en la mayoría de las computadoras personales. En la actualidad CISC debido a su complejidad, es altamente usado en ordenadores de escritorio, va de la mano con procesadores de la familia Intel x86 y AMD x86-64.

RISC ha evolucionado a la arquitectura ARM (Advanced RISC Machine). ARM debido a su bajo nivel de componentes, sencillez de su set de instrucciones, y bajo consumo, es ideal para dispositivos móviles. APPLE COMPUTERS esta trabajando para implementar ARM en su nueva línea de computadoras, abandonando a la familia de procesadores Intel.

Características	CISC	RISC
Memoria de control	Si	No
Set de instrucciones	Amplio	Reducido
Ciclos de reloj por instrucción	Indefinido	1 o 2
Acceso a MP mediante micro-instrucciones	Si	No, utiliza registros auxiliares (salvo en LOAD y STORE)
Pipe-line	No o ineficiente	Eficiente
Compiladores	Simple	Complejos
Formato de instrucción	Variable	Fijo
Implementación de paralelismo en la ejecución de instrucciones	Compleja	Simple

Velocidad de procesamiento	Reducida por los accesos a la memoria de control	Optimizada por el paralelismo de ejecución
Permite modularidad y crecimiento del set de instrucciones	Si	No
Predicción de bifurcaciones	No	Si

Arquitectura Von Neumann vs Arquitectura Harvard

Von Neumann:

la CPU se conecta con una memoria única, donde coexisten datos e instrucciones, a través de un mismo sistema de buses.

Esto tiene 2 desventajas:

- ★ El tamaño de los datos e instrucciones está fijado por el ancho del bus que comunica la memoria con la CPU. Así, con un bus de 8 bits, se tendrán que manejar datos e instrucciones de una o más unidades de 8 bits (1 byte) de longitud. Si quisiéramos acceder a una instrucción o dato de más de un byte de longitud, tendríamos que realizar más de un acceso a la memoria.
- ★ El procesador es más lento en su respuesta, ya que no puede buscar en memoria una nueva instrucción mientras no finalicen las transferencias de datos de la instrucción anterior, lo cual impide superponer ambos tiempos de acceso. O sea, la CPU puede estar bien leyendo una instrucción o leyendo/escribiendo datos desde/hacia la memoria, pero ambos procesos no pueden ocurrir al mismo tiempo, ya que las instrucciones y datos usan el mismo sistema de buses.



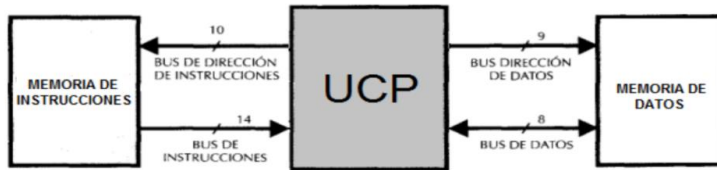
Harvard:

Es una arquitectura del tipo **RISC**.

Utiliza **Pipeline**, la memoria de instrucciones y la memoria de datos son independientes y cada una dispone de su propio sistema de buses para el acceso.

Este aislamiento y diferenciación tiene las siguientes ventajas:

- ★ Al tener buses separados y de diferente tamaño, la longitud de los datos y las instrucciones puede ser distinta, al igual que el tamaño de cada memoria, debido a la diferencia de tamaño de los buses de direcciones. Entonces, al no estar relacionados el tamaño de uno y otro, las instrucciones pueden optimizarse para ocupar una sola posición de memoria de programa, evitando acceder más veces a la memoria y logrando así mayor velocidad y menor longitud de programa.
- ★ Se puede acceder en forma simultánea e independiente a la memoria de datos y a la memoria de instrucciones, por lo que podemos completar la ejecución de una instrucción (memoria de datos), y al mismo tiempo leer la siguiente instrucción a ejecutar (memoria de programa), logrando mayor velocidad.



Pipe-Line

Básicamente, diremos que una unidad es del tipo pipe-line cuando le podemos exigir que se encargue de una nueva operación cada θ nanosegundos, mientras que, de hecho, cada operación dura $n\theta$ nanosegundos. Por tanto, en el plano de los rendimientos globales, las cosas suceden como si la unidad ejecutase una operación cada θ nanosegundos.

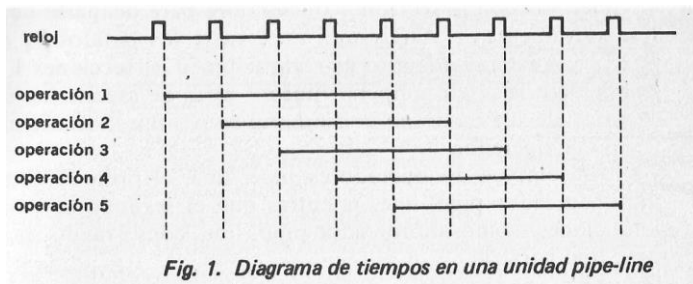
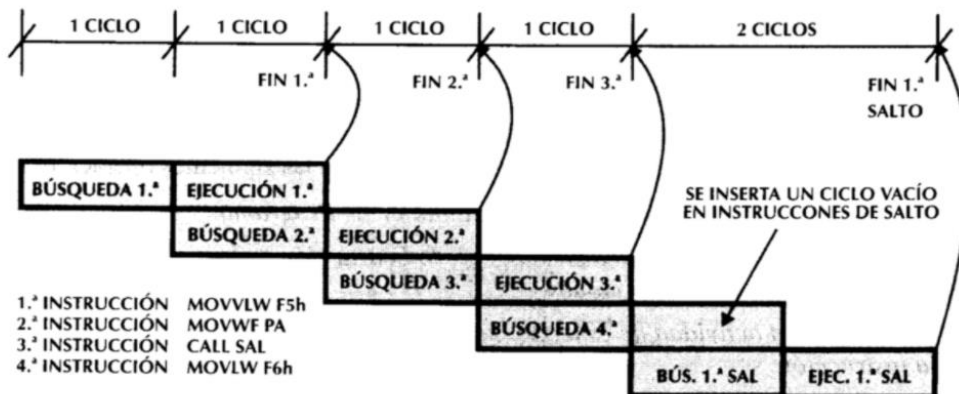


Fig. 1. Diagrama de tiempos en una unidad pipe-line

Esta organización favorece a la *segmentación o "pipeline"*, que aprovecha la separación de las memorias de la arquitectura Harvard, pudiendo **ejecutar una instrucción (lo que implica acceder a la memoria de datos) mientras se busca la siguiente (en la memoria de programa)**.

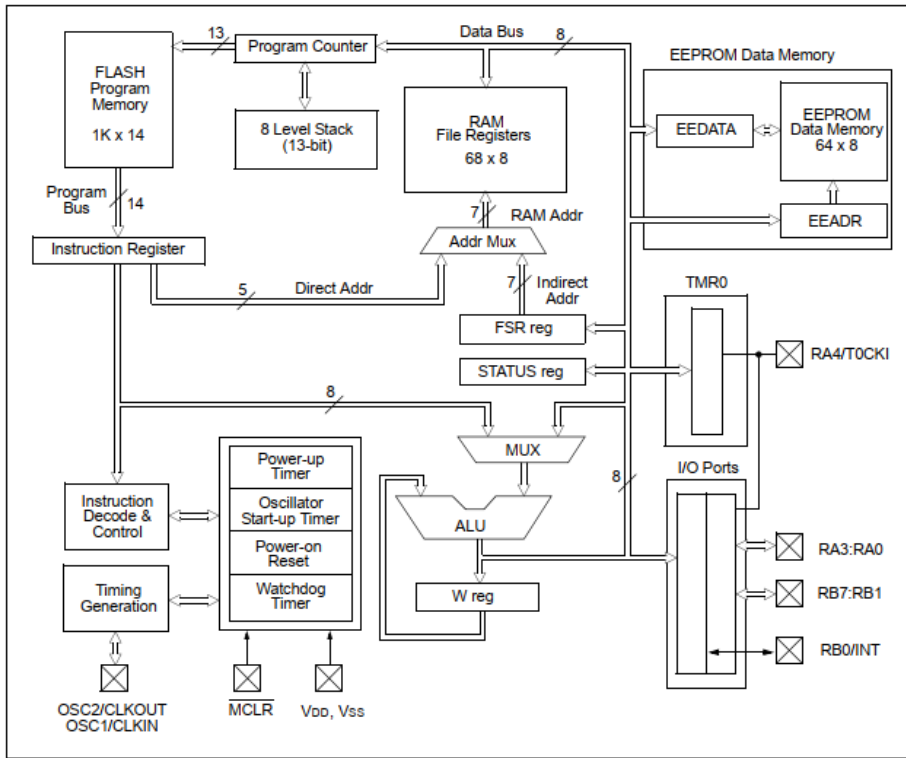
Esto permite solapar ambas fases, permitiendo que cada instrucción se ejecute de manera efectiva en un ciclo (excepto por las de salto que tardan dos), aumentando así el número de instrucciones ejecutadas simultáneamente en un determinado periodo de tiempo.



Si bien la duración de cada una no se reduce, si se reduce la duración del conjunto de ellas.

Esto último es a su vez, una característica importante de los procesadores RISC (una instrucción por ciclo), por lo que dichos procesadores hacen uso de arquitectura Harvard. Por ejemplo, los PIC16C84 que usamos en la práctica usan esta organización.

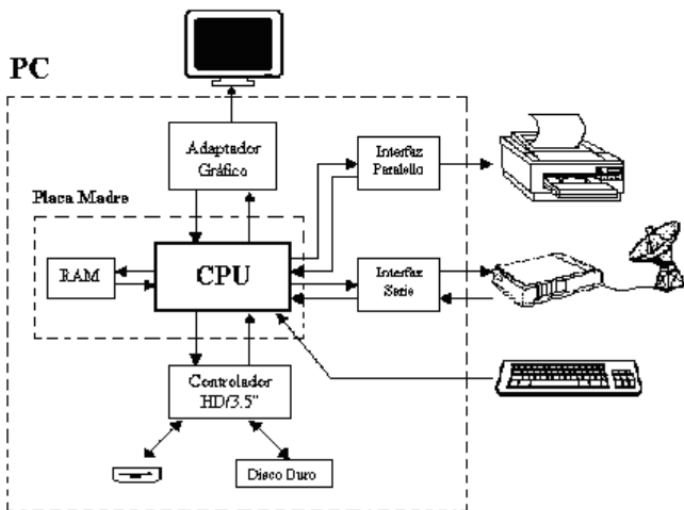
Diagrama de bloques de un PIC16F84A



Lenguaje de Maquina

PERIFÉRICOS

Se denominan periféricos tanto a las unidades o dispositivos a través de los cuales el ordenador se comunica con el mundo exterior, como a los sistemas que almacenan o archivan la información, sirviendo de memoria auxiliar de la memoria principal. Se entenderá por periférico a todo conjunto de dispositivos que, sin pertenecer al núcleo fundamental de la CPU-Memoria Central, permitan realizar operaciones de E/S, complementarias al proceso de datos que realiza la CPU.



El procesador solo se comunica con la memoria principal, y las características de esta son:

- ★ Su reducida capacidad para almacenar datos.
- ★ Su volatilidad, pues al desconectar el ordenador de la red eléctrica se borra su contenido.

Atendiendo a la definición dada, los periféricos se pueden clasificar en tres tipos: en unidades de entrada, unidades de salida y unidades de memoria masiva.

Y aunque la memoria puede considerarse también como una unidad de entrada / salida, hay una característica que la diferencia del resto de unidades: la información almacenada no es directamente entendible por el usuario, pues suele estar codificada, comprimida e incluso a veces encriptada por motivos de seguridad. Esto hace que la interacción con el usuario no sea directa como en el resto de las unidades.

Los dispositivos de E/S transforman la información externa como una pulsación en una tecla en impulsos eléctricos que son codificados enviados hacia el procesador para su interpretación, procesamiento y almacenamiento de forma automática. Estas señales eléctricas se codifican según unos códigos como el ASCII y la CPU recibe la información codificada siempre en binario.

Otros dispositivos como las impresoras realizan el paso contrario, unos valores binarios que codifican un determinado carácter son convertidos en una serie de marcas gráficas sobre un papel a fin de que el usuario lea o vea la información requerida.

Partes de los periféricos

Todos los periféricos suelen tener dos partes claramente diferenciadas, a saber: una parte mecánica y otra parte electrónica.

- ★ La parte **mecánica** está formada por dispositivos electromecánicos (conmutadores manuales, reles, motores, electroimanes, etc., que son controlados por elementos eléctricos.
- ★ La parte **electrónica** se encarga de controlar las ordenes que llegan de la CPU para la recepción o transmisión de datos, y de generar las señales de control para manejar adecuadamente la parte mecánica del periférico. En la

parte electrónica es común usar elementos optoelectrónicos que actúan como detectores o generadores de la información de entrada y salida, respectivamente. También estos elementos se usan como detectores de posición de los elementos mecánicos móviles del periférico. Juegan un papel importante los conversores analógicos/digitales.

Los dispositivos de E/S transforman la información externa en señales codificadas, permitiendo su transmisión, detección, interpretación, procesamiento y almacenamiento de forma automática.

Clasificación de los periféricos

★ Unidades de entrada.

- Teclado.
- Ratón (“mouse”).
- Dispositivos de captura directa de datos:
 - Lectora de banda magnética.
 - Detectores ópticos: de marcas, barras impresas, escáneres, caracteres impresos, cámaras digitales, ...
 - Detectores de caracteres impresos (OCR).
- Unidades de reconocimiento de voz.
- Lápiz óptico.
- Pantallas sensibles al tacto.
- Palanca manual de control (“joy-stick”).
- Digitalizador o tableta gráfica.

★ Unidades de salida.

- Monitores de visualización: tubo de e- y TFT.
- Impresoras.
- Sintetizador de voz.
- Visualizadores (“displays”).
- Registrador gráfico (“plotter”).
- Unidades de memoria masiva auxiliar.
- Discos magnéticos.
- Cintas magnéticas.
- Discos ópticos y magnetoópticos.
- Memoria de silicio (PenDrive).

★ Unidades mixtas.

- Terminal interactivo teclado-pantalla.
- Terminal teletipo.
- Pantalla sensible al tacto.

Hardware → lógica digital → microprogramación → maquina convencional(lenguaje de maquina) → sistema operativo → ensambladores → lenguajes de alto nivel

CONEXIÓN DE PERIFÉRICOS AL COMPUTADOR.

El bus que conecta la CPU con los otros elementos del procesador se conoce como bus local o bus de la CPU. Es un bus muy rápido y conecta la CPU con las tarjetas de la placa base y los controladores de los dispositivos externos.

Las conexiones entre los periféricos y los controladores o tarjetas de la placa base se realizan a través de un bus más general llamado bus del sistema. También suele conectar algunas ampliaciones de memoria.

Algunos periféricos requieren un bus especializado que se adapte a su velocidad de transferencia, sus niveles de tensión, la naturaleza de sus señales de control y otros requerimientos. A estos buses se les llama bus de entrada/salida o bus de expansión.

Por todo lo visto, los computadores grandes al disponer de varios tipos de buses requieren de dispositivos adaptadores o de interconexión entre buses.

Los procesadores suelen tener unas ranuras de expansión (6 normalmente) sobre la placa base que están conectadas al bus del sistema y que permiten conectar una serie de dispositivos a este bus a través de tarjetas de circuito integrado y que permiten conectar varios dispositivos a la CPU, como por ejemplo tarjetas digitalizadoras de imágenes, aceleradores gráficos con FPGAs, etc.

Todos los buses poseen unas especificaciones normalizadas, como son:

- ★ protocolos de transmisión de datos,
- ★ velocidades y temporización de las transferencias,
- ★ anchuras de los sub-buses,
- ★ y sistema físico de conexión (conectores estandarizados).

Los Buses normalizados más conocidos son:

- ★ S-100 Bus (IEEE 696). Puede considerarse como el primer bus normalizado para microcomputadores, siendo introducido por Atari para su computador 8080 (sistema de 8 bits). En total disponía de unos 100 hilos.
- ★ CAMAC (o IEEE 583). El bus CAMAC (“Computer Automated Measurement and Control”) Se introdujo para interconectar instrumentos de medida nucleares en 1969.
- ★ GPIB (o IEEE 488). El GPIB (“General Purpose Interface Bus”) fue ideado por Hewlett Packard (1965 a 1975) usa 24 hilos, 8 de los cuales son para datos y el resto para señales de control.
- ★ Multibus (o IEEE 796). Bus de 16 bit de datos introducido por Intel. En la actualidad hay una versión mejorada llamada Multibus-II (IEEE 1296) para transferir datos de 32 bits.
- ★ ISA Bus. El bus ISA (“Industrial Standard Architecture”), es el bus introducido con el IBM-PC. Tiene 64 hilos de los cuales 8 son para datos.
- ★ ISA AT Bus. Fue introducido con los IBM-AT (80286). Ideado para arquitecturas de 16 bits, posee subdirecciones de 24 bits (direcciona hasta 16 Mbytes) y es compatible, como no, con su antecesor de 16 bits.
- ★ MCA. (“Micro-Channel Architecture”) fue introducido por IBM en 1987 en sus equipos PS/2. Es un bus para arquitecturas de 32 bits y es 10 veces más rápido que el ISA AT, llegando a transferir hasta 20 Mbits/seg.
- ★ EISA (“Extendet Industry Satandard Architecture”). Es un bus ideado por 9 fabricantes de ordenadores, para arquitecturas de 32 bits. Posee velocidad de transferencia de 33Mbits/seg. Es compatible con el bus ISA. Este bus puede solo ser controlado por microprocesadores 80386, 80486 o superiores, y es autoconfigurable.

- ★ SCSI (“Small computer System Interface”) es un estándar universal para conexiones paralelas a periféricos. Suele utilizarse para unidades de discos magnéticos y ópticos. Admite hasta 7 dispositivos y fue ideado para entornos UNIX y Macintosh. Permite velocidades de transferencia de 5 Mbits/seg hasta 400 Mbits/seg. En la actualidad se está desarrollando el SCSI-3, de 32 bits, que podrá admitir hasta 32 periféricos conectados a gran distancia por fibra óptica (comunicación serie).
- ★ Futurebus+ (IEEE 896.1 e IEEE 896.2). Es una normalización proyectada para equipos de muy altas prestaciones, que puede considerarse como una evolución de las normas Multibus II y VME. Diseñado para arquitectura de 64 bits. Permite la construcción de sistemas multiprocesador (de hasta 32 procesadores) compartiendo memoria.
- ★ USB responde a las siglas Universal Serial Bus y hace referencia a un protocolo de conexión que permite enlazar diversos periféricos a un dispositivo electrónico (frecuentemente, un ordenador) para el intercambio de datos, el desarrollo de operaciones y, en algunos casos, la carga de la batería del dispositivo o dispositivos conectados. Es, por tanto, básicamente, un puerto que funciona de toma de conexión entre diferentes aparatos. Este protocolo fue creado en la década de los años 90 por un conjunto de siete grandes empresas de la tecnología (Intel, IBM, Northern Telecom, Compaq, Microsoft, NEC y Digital Equipment Corporation) con la única finalidad de alcanzar un estándar para la conexión de los periféricos a una computadora. Si bien es cierto que el primer modelo puesto en marcha fue el USB 1.0, la cadena evolutiva tecnológica ha llevado a éste hasta el USB 3.1. A continuación, entraremos en detalle en ésta y otras cuestiones.

El USB puede clasificarse en cuatro estándares distintos:

- **USB 1.0** (baja velocidad). Creado a mitad de los 90, la velocidad máxima a la que podía mover la información alcanzaba los 1,5 Mbps, una conexión suficiente para dispositivos como el ratón y el teclado.
- **USB 1.1** (velocidad completa). Incrementó la tasa de transferencia hasta los 12 Mbps.
- **USB 2.0** (alta velocidad). Es el estándar más extendido y transfiere los datos a una velocidad de 480 Mbps, aunque su tasa máxima real es de 280 Mbps. Su cable tiene cuatro líneas: dos para datos y dos para alimentación.
- **USB 3.0** (superalta velocidad). Diez veces más rápido que el estándar anterior, llega a velocidades de 4,8 Gbps. Es compatible con los anteriores e incluye cinco contactos adicionales. Convive junto con el 2.0.
- **USB 3.1** Este estándar surgido hace apenas un año aumenta la velocidad para la transferencia de datos hasta los 10 Gbps. Además, introduce un nuevo tipo de conector (el C), que será explicado en las siguientes líneas.

Comentado [MG3]: Esto lo googlee porque lo del libro era viejo

Hay otros tipos de bus, pensados para estar conectados directamente a la CPU, son los buses locales.

Los buses locales más conocidos son:

- ★ VL-bus, ideado por VESA para varios fabricantes.
- ★ PCI (“Peripheral Component Interconnected”), creado para Intel.
- ★ Quick-ring, propuesto por Apple.
- ★ VME, IEEE 1014 (“Versa Module Europe”) aparece en 1981 como sucesor del Versabus, utilizado por Motorola para la serie 68000. Es el más sencillo y el de menos prestaciones de todos los buses de sistemas, sin embargo su aceptación se debe a que es posible encontrar más de 200 módulos comerciales para este tipo de bus. Sus aplicaciones se centran en los campos de sistemas industriales en tiempo real, aplicaciones militares e

investigación. Es un bus multiproceso, lo que significa que varios procesadores pueden compartirlo y puede llegar a transmitir hasta 40 Mbytes/seg con datos de 32 bits.

- ★ NuBus, o IEEE 1196, de 96 hilos está diseñado para arquitecturas de 32 bits y usa en los Apple Macintosh.
- ★ M-bus y S-Bus, utilizados por las estaciones de SUN.

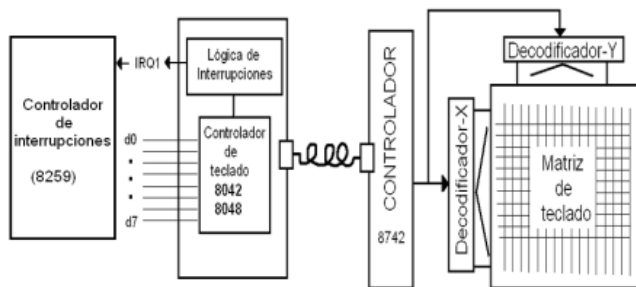
TECLADO

Los teclados son periféricos similares a las máquinas de escribir (Christopher L. Sholes en 1867), con sus teclas dispuestas en una distribución que se conoce como QWERTY, y que se basa en la idea de que las teclas más utilizadas (en inglés) estén lo más separadas posibles entre sí y además la mano izquierda se use en un 60% de las ocasiones (40% la mano derecha, más hábil).

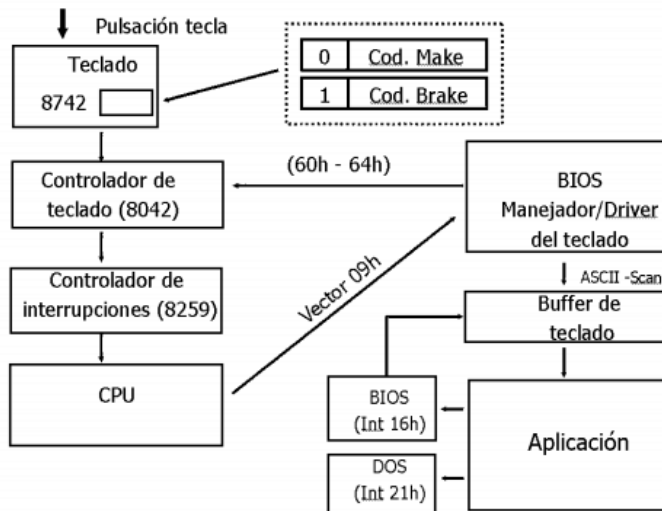
Al pulsar una tecla se cierra un conmutador que hay en el interior del teclado, y mediante el chip 8742 se detecta que tecla se ha pulsado para que posteriormente una lógica circuitos codificadores llamados controladores de teclado (8042) generen el código correspondiente al carácter seleccionado (ASCII, por ejemplo), y envíen una interrupción a la CPU para avisar de que tecla o teclas han sido pulsadas. Normalmente, el programa gestor de teclado hace un “eco” del carácter pulsado visualizándolo en el monitor, pero eso no quiere decir que ese carácter haya sido enviado a la CPU.

Su funcionamiento detallado es como sigue: los teclados poseen internamente una matriz de teclas y un circuito integrado (el 8742) que se encarga de supervisar esta matriz y detectar las pulsaciones. Cuando se detecta una pulsación, o la liberación de una tecla, se escribe su código en un buffer interno del teclado y a continuación el teclado transmite este código al controlador que se encuentra en la placa base del procesador vía serie a través del cable del teclado. Al pulsar se genera una interrupción o llamada que detecta el 8742 y genera el código ASCII de la tecla pulsada (con el bit-7 a “0”) y al soltar se genera el mismo código pero con el bit-7 activado a “1”. Por ejemplo, si se pulsa la 'A' se generará una INT 9 y aparecerá en el puerto del teclado (60h) el byte “1Eh”, al soltar la 'A' se generará otra INT 9 y se podrá leer el byte “9Eh” del puerto del teclado. Este controlador 8742 incluido en el teclado posee las siguientes funciones:

- Prevenir falsas repeticiones (rebotes) o pulsaciones incorrectas.
- Traducir la tecla pulsada en un código único, llamado “scan-code”. Suele ocupar un byte.
- Repetir un carácter si se ha pulsado durante cierto tiempo (del orden de un segundo como mínimo).
- Detectar las pulsaciones simultáneas de varias teclas.

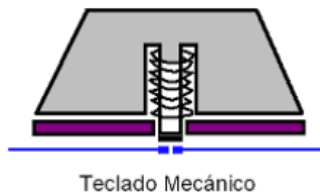


Cuando se produce una pulsación, el controlador transfiere un código al ordenador (“make-code”) que genera una interrupción para que la rutina de tratamiento lea el código enviado. Es el driver de teclado el que en estas interrupciones se encarga de combinar los códigos para generar los caracteres (mayúsculas, minúsculas, teclas de función, control, ...). Las teclas expandidas -las que han sido añadidas al teclado estándar de 83/84 teclas- o las combinaciones de teclas como ALT - teclado_número, tienen un comportamiento especial, ya que pueden generar hasta 4 interrupciones consecutivas.



Tecnologías utilizadas en los teclados:

Los **teclados mecánicos** constan de una serie de teclas con unos interruptores mecánicos colocadas encima de unos muelles, que son los que hacen retornar las teclas a la posición original, de modo que al ser pulsadas éstas hacen contacto con unas terminaciones metálicas del circuito impreso del propio teclado, cerrando así el circuito, y volviendo a abrirlo al dejar de pulsar por el efecto de retorno del muelle. El contacto establecido entre los terminales metálicos de las teclas y el del circuito impreso determina la señal diferenciada.



Los **teclados de membrana** se componen de cuatro capas: la inferior tiene una serie de pistas conductoras impresas; encima de ella, se coloca una capa de separación con agujeros justo debajo de cada una de las teclas; encima de esta se coloca una capa conductora con pequeñas montañas debajo de cada una de las teclas y en cada montaña un conector metálico; encima de éstas se coloca una capa de goma para producir el efecto de retorno a la posición inicial. Cuando pulsamos una tecla, lo que hacemos es poner en contacto las dos capas conductoras (la primera con el circuito y la tercera con los conectores) haciendo que el circuito se cierre, y la membrana de goma hace que se separen las capas al impulsar la tecla hacia su posición inicial (similar al mando a distancia del TV).



RATÓN

La función principal del ratón es transmitir los movimientos de nuestra mano sobre una superficie plana hacia el ordenador. Allí, el software denominado driver se encarga realmente de transformarlo a un movimiento del puntero por la pantalla dependiendo de varios parámetros.

En el momento de activar el ratón, se asocia su posición con la del cursor en la pantalla. Si desplazamos sobre una superficie el ratón, el cursor seguirá dichos movimientos. Es casi imprescindible en aplicaciones dirigidas por menús o entornos gráficos, como por ejemplo Windows.

Ratones Mecánicos

Son sencillos y tienen un bajo coste. Se basan en una bola de silicona que gira en la parte inferior del ratón a medida que desplazábamos éste. Dicha bola hace contacto con dos rodillos, uno perpendicular al ratón y otro transversal, de forma que uno recoge los movimientos de la bola en sentido horizontal y el otro en sentido vertical.

En cada extremo de los ejes donde están situados los rodillos, existe una pequeña rueda conocida como “codificador”, que gira en torno a cada rodillo. Estas ruedas poseen en su superficie, y a modo de radios, una serie de contactos de metal, que a medida que gira la rueda toca con dos pequeñas barras fijas conectadas al circuito integrado en el ratón.

Cada vez que se produce contacto entre el material conductor de la rueda y las barras, se origina una señal eléctrica. Así, el número de señales indicará la cantidad de puntos que han pasado éstas, lo que implica que, a mayor número de señales, mayor distancia habrá recorrido el ratón. Tras convertir el movimiento en señales eléctricas, se enviaban al software del ordenador por medio del cable.



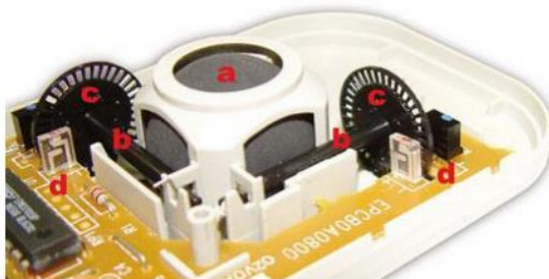
En la figura puede apreciarse una rueda dentada que al girar hacia uno u otro sentido genera una onda cuadrada en el terminal A o en el B.

Los botones son simples interruptores. Debajo de cada uno de ellos se encuentra un micro interruptor que en estado de “reposo” interrumpe un pequeño circuito. En cuanto se ejerce una ligera presión sobre estos, se activa el circuito, dejando pasar una señal eléctrica que será única en caso de que sólo se haga “clic” con el botón, o continua en caso de dejarlo pulsado.

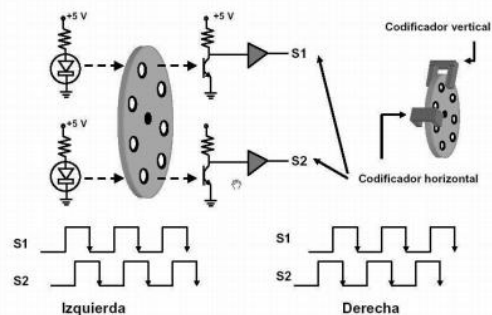
Por último las señales se dan cita en el pequeño chip que gobierna el ratón, y son enviadas al ordenador a través del cable que los une. Allí el controlador del ratón decidirá, en función del desplazamiento vertical y horizontal detectado, el movimiento final que llevará el cursor. También será capaz de aumentar o disminuir ese movimiento, dependiendo de factores como la resolución que se le haya especificado al ratón.

Ratones opto-mecánicos

Trabajan según el mismo principio que los mecánicos, pero aquí los cilindros están conectados a codificadores ópticos que convierten los pulsos luminosos en pulsos eléctricos que son enviados al ordenador. El modo de capturar el movimiento es distinto.



El hardware del ratón de bola



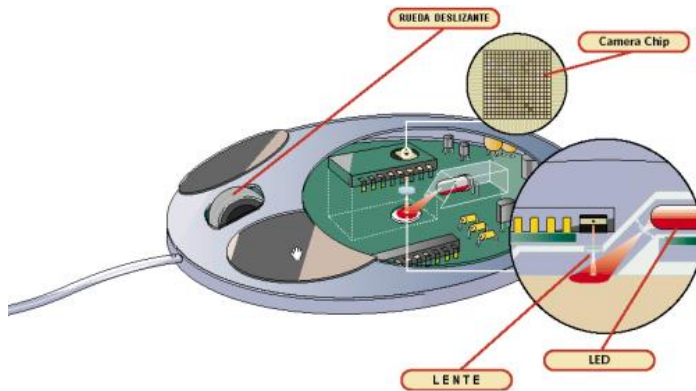
Los tradicionales rodillos que giran una rueda radiada ahora pueden girar una rueda ranurada, de forma que un haz de luz las atraviesa. De esta forma, el corte intermitente del haz de luz por la rueda es recogido en el otro lado por una célula fotoeléctrica que decide hacia dónde gira el ratón y a qué velocidad, dependiendo del desfase en las dos ondas cuadradas que se generan en el interior del ratón.



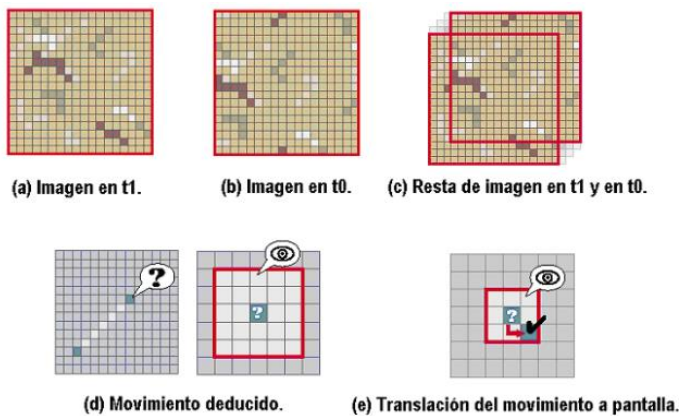
Existen también los llamados ratones de ruedas que sustituyen la bola giratoria por unas ruedas de material plástico, perpendiculares entre sí, dirigiendo así a los codificadores directamente.

Ratones ópticos

Los ratones ópticos carecen de bola y rodillos, y poseen unos fotosensores o sensores ópticos que detectan los cambios en los patrones de la superficie por la que se mueve el ratón. Antiguamente, estos ratones necesitaban una alfombrilla especial, pero actualmente no. La ventaja de estos ratones estriba en su precisión y en la carencia de partes móviles, aunque son lógicamente algo más caros que el resto.



Estos ratones ópticos disponen de un LED que ilumina la superficie sobre la que se mueve el ratón. Una microcámara se encarga de digitalizar con un CCD una pequeña región y luego un integrado compara la imagen producida con la imagen anterior para decidir hacia donde se está desplazando el ratón.

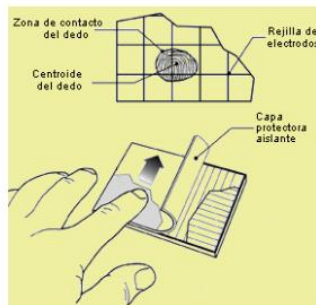


Una característica a tener en cuenta será la resolución, o sensibilidad mínima del sistema de seguimiento: en el momento en que el ratón detecte una variación en su posición, enviará las señales correspondientes al ordenador. La resolución se expresa en puntos por pulgada (ppp). Un ratón de 200 ppp podrá detectar cambios en la posición tan pequeños como $1/200$ de pulgada, y así, por cada pulgada que se mueva el ratón, el cursor se desplazará 200 píxeles en la pantalla. El problema es que la relación entre la sensibilidad del movimiento y el movimiento en pantalla es de 1:1 (un desplazamiento equivalente a la sensibilidad mínima provoca un desplazamiento de un píxel en la pantalla); como consecuencia, cuanto mayor sea la resolución del monitor, mayor será el desplazamiento que habrá que imprimir al ratón para conseguir un desplazamiento equivalente en pantalla. Para solucionar este problema los fabricantes desarrollaron el seguimiento dinámico, que permite variar la relación anterior a $1:N$, donde $N > 1$.

Antes habían dos tipos de conexiones para el ratón: Serie y PS/2. En la práctica no hay ventaja de un tipo de puerto sobre otro. Actualmente se conectan al puerto USB, mayormente.

"Touchpad" capacitivos

Son los utilizados normalmente en los ordenadores portátiles para suplir al ratón. El touchpad está formado por una rejilla de dos capas de tiras de electrodos, una vertical y otra horizontal, separadas por un aislante y conectadas a un sofisticado circuito. El circuito se encarga de medir la capacidad mutua entre cada electrodo vertical y cada electrodo horizontal. Un dedo situado cerca de la intersección de dos electrodos modifica la capacidad mutua entre ellos al modificarse las propiedades dieléctricas de su entorno. El dedo tiene unas propiedades dieléctricas muy diferentes a las del aire.



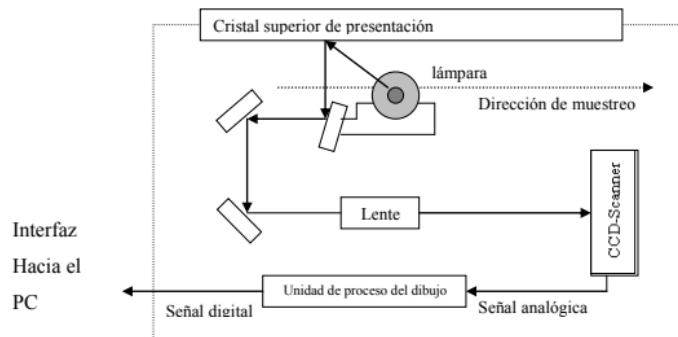
La posición del dedo se calcula con precisión basándose en las variaciones de la capacidad mutua en varios puntos hasta determinar el centroide de la superficie de contacto. La resolución de este sistema es impresionante, hasta 1/40 mm. Además se puede medir también la presión que se hace con el dedo. No se pueden usar lápices u otros materiales no conductores como punteros. Es muy resistente al entorno, soporta perfectamente polvo, humedad, electricidad estática, etc. Además es ligero, fino y puede ser flexible o transparente.

ESCÁNERS

Los escáneres ópticos son unos dispositivos de entrada para el ordenador que utilizan un haz luminoso para detectar los patrones de luz y oscuridad (o los colores) de la superficie del papel, convirtiendo la imagen en señales digitales que se pueden manipular por medio de un software de tratamiento de imágenes o con reconocimiento óptico de caracteres. Transforma las imágenes a formato digital, es decir en series de 0 y de 1, pudiendo entonces ser almacenadas, retocadas o impresas o ser utilizadas para ilustrar un texto. Si el documento que se desea escanear o digitalizar es un texto, por medio de programas de reconocimiento de caracteres, también llamados por las siglas inglesas OCR ("Optical Character Recognition"), es posible reconstituirlo y convertirlo en texto reconocible por el ordenador, pudiendo ser corregido o añadir texto nuevo, es decir, nos evita tener que teclearlo.

Un escáner se compone de dos piezas básicas: la primera de ellas es el cabezal de reconocimiento óptico, la segunda es un simple mecanismo de avance por debajo de un cristal que hace las veces de soporte para los objetos que se van a escanear. En principio, el cabezal de reconocimiento óptico realiza un muestreo del objeto en sí, reconociendo un determinado número de puntos por pulgada y a cada uno de estos puntos le asigna un valor en función del número de bits del proceso: 1 bit sería 1 color (negro o blanco), 2 bits serían 4 colores, 8 bits serían 256 colores y así sucesivamente hasta llegar a los 32 bits (color verdadero).

Los elementos CCD están colocados en una sola fila de forma que a cada elemento le corresponde un píxel de cada una de las filas de puntos que forman la imagen.



Los escáneres en color suelen disponer de 3 fuentes de luz (rojo, verde y azul), y utilizando simultáneamente cada una de ellas, se pueden formar las imágenes en color.

El documento se ilumina línea a línea por una fuente de luz fluorescente o incandescente. La luz reflejada incide a través de una lente sobre un fotosensor denominado CCD ("charge coupled device").